From Molecules to Quantum Circuits

A Practical Guide to Fermion-to-Qubit Encodings

Dr John Azariah

March 2026

Contents

From Molecules to Quantum Circuits **1**
Preface . . . 1

Chapter 1: The Electronic Structure Problem **5**
In This Chapter . . . 5
The Question . . . 5
A Molecule Is a Collection of Charges . . . 6
The Born–Oppenheimer Approximation: Freezing the Nuclei . . . 7
Basis Sets: Making the Infinite Finite . . . 8
Molecular Orbitals for H_2 . . . 9
The Six Configurations of H_2 . . . 10
Second Quantization: A Preview . . . 11
The Numbers: H_2 Integrals in STO-3G . . . 12
Key Takeaways . . . 12
Common Mistakes . . . 13
Exercises . . . 13
Further Reading . . . 13

Chapter 2: The Notation Minefield **15**
In This Chapter . . . 15
The Problem . . . 15
Convention 1: Chemist's Notation (Mulliken) . . . 16
Convention 2: Physicist's Notation (Dirac) . . . 16
The Conversion: One Equation to Rule Them All . . . 17
The Hamiltonian: Which Convention Goes Where? . . . 18
The Symmetries That Save You (and Can Mislead You) . . . 19
The $\frac{1}{2}$ Prefactor . . . 19
A Complete Example: H_2 Two-Body Integrals . . . 20
The Companion Library: FockMap . . . 20
FockMap's Convention . . . 21
Key Takeaways . . . 22
Common Mistakes . . . 22
Exercises . . . 22
Further Reading . . . 22

Chapter 3: From Spatial to Spin-Orbital Integrals **25**

In This Chapter . . . 25
Why We Need Spin-Orbitals . . . 25
Spin-Orbital Indexing . . . 27
One-Body Expansion . . . 27
Two-Body Expansion . . . 28
The Complete Integral Tables . . . 29
From Tables to Code . . . 30
A Preview of What's Coming . . . 32
Key Takeaways . . . 32
Common Mistakes . . . 32
Exercises . . . 33
Further Reading . . . 33

Chapter 4: The Quantum Computer's Vocabulary 35
In This Chapter . . . 35
Two Worlds, One Problem . . . 35
Qubits: The Power of Superposition . . . 36
Operations on a Single Qubit . . . 36
The Hard Part: Entanglement . . . 37
Is This Enough? Universality . . . 39
The Cost Table . . . 39
Key Takeaways . . . 40
Common Mistakes . . . 40
Exercises . . . 40
Further Reading . . . 41

Chapter 5: A Visual Guide to Encodings 43
In This Chapter . . . 43
Where We Stand . . . 43
The Tempting (Wrong) Idea . . . 45
What Makes Electrons Different from Qubits . . . 46
Jordan–Wigner: The Z-Chain . . . 46
Bravyi–Kitaev: Partial Parity Sums . . . 48
Tree Encodings: Going Below $\log_2 n$ . . . 50
The Complete Picture . . . 50
One More Thing: Number Operators Stay Cheap . . . 51
Choosing an Encoding: A Decision Guide . . . 51
Key Takeaways . . . 52
Common Mistakes . . . 52
Exercises . . . 52
Further Reading . . . 53

Chapter 6: Building the Qubit Hamiltonian 55
In This Chapter . . . 55
The Density Matrix: Where Classical Ends and Quantum Begins . . . 55
What We Have and What We'll Do . . . 58
The Recipe . . . 58
The Complete 15-Term Hamiltonian . . . 61

Reading the Hamiltonian: What's Hartree–Fock, What's Quantum 62
Reproducing This with FockMap 62
Key Takeaways 64
Common Mistakes 64
Exercises 64
Further Reading 64

Chapter 7: Five Encodings, One Interface **65**
In This Chapter 65
Why Would We Want a Different Encoding? 65
Do Different Encodings Give the Same Answer? 66
Why Equivalence Holds: The CAR Theorem 67
Five Encodings in Five Lines 68
Where the Differences Emerge: Scaling 68
The Three Frameworks 68
Defining Your Own Encoding 69
A Decision Framework 70
Key Takeaways 70
Common Mistakes 70
Exercises 71
Further Reading 71

Chapter 8: Checking Our Answer **73**
In This Chapter 73
The Verification Problem 73
From Pauli Sum to Matrix 74
The Eigenspectrum of H_2 74
How Good Is This? 75
Cross-Encoding Verification 75
A Verification Checklist 76
Stage 2 Complete 77
Key Takeaways 77
Common Mistakes 78
Exercises 78
Further Reading 78

Chapter 9: Why Tapering? **79**
In This Chapter 79
The Observation 79
What Tapering Gains You 80
The Two Levels of Tapering 81
Why Tapering Is Free 81
Key Takeaways 82
Common Mistakes 82
Exercises 82
Further Reading 83

Chapter 10: Diagonal Z_2 Symmetries **85**

In This Chapter . . . 85
The Detection Algorithm . . . 85
Sectors: Choosing Eigenvalues . . . 87
How Sector Fixing Modifies Terms . . . 87
The Convenience Helper . . . 88
Validation . . . 88
Key Takeaways . . . 89
Sector Selection: A Practical Workflow . . . 89
Common Mistakes . . . 90

Chapter 11: General Clifford Tapering 91
In This Chapter . . . 91
The Limitation of Diagonal Tapering . . . 91
The Binary Pauli Representation . . . 92
Finding All Symmetry Generators . . . 93
Clifford Rotation: Making Generators Diagonal . . . 93
Applying the Clifford to the Hamiltonian . . . 94
The Unified Pipeline . . . 95
Worked Example: Heisenberg Model . . . 95
Key Takeaways . . . 97
Further Reading . . . 97

Chapter 12: Tapering Benchmarks 99
In This Chapter . . . 99
Methodology . . . 99
Benchmark Results . . . 99
The Impact on Circuit Cost . . . 101
Tapering + Encoding: Compounding Savings . . . 101
Stage 3 Complete . . . 102
Key Takeaways . . . 102

Chapter 13: From Hamiltonian to Time Evolution 103
In This Chapter . . . 103
What We Have, and What We Need . . . 103
Energy and Time: Two Sides of the Same Operator . . . 104
The Gap in Our Pipeline . . . 104
The Problem: Non-Commuting Terms . . . 105
The Trotter Idea . . . 106
First vs Second Order . . . 107
Beyond Trotterization: Qubitization . . . 107
What Comes Next . . . 108
Key Takeaways . . . 108
Further Reading . . . 108

Chapter 14: Trotterization in Practice 111
In This Chapter . . . 111
What Happens When Theory Meets H_2 . . . 111
The H_2 Hamiltonian, One Last Time . . . 112

Second-Order Trotter: Symmetry for Accuracy . . . 113
Choosing the Time Step . . . 114
Quick CNOT Estimate (Before Gate Decomposition) . . . 114
How Many Trotter Steps Do I Need? . . . 115
Key Takeaways . . . 116
Common Mistakes . . . 116
Exercises . . . 116

Chapter 15: The CNOT Staircase 119
In This Chapter . . . 119
The Big Picture . . . 119
The Decomposition Recipe . . . 119
Gate Counts . . . 120
Worked Example: XXYY Rotation . . . 121
Why This Makes Encoding Choice Concrete . . . 121
Key Takeaways . . . 122

Chapter 16: Cost Analysis Across Encodings 123
In This Chapter . . . 123
The Optimization Stack . . . 123
The Full Cost Formula . . . 124
H_2 (4 qubits, 15 terms) . . . 124
H_2O (12 qubits, frozen core, STO-3G) . . . 124
Scaling to Larger Molecules . . . 125
Key Takeaways . . . 125
Common Mistakes . . . 126
Exercises . . . 126

Chapter 17: The Question We Can Now Answer 127
In This Chapter . . . 127
The Payoff . . . 127
The Script . . . 128
What Each Line Does . . . 129
The Numbers . . . 130
A Closer Look at the Output . . . 130
Extra Credit: The H_2 Dissociation Curve . . . 131
From One Molecule to Any Molecule . . . 133
Key Takeaways . . . 133

Chapter 18: The Water Bond Angle 135
In This Chapter . . . 135
The Question . . . 135
The Strategy: A Potential Energy Surface Scan . . . 136
The Skeleton Trick . . . 136
The Integrals . . . 137
The Scan . . . 138
The Result: Coarse Scan . . . 139
Zooming In: Fine Scan . . . 139

What Do the Numbers Mean? . . . 140
Why Does Water Bend? . . . 140
What About the Encoding? . . . 141
The Greenhouse Connection . . . 141
Key Takeaways . . . 142

Chapter 19: Algorithms — VQE and QPE 143
In This Chapter . . . 143
Two Algorithms, One Input . . . 143
VQE: The Near-Term Algorithm . . . 144
QPE: The Fault-Tolerant Algorithm . . . 146
VQE vs QPE: A Summary . . . 147
The Measurement Program in Practice . . . 148
What FockMap Does — and What It Doesn't . . . 148
Key Takeaways . . . 149

Chapter 20: Speaking the Hardware's Language 151
In This Chapter . . . 151
Three Formats, One Circuit . . . 151
OpenQASM: The Universal Format . . . 152
Q#: Azure Quantum . . . 153
JSON: The Python Bridge . . . 154
Same Circuit, Three Formats . . . 155
When to Use Which . . . 155
What Export Does Not Solve . . . 156
Verification Checklist . . . 156
Key Takeaways . . . 157

Chapter 21: Scaling — From H_2 to FeMo-co 159
In This Chapter . . . 159
The Scaling Landscape . . . 159
The Encoding Crossover . . . 160
The Tapering Dividend . . . 161
Application: Encoding Choice at H_2O Scale . . . 161
The Grand Challenge: FeMo-co . . . 162
Where Classical Methods Still Win . . . 163
Key Takeaways . . . 163

Chapter 22: What Comes Next 165
In This Chapter . . . 165
What We Built . . . 165
Near-Term Extensions . . . 166
Bosonic Simulation . . . 167
Lattice Models . . . 167
Quantum Error Correction . . . 168
Open Problems . . . 168
The Pipeline Is Ready . . . 169
Key Takeaways . . . 169

Appendix A: Library Cookbook Reference **171**
Type System at a Glance 171
Key Functions 172

Appendix B: Mathematical Background **175**
B.1 Why Encodings? (Theory Ch. 1) 175
B.2 Second Quantization (Theory Ch. 2) 175
B.3 Pauli Algebra (Theory Ch. 3) 176
B.4 Jordan–Wigner Transform (Theory Ch. 4) 176
B.5 Beyond Jordan–Wigner (Theory Ch. 5) 176
B.6 Bosonic Operators (Theory Ch. 6) 176
B.7 Mixed Systems (Theory Ch. 7) 177

From Molecules to Quantum Circuits

A Computational Guide to Fermion-to-Qubit Encodings

John S Azariah

Centre for Quantum Software and Information University of Technology Sydney

---

Preface

Every chemistry textbook tells you that water's bond angle is 104.5° and waves at VSEPR theory. Every quantum computing textbook tells you that fermions need to be encoded as qubits and shows you the Jordan-Wigner transform. Very few resources connect these two worlds with enough detail that you could actually *compute* the bond angle from the quantum simulation, or understand *why* the encoding choice matters for the circuit you'll run on a quantum computer.

This book fills that gap.

We walk through the complete pipeline — from molecular geometry to quantum circuit — with two running examples: the hydrogen molecule (H_2), because it is the simplest system that exhibits all the essential structure, and the water molecule (H_2O), because it is the most important molecule on Earth and rich enough to make the engineering trade-offs tangible.

Every formula in this book has a corresponding executable computation in the FockMap library, an open-source F# framework for symbolic Fock-space operator algebra. Every sign, every coefficient, every intermediate Pauli string is computed explicitly. Nothing is left as an exercise for the reader — though there are plenty of exercises at the end of each chapter for readers who want to deepen their understanding.

The book follows a deliberate pedagogical ordering: chemistry and physics first, mathematical formalism second, executable code third. We believe the question "why does this matter?" should always be answered before the question "how does this work?" — and both should be answered before "how do I compute it?"

Who This Book Is For

- **Graduate students** starting in quantum chemistry simulation who need to understand the encoding layer between chemistry and circuits
- **Physicists** crossing into quantum computing who know Hamiltonians but not encodings
- **Software engineers** building quantum simulation pipelines who need the physics explained carefully
- **Lecturers** building a course module on quantum simulation who want homework exercises with verifiable answers

What You Need to Know

We assume familiarity with linear algebra (vectors, matrices, eigenvalues) and introductory quantum mechanics (wavefunctions, bra-ket notation, the hydrogen atom). We do *not* assume prior knowledge of:

- Second quantization or Fock space
- Pauli algebra or qubit representations
- Fermion-to-qubit encodings
- F# or functional programming

All of these are developed from scratch within the book.

How to Read This Book

The 21 chapters are organized into six stages, following the quantum simulation pipeline:

1. **The Electronic Structure Problem** (Chapters 1–3) — from molecules to integrals
2. **Encoding** (Chapters 4–7) — from fermions to qubits
3. **Tapering** (Chapters 8–11) — removing redundant qubits
4. **Trotterization** (Chapters 12–15) — from Hamiltonian to gates
5. **Circuit Output** (Chapters 16–18) — generating executable code
6. **Putting It Together** (Chapters 19–21) — the complete pipeline and what comes next

You can read straight through (recommended for first reading), or jump to a specific stage if you already know the earlier material. Each chapter begins with "In This Chapter" learning objectives and ends with "Key Takeaways" and exercises.

The Companion Software

All computations in this book are reproducible using the FockMap library:

- **Source code:** https://github.com/johnazariah/encodings
- **NuGet package:** `dotnet add package FockMap`
- **Web documentation:** https://johnazariah.github.io/encodings/

The library is open-source (MIT license), runs on Windows, macOS, and Linux via .NET 10, and has zero runtime dependencies.

Acknowledgements

This work grew out of research at the Centre for Quantum Software and Information at the University of Technology Sydney. I am grateful to the F# Software Foundation and the .NET open-source community for the language and runtime ecosystem that made this library possible.

Sydney, March 2026

Chapter 1: The Electronic Structure Problem

We have a molecule. We want a quantum circuit. This chapter is about the first question: what, exactly, are we trying to compute?

In This Chapter

- **What you'll learn:** How a molecule becomes a finite mathematical problem — from the Schrödinger equation, through the Born–Oppenheimer approximation, to a small set of numbers called integrals.
- **Why this matters:** Every step in the rest of this book — encoding, tapering, Trotterization, circuit compilation — operates on the output of this chapter. If you don't understand what the integrals represent, you can't understand what the quantum computer is computing.
- **Prerequisites:** Linear algebra (matrices, eigenvalues). Introductory quantum mechanics (wavefunctions, bra-ket notation). No prior knowledge of molecular orbital theory or second quantization is assumed.

The Question

Here is a question that a first-year chemistry student can state but no classical computer can answer exactly for anything larger than helium:

> **Given a molecule — its atoms and their positions — what is its ground-state energy?**

The ground-state energy determines whether a chemical reaction will happen, how strong a bond is, what shape a molecule takes, and why water boils at 100°C rather than −50°C. It is arguably the single most important quantity in all of chemistry. And for any system with more than one electron, we cannot compute it analytically — the electron–electron repulsion couples the electrons' coordinates, making the Schrödinger equation non-separable. Even helium, the simplest multi-electron atom, has no closed-form solution. Perturbation theory can improve the picture incrementally — each order of the expansion buys another digit or two of accuracy — but it

never closes the gap to an exact result, and it converges poorly (or not at all) for strongly correlated systems.

Classical computational chemistry has developed an extraordinary arsenal of approximation methods — Hartree–Fock, density functional theory, coupled cluster, configuration interaction — each trading accuracy for tractability in a different way. These methods have transformed chemistry and earned multiple Nobel Prizes. But they all share a fundamental limitation: the computational cost of capturing electron correlation grows exponentially with the number of electrons.

This is where quantum simulation enters the picture. A quantum computer can, in principle, represent the quantum state of the electrons directly — using qubits in place of orbitals — and extract the ground-state energy without the exponential cost. The catch is that translating the molecular problem into a form that a quantum computer can execute requires a specific sequence of mathematical transformations, each with its own conventions, sign choices, and opportunities for error.

This chapter covers the first transformation: turning the continuous, infinite-dimensional molecular problem into a finite-dimensional matrix problem. The result will be a set of numbers — the **molecular integrals** — that encode everything we need to know about the molecule.

We will do this for the hydrogen molecule, H_2. Not because H_2 is interesting in itself (it isn't — any laptop can solve H_2 exactly in milliseconds), but because H_2 is small enough that we can see every step, check every number, and build intuition for what happens at larger scale. Later, when we work with H_2O, the same pipeline will produce larger numbers but the same kinds of objects.

A Molecule Is a Collection of Charges

Strip away the language of orbitals and bonds and wavefunctions, and what remains is electrostatics: a molecule is a collection of positively charged nuclei and negatively charged electrons, interacting via Coulomb's law.

For H_2, this means: - **Two protons** (charge $+e$ each), separated by a distance R - **Two electrons** (charge $-e$ each), somewhere in the space around them

The total energy of this system depends on five types of interaction:

Interaction	Formula	Sign	Strength
Proton kinetic energy	$-\frac{\hbar^2}{2M_p}\nabla_A^2$	—	Tiny (protons are heavy)
Electron kinetic energy	$-\frac{\hbar^2}{2m_e}\nabla_i^2$	—	Significant
Proton–proton repulsion	$\frac{e^2}{\lvert\mathbf{R}_A-\mathbf{R}_B\rvert}$	$+$	Repulsive

Interaction	Formula	Sign	Strength
Electron–proton attraction	$-\frac{Z_A e^2}{\lvert\mathbf{r}_i-\mathbf{R}_A\rvert}$	$-$	Attractive (this is what holds the molecule together)
Electron–electron repulsion	$\frac{e^2}{\lvert\mathbf{r}_i-\mathbf{r}_j\rvert}$	$+$	Repulsive (and this is what makes the problem hard)

The full Hamiltonian is the sum of all five:

$$\hat{H} = \underbrace{-\sum_{A=1}^{M}\frac{\hbar^2}{2M_A}\nabla_A^2}_{\text{nuclear KE}} \underbrace{-\sum_{i=1}^{N}\frac{\hbar^2}{2m_e}\nabla_i^2}_{\text{electronic KE}} + \underbrace{\sum_{A<B}\frac{Z_A Z_B e^2}{\lvert\mathbf{R}_A-\mathbf{R}_B\rvert}}_{\text{nuclear repulsion}} \underbrace{-\sum_{i,A}\frac{Z_A e^2}{\lvert\mathbf{r}_i-\mathbf{R}_A\rvert}}_{\text{electron-nuclear attraction}} + \underbrace{\sum_{i<j}\frac{e^2}{\lvert\mathbf{r}_i-\mathbf{r}_j\rvert}}_{\text{electron repulsion}}$$

This is exact. No approximations. If we could solve this equation, we would have the exact energy of any molecule.

We can't. Not for H_2, not for anything with more than one electron. The electron–electron repulsion term couples the coordinates of every pair of electrons, making the equation non-separable.

> **Common Mistake #1:** Students sometimes think the difficulty is the number of particles. It isn't — classical N-body problems with gravitational interactions are also hard. The quantum difficulty is that we must find the *wavefunction* $\Psi(\mathbf{r}_1, \mathbf{r}_2, \dots, \mathbf{r}_N)$, which lives in a $3N$-dimensional space. For a modest molecule with 50 electrons, this is a function of 150 continuous variables. No grid-based method can touch that.

The Born–Oppenheimer Approximation: Freezing the Nuclei

Protons are 1836 times heavier than electrons. On the timescale of electronic motion, the nuclei are essentially stationary. This observation — due to Born and Oppenheimer (1927) — is the first simplification, and by far the most consequential: we treat the nuclear positions $\{\mathbf{R}_A\}$ as fixed parameters, not dynamical variables.

The result is the **electronic Hamiltonian**:

$$\hat{H}_{\text{el}} = -\sum_{i=1}^{N}\frac{\hbar^2}{2m_e}\nabla_i^2 - \sum_{i,A}\frac{Z_A e^2}{\lvert\mathbf{r}_i-\mathbf{R}_A\rvert} + \sum_{i<j}\frac{e^2}{\lvert\mathbf{r}_i-\mathbf{r}_j\rvert}$$

The nuclear repulsion energy becomes a constant for a given geometry:

$$V_{nn} = \frac{Z_A Z_B e^2}{R}$$

For H_2 at the equilibrium bond length $R = 0.7414$ Å (= 1.401 Bohr): $V_{nn} = 0.7151$ Ha. We add this constant back at the end.

What remains is a problem in the electronic coordinates alone. Solve it for one nuclear geometry, and you get the electronic energy $E_{\text{el}}(R)$. Repeat for many geometries, and you trace out the **potential energy surface** — the curve that tells you bond lengths, bond angles, and vibrational frequencies.

> **Why this matters for quantum simulation:** The Born–Oppenheimer approximation is not unique to quantum computing — every classical electronic structure method uses it too. It means the quantum computer's job is to solve the electronic problem at a *fixed* geometry. If you want a potential energy surface, you run the quantum computer many times, once per geometry. (This is exactly what our H_2O bond-angle scan will do in later chapters.)

Basis Sets: Making the Infinite Finite

The electronic Hamiltonian acts on wavefunctions $\Psi(\mathbf{r}_1, \mathbf{r}_2)$ — functions of continuous 3D coordinates. A computer (classical or quantum) cannot represent a continuous function exactly. We need to discretize.

The standard approach: expand each molecular orbital as a **linear combination of known functions**, called basis functions. This is the same idea as representing a vector in a finite basis, except the "vectors" are functions and the "basis" is a set of atomic orbital shapes.

The Hydrogen Atom: Exact Solutions We Can't Use Directly

For a single hydrogen atom, the Schrödinger equation has exact solutions: the familiar $1s$, $2s$, $2p$, $3d$, ... orbitals. These have the form

$$\phi_{1s}(r) \propto e^{-\zeta r}$$

where ζ determines how tightly the electron is bound. These Slater-type orbitals are physically correct, but they have a computational problem: the integrals involving products of Slater functions on *different* atomic centres cannot be evaluated in closed form.

Gaussians: The Practical Compromise

The solution, introduced by S. F. Boys in 1950, is to replace Slater-type orbitals with sums of Gaussian functions:

$$g(r) \propto e^{-\alpha r^2}$$

The product of two Gaussians centred at different points is another Gaussian centred at a third point — a property that makes all the necessary integrals analytically tractable. The price: Gaussians decay too quickly at large r and have the wrong cusp at $r = 0$. The fix: use several Gaussians to approximate each Slater orbital.

STO-3G: The Smallest Meaningful Basis

The "Slater-Type Orbital approximated by 3 Gaussians" (STO-3G) basis set fits three Gaussians to each atomic orbital. For hydrogen, STO-3G provides one basis function per atom: an approximation to the $1s$ orbital.

Is STO-3G a good basis set? No — it is the smallest possible choice, and it captures only the crudest features of the electronic structure. Serious computational chemistry uses much larger basis sets (cc-pVDZ, cc-pVTZ, aug-cc-pVQZ, ...). But STO-3G is perfect for *learning*, because it keeps the numbers small enough to track by hand while still exhibiting all the essential structure.

> **Common Mistake #2:** Confusing "basis set" with "basis states." The basis set (STO-3G) determines which *orbitals* we use. The basis states (the 6 configurations in the table below) are the many-electron states built from those orbitals. A bigger basis set gives more orbitals, which gives exponentially more configurations — and this is where the computational hardness lives.
>
> **What "exact" means in this book:** When we say a computation is "exact" (e.g., Full Configuration Interaction), we mean exact *within the chosen basis set*. STO-3G H_2 has only 4 spin-orbitals and 6 configurations, so FCI is trivial and gives the exact answer for that basis. But STO-3G itself is a crude approximation to the true electronic wavefunction — a larger basis set (cc-pVDZ, cc-pVTZ) would give a more accurate energy. The quantum simulation pipeline operates at the basis-set level: it solves the finite-dimensional problem exactly, but the finite-dimensional problem is only as good as the basis set that defines it. Throughout this book, "exact" always means "exact within the basis."

Molecular Orbitals for H_2

With one STO-3G basis function on each hydrogen atom ($1s_A$ and $1s_B$), the Linear Combination of Atomic Orbitals (LCAO) procedure gives two molecular orbitals:

$$\sigma_g = \frac{1s_A + 1s_B}{\sqrt{2(1+S)}} \qquad \text{(bonding)}$$

$$\sigma_u = \frac{1s_A - 1s_B}{\sqrt{2(1-S)}} \qquad \text{(antibonding)}$$

where $S = \langle 1s_A \mid 1s_B \rangle$ is the overlap integral between the two atomic orbitals.

The **bonding** orbital σ_g has its electron density concentrated *between* the nuclei — this is what holds the molecule together. The **antibonding** orbital σ_u has a nodal plane at the midpoint — electron density here pushes the nuclei apart.

Each spatial orbital can hold one electron of each spin (α = spin-up, β = spin-down), giving us 2 spatial orbitals × 2 spins = **4 spin-orbitals**:

Index p	Spatial orbital	Spin
0	σ_g	α
1	σ_g	β
2	σ_u	α
3	σ_u	β

The Six Configurations of H_2

Two electrons distributed among 4 spin-orbitals can occupy $\binom{4}{2} = 6$ distinct configurations. We write each as an occupation vector $|n_0 n_1 n_2 n_3\rangle$, where $n_p \in \{0, 1\}$ indicates whether spin-orbital p is occupied:

Configuration	Occupation	Description
$\|1100\rangle$	$\sigma_{g\alpha}\,\sigma_{g\beta}$	Both electrons in the bonding orbital (ground state in Hartree–Fock)
$\|1010\rangle$	$\sigma_{g\alpha}\,\sigma_{u\alpha}$	One in each orbital, same spin (triplet)
$\|1001\rangle$	$\sigma_{g\alpha}\,\sigma_{u\beta}$	One in each, opposite spin
$\|0110\rangle$	$\sigma_{g\beta}\,\sigma_{u\alpha}$	One in each, opposite spin
$\|0101\rangle$	$\sigma_{g\beta}\,\sigma_{u\beta}$	One in each, same spin (triplet)

Configuration	Occupation	Description
$\lvert 0011\rangle$	$\sigma_{u\alpha}\,\sigma_{u\beta}$	Both in antibonding orbital (highest energy)

The **exact** ground state of H_2 is a *superposition* of some of these configurations. The Hartree–Fock approximation uses only the first ($\lvert 1100\rangle$), capturing about 99% of the ground-state energy. The remaining ~1% is the **correlation energy** — the part that arises from electron–electron interactions that a single-configuration picture misses.

This 1% sounds small. It isn't. The correlation energy determines whether a reaction happens, which isomer is more stable, and what the dissociation curve looks like. Classical methods that capture correlation (CCSD(T), full CI) scale exponentially. This is precisely the gap that quantum simulation aims to fill.

> **A hint of what's coming:** Those occupation vectors $\lvert n_0 n_1 n_2 n_3\rangle$ look exactly like qubit computational basis states $\lvert q_0 q_1 q_2 q_3\rangle$. Four spin-orbitals → four qubits. Six configurations → a 16-dimensional Hilbert space (of which 6 states have two electrons). A quantum computer can represent superpositions of these configurations natively.
>
> The mapping is less straightforward than it appears, though — we'll see why in Chapter 4.

Second Quantization: A Preview

At this point we could write down the 6×6 Hamiltonian matrix in the configuration basis and diagonalize it. For H_2, that would work fine. But it wouldn't scale — for H_2O with 14 spin-orbitals, the configuration space has $\binom{14}{10} = 1001$ states, and for larger molecules the dimension grows combinatorially.

There is a more compact way to write the Hamiltonian — using **creation and annihilation operators** rather than wavefunctions. We will develop this formalism properly in Chapter 4, where we'll need it to understand encoding. For now, the key result is that the electronic Hamiltonian can be written as:

$$\hat{H} = \sum_{pq} h_{pq}\, a_p^\dagger a_q + \frac{1}{2}\sum_{pqrs} \langle pq \mid rs\rangle\, a_p^\dagger a_q^\dagger a_s a_r + V_{nn}$$

where h_{pq} are **one-body integrals**, $\langle pq \mid rs\rangle$ are **two-body integrals**, and V_{nn} is the nuclear repulsion constant. The operators $a_p^\dagger$ and a_p create and destroy electrons in specific spin-orbitals, and they obey algebraic rules (the canonical anti-commutation relations) that automatically enforce the Pauli exclusion principle.

This is the object that the rest of the book operates on. Everything that follows — encoding, tapering, Trotterization — takes this Hamiltonian and transforms it into something a quantum computer can execute.

The Numbers: H_2 Integrals in STO-3G

For H_2 in STO-3G at the equilibrium bond length, the non-zero one-body integrals (in the spatial orbital basis) are:

Integral	Value (Hartree)	Physical meaning
h_{00}	-1.2563	σ_g orbital energy (kinetic + nuclear attraction)
h_{11}	-0.4719	σ_u orbital energy

The off-diagonal elements $h_{01} = h_{10} = 0$ by symmetry (σ_g and σ_u are orthogonal).

The two-body integrals are a $2 \times 2 \times 2 \times 2$ tensor — 16 elements, of which only a few are distinct by symmetry. We will develop these fully in Chapter 3, after sorting out the notation conventions in Chapter 2.

The nuclear repulsion constant is $V_{nn} = 0.7151$ Ha.

These numbers — the integrals and the nuclear repulsion — are the *output* of this chapter and the *input* to everything that follows. A classical electronic structure code (PySCF, Gaussian, ORCA) computes them from the molecular geometry and basis set. We will treat them as given.

Key Takeaways

- A molecule is a collection of charged particles interacting via Coulomb's law. The ground-state energy is the lowest eigenvalue of the molecular Hamiltonian.
- The **Born–Oppenheimer approximation** fixes the nuclear positions, reducing the problem to the electronic Hamiltonian at a specific geometry.
- A **basis set** (STO-3G) discretizes the continuous orbital space into a finite set of molecular orbitals. For H_2, this gives 2 spatial orbitals → 4 spin-orbitals → 6 two-electron configurations.
- **Second quantization** rewrites the Hamiltonian in terms of creation and annihilation operators, encoding the Pauli exclusion principle through anti-commutation relations (CAR).
- The result is a Hamiltonian specified by one-body integrals h_{pq}, two-body integrals $\langle pq \mid rs \rangle$, and a nuclear repulsion constant V_{nn}. These numbers are the starting point for encoding.

Common Mistakes

1. **Confusing basis set with basis states.** The basis set (STO-3G) determines the *orbitals*. The basis states ($|1100\rangle$, etc.) are many-electron configurations built from those orbitals. More orbitals → exponentially more configurations.
2. **Thinking the difficulty is the particle count.** The quantum difficulty is not that we have many particles — it's that the wavefunction lives in an exponentially large Hilbert space. 50 electrons in 100 spin-orbitals gives $\binom{100}{50} \approx 10^{29}$ configurations.
3. **Forgetting V_{nn}.** The nuclear repulsion energy is a constant, not an operator, but it must be added back to get the total energy. Many encoding tutorials drop it, leading to energies that are off by ~0.7 Ha for H_2.

Exercises

1. **Configuration counting.** How many two-electron configurations exist for H_2O in a minimal (STO-3G) basis with 7 spatial orbitals (14 spin-orbitals, 10 electrons)? How does this compare with H_2?
2. **Born–Oppenheimer curve.** If you vary the bond length R from 0.5 to 3.0 Å and compute the electronic energy $E_{\text{el}}(R)$ at each point, what shape does the curve $E_{\text{el}}(R) + V_{nn}(R)$ have? Where is the minimum? (You don't need to compute this — sketch it from physical reasoning.)
3. **Basis set scaling.** If we used the cc-pVDZ basis instead of STO-3G, hydrogen would have 5 basis functions per atom instead of 1. How many spatial orbitals, spin-orbitals, and two-electron configurations would H_2 have?

Further Reading

- Szabo, A. and Ostlund, N. S. *Modern Quantum Chemistry: Introduction to Advanced Electronic Structure Theory*. Dover, 1996. Chapters 1–3 cover the material in this chapter at greater depth.
- Helgaker, T., Jørgensen, P., and Olsen, J. *Molecular Electronic-Structure Theory*. Wiley, 2000. The definitive reference for basis sets and integral evaluation.

Next: Chapter 2 — The Notation Minefield

Chapter 2: The Notation Minefield

Two communities, two integral conventions, one factor-of-two error that will derail your entire Hamiltonian if you get it wrong. This chapter exists to make sure you don't.

In This Chapter

- **What you'll learn:** The difference between chemist's (Mulliken) and physicist's (Dirac) two-electron integral notation, how to convert between them, and the three most common errors that produce plausible-but-wrong Hamiltonians.
- **Why this matters:** Every sign, every coefficient, every eigenvalue in the rest of this book depends on getting the integral convention right *here*. The errors are insidious: wrong-convention Hamiltonians have the right structure, the right symmetry, and the right number of terms. Only the numbers are wrong.
- **Prerequisites:** Chapter 1 (you know what h_{pq} and the two-body integrals represent physically).

The Problem

There are two standard notations for two-electron repulsion integrals. They are both in widespread use. They are both perfectly valid. And they index the same physical quantity with the indices in **different positions**.

If you take integrals from a chemistry code (which typically outputs chemist's notation) and plug them into a physics formula (which uses physicist's notation) without converting, you will get the wrong Hamiltonian. You will not get an error message. Your code will run. Your Pauli strings will have the right structure. Your eigenvalues will be wrong — silently, plausibly, maddeningly wrong.

This chapter names the two conventions, shows you exactly how they differ, gives you the conversion rule, and then catalogues the three errors that trap essentially every student who encounters this material for the first time. If you remember nothing else from this chapter, remember the boxed equation at the centre.

Convention 1: Chemist's Notation (Mulliken)

Chemist's notation — also called Mulliken notation or charge-density notation — groups indices by **spatial coordinate**. The integral is written with square brackets:

$$[pq \mid rs] = \iint \phi_p^*(\mathbf{r}_1)\,\phi_q(\mathbf{r}_1)\,\frac{1}{r_{12}}\,\phi_r^*(\mathbf{r}_2)\,\phi_s(\mathbf{r}_2)\,d\mathbf{r}_1\,d\mathbf{r}_2$$

Read it as: "the charge density $\phi_p^*\phi_q$ at position $\mathbf{r}_1$ interacts via Coulomb repulsion with the charge density $\phi_r^*\phi_s$ at position $\mathbf{r}_2$."

The bracket $[pq \mid$ contains the two orbitals that share electron 1's coordinate. The bracket $\mid rs]$ contains the two orbitals that share electron 2's coordinate. Within each bracket: complex conjugate (bra) first, ket second.

Why chemists like it: The indices are grouped the way charge densities are: $\rho_1(\mathbf{r}_1) = \phi_p^*(\mathbf{r}_1)\phi_q(\mathbf{r}_1)$, and the integral is just the Coulomb interaction between two charge distributions. It makes the *physics* of repulsion visually clear.

> I first encountered this notation in Dr. Robert Ditchfield's Quantum Chemistry course (Chem 81) at Dartmouth in 1992. He would write the charge-density form on the blackboard and say, "This is the natural way to think about it." He was right — it *is* the natural way to think about it. It's just not the way the second-quantized Hamiltonian is written.

Where you'll encounter it: Most quantum chemistry codes (PySCF, Gaussian, ORCA, NWChem) output integrals natively in chemist's notation. OpenFermion's `MolecularData` object stores them this way. If you import integrals from a chemistry pipeline, assume chemist's notation unless the documentation explicitly says otherwise.

Convention 2: Physicist's Notation (Dirac)

Physicist's notation — sometimes called Dirac notation for integrals (not to be confused with Dirac bra-ket notation for states) — groups indices by **particle identity**:

$$\langle pq \mid rs \rangle = \iint \phi_p^*(\mathbf{r}_1)\,\phi_q^*(\mathbf{r}_2)\,\frac{1}{r_{12}}\,\phi_r(\mathbf{r}_1)\,\phi_s(\mathbf{r}_2)\,d\mathbf{r}_1\,d\mathbf{r}_2$$

Read it as: "electron 1 scatters from orbital p to orbital r, while electron 2 scatters from orbital q to orbital s."

Here p and r both belong to electron 1 (bra and ket respectively), while q and s both belong to electron 2. The angle brackets $\langle\,\rangle$ signal physicist's convention; the square brackets $[\,]$ signal chemist's.

Why physicists like it: The operator ordering in second quantization directly mirrors the integral indices: $\langle pq \mid rs\rangle\, a_p^\dagger a_q^\dagger a_s a_r$. The bra indices (p, q) match the creation operators; the ket indices (r, s) match the annihilation operators (in reverse order — more on this below).

Where you'll encounter it: Most quantum computing papers, including the foundational encoding papers (Jordan–Wigner, Bravyi–Kitaev, Seeley–Richard–Love), use physicist's notation. The second-quantized Hamiltonian is almost always written with angle brackets.

The Conversion: One Equation to Rule Them All

Comparing the two definitions term by term — matching which orbital sits at which coordinate — yields:

$$\boxed{\langle pq \mid rs\rangle = [pr \mid qs]}$$

That's it. Memorize this. Tattoo it if necessary.

The indices are **shuffled**, not just relabelled. The physicist's pair (p, r) — the two orbitals for electron 1 — becomes the *left* bracket of the chemist's notation, with p in the bra position and r in the ket position. The physicist's pair (q, s) — the two orbitals for electron 2 — becomes the *right* bracket.

Worked example

Suppose you have the chemist's integral $[00 \mid 00] = 0.6745$ Ha (the Coulomb self-repulsion of two electrons both in the σ_g orbital). What is the corresponding physicist's integral?

Use the conversion: $\langle pq \mid rs\rangle = [pr \mid qs]$. We need $[pr \mid qs] = [00 \mid 00]$, so $p = 0, r = 0, q = 0, s = 0$. Therefore:

$$\langle 00 \mid 00\rangle = [00 \mid 00] = 0.6745 \text{ Ha}$$

That one was trivial — all indices are the same. Now consider a more revealing example: $[00 \mid 11] = 0.6636$ Ha (the Coulomb repulsion between an electron in σ_g and one in σ_u). We need $[pr \mid qs] = [00 \mid 11]$, so $p = 0, r = 0, q = 1, s = 1$. Therefore:

$$\langle 01 \mid 01\rangle = [00 \mid 11] = 0.6636 \text{ Ha}$$

Notice: $\langle 01 \mid 01 \rangle \neq \langle 00 \mid 11 \rangle$ in general. The integral $\langle 00 \mid 11 \rangle = [01 \mid 01]$ is a different integral (the exchange integral). For H_2 in STO-3G, $[01 \mid 01] = 0.6976$ Ha — a different value from $[00 \mid 11] = 0.6636$ Ha.

> **Common Mistake #1: Using the wrong convention.** If you take $[00 \mid 11]$ from PySCF and use it as $\langle 00 \mid 11 \rangle$ in the physicist's Hamiltonian, you have swapped a Coulomb integral for an exchange integral. The Hamiltonian will have plausible structure, but the eigenvalues will be wrong. There is no error message. The only way to catch it is to verify against a known result.

The Hamiltonian: Which Convention Goes Where?

The standard second-quantized electronic Hamiltonian is written in **physicist's** notation:

$$\hat{H} = \sum_{pq} h_{pq}\, a_p^\dagger a_q + \frac{1}{2} \sum_{pqrs} \langle pq \mid rs \rangle\, a_p^\dagger a_q^\dagger a_s a_r + V_{nn}$$

Note three things:

1. **The one-body term** $h_{pq}\, a_p^\dagger a_q$ is convention-independent — there's only one standard for one-electron integrals.
2. **The two-body term** uses physicist's notation $\langle pq \mid rs \rangle$. If you have chemist's integrals, convert first: $\langle pq \mid rs \rangle = [pr \mid qs]$.
3. **The operator ordering is** $a_p^\dagger a_q^\dagger a_s a_r$ — the annihilation operators are in *reverse* order relative to the ket indices of the integral (r, s become $a_s a_r$, not $a_r a_s$). This is not a typo. It follows from the definition of the antisymmetrized matrix element and the anti-commutation relations. Getting this order wrong flips the sign on certain exchange terms.

If you prefer to work entirely in chemist's notation, the Hamiltonian reads:

$$\hat{H} = \sum_{pq} h_{pq}\, a_p^\dagger a_q + \frac{1}{2} \sum_{pqrs} [pr \mid qs]\, a_p^\dagger a_q^\dagger a_s a_r + V_{nn}$$

This is the same equation — we just substituted the conversion rule. Use whichever form matches your integral source, but be consistent.

The Symmetries That Save You (and Can Mislead You)

Two-electron integrals have symmetries that reduce the number of independent values:

Chemist's notation symmetries

$$[pq \mid rs] = [qp \mid rs]^* = [pq \mid sr]^* = [rs \mid pq]$$

For **real** orbitals (which is the case for all our examples):

$$[pq \mid rs] = [qp \mid rs] = [pq \mid sr] = [qp \mid sr] = [rs \mid pq] = [sr \mid pq] = [rs \mid qp] = [sr \mid qp]$$

That's 8-fold symmetry — the number of independent integrals is roughly $n^4/8$ rather than n^4.

Physicist's notation symmetries

$$\langle pq \mid rs \rangle = \langle qp \mid sr \rangle = \langle rs \mid pq \rangle^*$$

For real orbitals:

$$\langle pq \mid rs \rangle = \langle qp \mid sr \rangle = \langle rs \mid pq \rangle = \langle sr \mid qp \rangle$$

Note the difference: the symmetry in physicist's notation swaps *both* bra indices and *both* ket indices simultaneously, while in chemist's notation you can swap within each bracket independently.

> **Common Mistake #2: Applying chemist's symmetries to physicist's integrals.** If you assume $\langle pq \mid rs \rangle = \langle qp \mid rs \rangle$ (swapping only the bra indices), you are applying a chemist's symmetry to a physicist's integral. In physicist's notation, the correct symmetry is $\langle pq \mid rs \rangle = \langle qp \mid sr \rangle$ — you must swap *both* bra indices and *both* ket indices together.

The $\frac{1}{2}$ Prefactor

The factor of $\frac{1}{2}$ in front of the two-body term:

$$\frac{1}{2} \sum_{pqrs} \langle pq \mid rs \rangle \, a_p^\dagger a_q^\dagger a_s a_r$$

exists because the sum over all four indices counts every pair of electrons twice (once as (p, q) and once as (q, p)). The $\frac{1}{2}$ corrects for this double-counting.

Some references absorb the $\frac{1}{2}$ into the integrals, defining $g_{pqrs} = \frac{1}{2}\langle pq \mid rs\rangle$. Others restrict the summation to $p < q$ and $r < s$ to avoid double-counting, eliminating the prefactor. Both are correct. But if you mix conventions — using the unrestricted sum without the $\frac{1}{2}$, or using the restricted sum with it — your two-body energy will be off by a factor of 2.

> **Common Mistake #3: Losing the $\frac{1}{2}$.** This one produces eigenvalues that are in the right ballpark but wrong in the second decimal place. Because the one-body terms are unaffected, the error is systematic and can look like a "correlation energy correction" rather than a bug. It is a bug.

A Complete Example: H_2 Two-Body Integrals

Here are all the non-zero two-body integrals for H_2 in STO-3G, in *both* conventions, so you can verify conversions:

Chemist's $[pq \mid rs]$	Value (Ha)	Physicist's $\langle pq \mid rs\rangle$	Conversion used
$[00 \mid 00]$	0.6745	$\langle 00 \mid 00\rangle$	$\langle 00 \mid 00\rangle = [00 \mid 00]$
$[11 \mid 11]$	0.6974	$\langle 11 \mid 11\rangle$	$\langle 11 \mid 11\rangle = [11 \mid 11]$
$[00 \mid 11]$	0.6636	$\langle 01 \mid 01\rangle$	$\langle 01 \mid 01\rangle = [00 \mid 11]$
$[01 \mid 01]$	0.6976	$\langle 00 \mid 11\rangle$	$\langle 00 \mid 11\rangle = [01 \mid 01]$
$[01 \mid 10]$	0.1809	$\langle 01 \mid 10\rangle$	$\langle 01 \mid 10\rangle = [01 \mid 10]$

Study this table. Make sure you can derive each entry in the rightmost column from the boxed conversion rule. If you can do that, you will never make error #1.

The Companion Library: FockMap

Throughout this book, we verify every formula and every table against executable code. The library we use is **FockMap** — an open-source F# framework for symbolic Fock-space operator algebra.

A few things worth knowing about it:

- **It is symbolic, not numerical.** When FockMap multiplies two Pauli strings, it uses the exact Pauli multiplication table with algebraic phase tracking — not

floating-point matrix multiplication. Intermediate results are inspectable and exact down to the sign.

- **It is functional.** FockMap is written in F#, a functional-first language on .NET. If you've never used F#, don't worry — the code snippets in this book are short and read like mathematical notation. You don't need to be an F# programmer to follow them; you only need to be able to read `let x = ...` as "define x to be ..."
- **It runs anywhere.** FockMap targets .NET 10 and runs on Windows, macOS, and Linux. Install it with `dotnet add package FockMap`, or clone the repository and build from source.
- **It is the subject of the JOSS paper** cited in the preface. If you want to understand the library's design — the type system, the two encoding frameworks, the test suite — that paper is the place to look. This book focuses on *using* the library to learn the physics.

When you see code in this book, it serves one purpose: to show that the mathematics we just derived actually computes the right answer. The code is evidence, not the lesson.

FockMap's Convention

FockMap uses **physicist's notation** throughout. The `computeHamiltonianWith` function expects integrals in physicist's convention:

- One-body key `"p,q"` → h_{pq}
- Two-body key `"p,q,r,s"` → $\langle pq \mid rs \rangle$

If your integral source provides chemist's notation (most do), convert the indices before building the coefficient factory:

```
// chemist_integrals stores [pq|rs] indexed as (p,q,r,s)
// physicist's convention: ⟨pq|rs⟩ = [pr|qs]
let physicistFactory (key : string) =
    let parts = key.Split(',') |> Array.map int
    match parts.Length with
    | 2 -> // one-body: same convention
        let p, q = parts.[0], parts.[1]
        chemist_integrals |> Map.tryFind (sprintf "%d,%d" p q)
    | 4 -> // two-body: convert ⟨pq|rs⟩ → [pr|qs]
        let p, q, r, s = parts.[0], parts.[1], parts.[2], parts.[3]
        chemist_integrals |> Map.tryFind (sprintf "%d,%d,%d,%d" p r q s)
    | _ -> None
```

This is the single most important function in any quantum chemistry pipeline. Get it right and everything downstream works. Get it wrong and everything downstream is silently, plausibly wrong.

Key Takeaways

- Two-electron integrals come in two flavours: chemist's $[pq \mid rs]$ (grouped by coordinate) and physicist's $\langle pq \mid rs \rangle$ (grouped by particle).
- The conversion is $\langle pq \mid rs \rangle = [pr \mid qs]$ — indices are shuffled, not just relabelled.
- The second-quantized Hamiltonian uses physicist's notation. If your integrals come from a chemistry code, convert first.
- The three most common errors are: (1) using integrals in the wrong convention, (2) applying chemist's symmetries to physicist's integrals, and (3) mishandling the $\frac{1}{2}$ prefactor.
- When in doubt, verify against known eigenvalues. There is no other reliable check.

Common Mistakes

1. **Wrong convention.** Using $[pq \mid rs]$ where $\langle pq \mid rs \rangle$ is expected (or vice versa). Produces plausible but wrong Hamiltonians.
2. **Wrong symmetries.** Physicist's integrals have 4-fold symmetry ($\langle pq \mid rs \rangle = \langle qp \mid sr \rangle$), not 8-fold. Apply the wrong symmetry rule and you'll generate integrals that don't exist.
3. **The missing $\frac{1}{2}$.** Double-counts the two-body interaction. The error looks like a systematic energy shift, not a bug.

Exercises

1. **Conversion drill.** Given $[01 \mid 10] = 0.1809$ Ha, what is $\langle 00 \mid 11 \rangle$? What is $\langle 01 \mid 10 \rangle$? (Hint: one of these equals $[01 \mid 10]$; the other does not.)
2. **Symmetry check.** Starting from $\langle 01 \mid 01 \rangle = 0.6636$ Ha, use the physicist's symmetry $\langle pq \mid rs \rangle = \langle qp \mid sr \rangle$ to find $\langle 10 \mid 10 \rangle$. Now convert both to chemist's notation and verify they correspond to the same integral.
3. **Error detection.** You compute a 4-qubit H_2 Hamiltonian and get a ground-state energy of -1.72 Ha (before adding V_{nn}). The correct value is -1.89 Ha. Which of the three errors is most likely, and why?

Further Reading

- Szabo, A. and Ostlund, N. S. *Modern Quantum Chemistry*. Chapter 2, §2.3.2–2.3.3 gives both conventions and the conversion explicitly.
- Helgaker, T., Jørgensen, P., and Olsen, J. *Molecular Electronic-Structure Theory*. Chapter 9 is the definitive treatment of two-electron integrals, including symmetry properties.

- PySCF documentation on `mol.intor('int2e')` — returns integrals in chemist's notation.

Previous: Chapter 1 — The Electronic Structure Problem

Next: Chapter 3 — From Spatial to Spin-Orbital Integrals

Chapter 3: From Spatial to Spin-Orbital Integrals

Every electron has a spin. Every integral must account for it. This chapter doubles the index space and fills in the complete integral tables that drive the encoding pipeline.

In This Chapter

- **What you'll learn:** How to expand spatial-orbital integrals into spin-orbital one-body and two-body terms, including cross-spin interactions that many implementations forget.
- **Why this matters:** The encoding step (Jordan–Wigner, Bravyi–Kitaev, etc.) operates on spin-orbital creation and annihilation operators. If your integral tables are in the spatial basis, you must expand them to spin-orbitals first. Get this wrong and your Hamiltonian will be missing the off-diagonal (XX, YY) Pauli terms entirely.
- **Prerequisites:** Chapters 1–2 (you know the spatial integrals and the notation conventions).

Why We Need Spin-Orbitals

In Chapter 1, we worked with **spatial orbitals** — the molecular orbitals σ_g and σ_u, described by their spatial wavefunctions. There were two of them, and the integrals were 2×2 matrices and $2 \times 2 \times 2 \times 2$ tensors.

But an electron is not just a spatial wavefunction. It also has **spin**.

What spin is (and isn't)

Electron spin is an intrinsic angular momentum that has no classical analogue. It is not the electron "spinning on its axis" — that mental picture, while universal in introductory courses, breaks down immediately under scrutiny (the electron would need to rotate faster than light). Spin is a purely quantum-mechanical degree of freedom that happens to behave like angular momentum: it has a magnitude ($s = 1/2$) and a projection along any chosen axis.

For a spin-$1/2$ particle, the projection along the z-axis can take only two values: - $m_s = +1/2$, which we call α or "spin-up" (↑) - $m_s = -1/2$, which we call β or "spin-down" (↓)

This is not a choice of convention — it is a consequence of the mathematics of angular momentum in quantum mechanics. The state space of a single electron's spin is exactly two-dimensional: $\{|\alpha\rangle, |\beta\rangle\}$. This is, not coincidentally, the same dimension as a qubit.

Why spin doubles the orbital space

The **Pauli exclusion principle** says that no two electrons can occupy exactly the same quantum state. A quantum state is specified by *both* the spatial orbital *and* the spin. So the spatial orbital σ_g can hold: - one electron with spin α (the spin-orbital σ_g, α) - one electron with spin β (the spin-orbital σ_g, β) - but NOT two electrons with the same spin

Each spatial orbital produces **two** spin-orbitals. Two spatial orbitals give four spin-orbitals. Seven spatial orbitals (H_2O in STO-3G) give fourteen spin-orbitals. The spin degree of freedom doubles the index space — and quadruples the two-body integral tensor.

Spin-orbit coupling: when spin and space talk to each other

In the non-relativistic Hamiltonian we've been using, spin and spatial degrees of freedom are completely independent — the Hamiltonian contains no term that couples them. An electron in σ_g with spin α has exactly the same energy as one with spin β. The one-body integrals are spin-independent, and the two-body integrals only require each electron to conserve its own spin (as we'll see below).

This is an approximation. In reality, **spin-orbit coupling** — the interaction between an electron's spin angular momentum and its orbital angular momentum — mixes the spin and spatial degrees of freedom. The coupling strength scales as Z^4 (the fourth power of the nuclear charge), so it is negligible for hydrogen ($Z = 1$) and small for light elements like carbon and oxygen, but significant for heavy atoms (iodine, lead, actinides).

For the molecules in this book (H_2, H_2O), spin-orbit coupling is negligible and we work with the non-relativistic Hamiltonian throughout. This means: - One-body integrals are diagonal in spin: $\langle \alpha \mid \beta \rangle = 0$ - Two-body integrals conserve each electron's spin independently - The total spin quantum numbers (S, M_S) are good quantum numbers

When spin-orbit coupling matters (heavy-element chemistry, relativistic quantum chemistry), the spin-orbital structure becomes more complex — spin is no longer a good quantum number, and the integral expansion rules in this chapter must be modified. We won't need that here, but it's worth knowing the boundary of our approximation.

The upshot for encoding

The second-quantized Hamiltonian

$$\hat{H} = \sum_{pq} h_{pq}\, a_p^\dagger a_q + \frac{1}{2} \sum_{pqrs} \langle pq \mid rs \rangle\, a_p^\dagger a_q^\dagger a_s a_r$$

sums over **spin-orbital** indices p, q, r, s. The creation operator $a_p^\dagger$ creates an electron in spin-orbital p — a specific spatial orbital *with a specific spin*. This means we need integral tables indexed by spin-orbitals, not spatial orbitals.

For H_2: 2 spatial orbitals → **4 spin-orbitals**. The one-body integral matrix goes from 2×2 to 4×4. The two-body integral tensor goes from $2^4 = 16$ elements to $4^4 = 256$ elements (most of which are zero).

Spin-Orbital Indexing

Each spatial orbital μ gives rise to two spin-orbitals. We use **interleaved** indexing — alternating α and β for each spatial orbital:

Spin-orbital index p	Spatial orbital	Spin	Notation
0	σ_g (orbital 0)	α	0α
1	σ_g (orbital 0)	β	0β
2	σ_u (orbital 1)	α	1α
3	σ_u (orbital 1)	β	1β

The conversion between spin-orbital index p and spatial orbital/spin:

$$\text{spatial orbital} = \lfloor p/2 \rfloor \qquad \text{spin} = p \bmod 2 \quad (0 = \alpha,\ 1 = \beta)$$

> **Convention alert:** Some references use **blocked** indexing (α orbitals first, then all β), so the ordering would be $0\alpha, 1\alpha, 0\beta, 1\beta$. We use interleaved indexing because it matches PySCF's default and FockMap's convention. If your integral source uses a different spin-orbital ordering, you must permute before proceeding. This is another silent-error source.

One-Body Expansion

The spin-orbital one-body integral is straightforward: the spatial integral survives only when the spins match.

$$h_{pq}^{\text{spin}} = h_{\lfloor p/2 \rfloor, \lfloor q/2 \rfloor}^{\text{spatial}} \times \delta(\sigma_p, \sigma_q)$$

In words: the one-body integral between spin-orbitals p and q equals the spatial integral between their parent spatial orbitals, *provided they have the same spin*. If the spins differ, the integral is zero.

Why? The one-body Hamiltonian (kinetic energy + electron-nucleus attraction) does not act on spin. The spin part of the wavefunction integrates to $\langle \alpha \mid \alpha \rangle = 1$ or $\langle \beta \mid \beta \rangle = 1$ for same spins, and $\langle \alpha \mid \beta \rangle = 0$ for opposite spins.

H_2 one-body integrals (spin-orbital basis)

p	q	h_{pq}^{spin} (Ha)	How we got it
0 (σ_g, α)	0 (σ_g, α)	-1.2563	h_{00}^{spatial}, same spin □
1 (σ_g, β)	1 (σ_g, β)	-1.2563	h_{00}^{spatial}, same spin □
2 (σ_u, α)	2 (σ_u, α)	-0.4719	h_{11}^{spatial}, same spin □
3 (σ_u, β)	3 (σ_u, β)	-0.4719	h_{11}^{spatial}, same spin □

All off-diagonal entries ($p \neq q$) are zero — either because the spatial integral is zero ($h_{01}^{\text{spatial}} = 0$ by symmetry) or because the spins don't match.

This is the simple case. The two-body expansion is where the subtlety lives.

Two-Body Expansion

The spin-orbital two-body integral in physicist's notation is:

$$\langle pq \mid rs \rangle_{\text{spin}} = \left[\frac{p}{2}\frac{r}{2} \middle| \frac{q}{2}\frac{s}{2} \right]_{\text{spatial}} \times \delta(\sigma_p, \sigma_r) \times \delta(\sigma_q, \sigma_s)$$

where we've used the conversion $\langle pq \mid rs \rangle = [pr \mid qs]$ from Chapter 2 to express the result in terms of chemist's spatial integrals.

The key insight: **each electron independently conserves its spin**. Electron 1 (described by indices p and r) must have the same spin in both orbitals. Electron 2 (indices q and s) must also conserve its spin. But the two electrons **need not have the same spin as each other**.

This means there are **four spin blocks** that contribute non-zero integrals:

Block	Electron 1	Electron 2	Contributes?
$\alpha\alpha - \alpha\alpha$	$\sigma_p = \sigma_r = \alpha$	$\sigma_q = \sigma_s = \alpha$	▯
$\beta\beta - \beta\beta$	$\sigma_p = \sigma_r = \beta$	$\sigma_q = \sigma_s = \beta$	▯
$\alpha\beta - \alpha\beta$	$\sigma_p = \sigma_r = \alpha$	$\sigma_q = \sigma_s = \beta$	▯
$\beta\alpha - \beta\alpha$	$\sigma_p = \sigma_r = \beta$	$\sigma_q = \sigma_s = \alpha$	▯

All four blocks contribute. The cross-spin blocks ($\alpha\beta$ and $\beta\alpha$) are the ones that beginners most commonly forget.

> **Common Mistake #1: Omitting cross-spin integrals.** If you include only the same-spin blocks ($\alpha\alpha$ and $\beta\beta$) and forget the cross-spin blocks, your encoded Hamiltonian will contain only Z-type (diagonal) Pauli terms and no XX/YY excitation terms. The Hamiltonian will commute with all computational basis states, meaning it cannot produce entanglement or represent any quantum correlation. The eigenvalues will be the Hartree–Fock values, not the exact (full-CI) values. This was our actual first-implementation bug.

Worked example: a cross-spin integral

Consider $\langle 0\alpha,\, 1\beta \mid 0\alpha,\, 1\beta\rangle$. In our index scheme: $p = 0, q = 3, r = 0, s = 3$.

Check spin conservation: - Electron 1: $\sigma_p = \alpha$, $\sigma_r = \alpha$ → same spin ▯ - Electron 2: $\sigma_q = \beta$, $\sigma_s = \beta$ → same spin ▯

Spatial indices: $\lfloor p/2 \rfloor = 0$, $\lfloor q/2 \rfloor = 1$, $\lfloor r/2 \rfloor = 0$, $\lfloor s/2 \rfloor = 1$.

Convert to chemist's spatial: $[\lfloor p/2 \rfloor \lfloor r/2 \rfloor \mid \lfloor q/2 \rfloor \lfloor s/2 \rfloor] = [00 \mid 11] = 0.6636$ Ha.

This integral represents the Coulomb repulsion between a spin-α electron in σ_g and a spin-β electron in σ_u. It has nothing to do with spin flips — both electrons keep their original spins — but it is a *cross-spin interaction* because the two electrons have different spins.

The Complete Integral Tables

For reference, here are all the inputs to the encoding pipeline.

Molecular Parameters

Parameter	Value
Bond length R	0.7414 Å = 1.401 Bohr
Nuclear repulsion V_{nn}	0.7151043391 Ha
Spatial orbitals	2 (σ_g, σ_u)

Parameter	Value
Spin-orbitals	4
Electrons	2
Two-electron configurations	$\binom{4}{2} = 6$

Spatial One-Body Integrals h_{pq} (Ha)

	$q = 0$ (σ_g)	$q = 1$ (σ_u)
$p = 0$	-1.2563390730	0
$p = 1$	0	-0.4718960244

Spatial Two-Body Integrals $[pq \mid rs]$ (Ha)

Chemist's integral	Value	Physicist's equivalent
$[00 \mid 00]$	0.6744887663	$\langle 00 \mid 00 \rangle$
$[11 \mid 11]$	0.6973979495	$\langle 11 \mid 11 \rangle$
$[00 \mid 11] = [11 \mid 00]$	0.6636340479	$\langle 01 \mid 01 \rangle = \langle 10 \mid 10 \rangle$
$[01 \mid 10] = [10 \mid 01]$	0.1809312700	$\langle 01 \mid 10 \rangle = \langle 10 \mid 01 \rangle$
$[01 \mid 01] = [10 \mid 10]$	0.6975782469	$\langle 00 \mid 11 \rangle = \langle 11 \mid 00 \rangle$

All other spatial two-body integrals are zero.

> **Note:** We show both conventions side by side so you can verify the conversion $\langle pq \mid rs \rangle = [pr \mid qs]$ from Chapter 2 on every line. If any entry surprises you, stop and work through the index shuffle until it doesn't.

From Tables to Code

With the integral tables complete, we can build the coefficient factory that FockMap's Hamiltonian construction functions consume:

```
open System.Numerics
open Encodings

let h2Integrals = Map [
    // One-body (spin-orbital, interleaved indexing)
    ("0,0", Complex(-1.2563390730, 0.0))
    ("1,1", Complex(-1.2563390730, 0.0))
    ("2,2", Complex(-0.4718960244, 0.0))
    ("3,3", Complex(-0.4718960244, 0.0))
```

```
    // Two-body (physicist's convention, spin-orbital)
    // Same-spin αα-αα
    ("0,0,0,0", Complex(0.6744887663, 0.0))
    ("2,2,2,2", Complex(0.6973979495, 0.0))
    ("0,2,2,0", Complex(0.1809312700, 0.0))
    ("2,0,0,2", Complex(0.1809312700, 0.0))
    ("0,2,0,2", Complex(0.6975782469, 0.0))
    ("2,0,2,0", Complex(0.6975782469, 0.0))
    ("0,0,2,2", Complex(0.6636340479, 0.0))
    ("2,2,0,0", Complex(0.6636340479, 0.0))

    // Same-spin ββ-ββ (identical values, odd indices)
    ("1,1,1,1", Complex(0.6744887663, 0.0))
    ("3,3,3,3", Complex(0.6973979495, 0.0))
    ("1,3,3,1", Complex(0.1809312700, 0.0))
    ("3,1,1,3", Complex(0.1809312700, 0.0))
    ("1,3,1,3", Complex(0.6975782469, 0.0))
    ("3,1,3,1", Complex(0.6975782469, 0.0))
    ("1,1,3,3", Complex(0.6636340479, 0.0))
    ("3,3,1,1", Complex(0.6636340479, 0.0))

    // Cross-spin αβ-αβ (electron 1 = α, electron 2 = β)
    ("0,1,0,1", Complex(0.6744887663, 0.0))
    ("0,3,0,3", Complex(0.6636340479, 0.0))
    ("2,1,2,1", Complex(0.6636340479, 0.0))
    ("2,3,2,3", Complex(0.6973979495, 0.0))
    ("0,1,2,3", Complex(0.6975782469, 0.0))
    ("2,3,0,1", Complex(0.6975782469, 0.0))
    ("0,3,2,1", Complex(0.1809312700, 0.0))
    ("2,1,0,3", Complex(0.1809312700, 0.0))

    // Cross-spin βα-βα (electron 1 = β, electron 2 = α)
    ("1,0,1,0", Complex(0.6744887663, 0.0))
    ("1,2,1,2", Complex(0.6636340479, 0.0))
    ("3,0,3,0", Complex(0.6636340479, 0.0))
    ("3,2,3,2", Complex(0.6973979495, 0.0))
    ("1,0,3,2", Complex(0.6975782469, 0.0))
    ("3,2,1,0", Complex(0.6975782469, 0.0))
    ("1,2,3,0", Complex(0.1809312700, 0.0))
    ("3,0,1,2", Complex(0.1809312700, 0.0))
]

let h2Factory key = h2Integrals |> Map.tryFind key

// Build the 4-qubit H₂ Hamiltonian with Jordan-Wigner
let h2Hamiltonian = computeHamiltonianWith jordanWignerTerms h2Factory 4u
```

This is our first substantial code block — the coefficient factory that will feed the entire encoding pipeline. Notice the pattern: we built the chemistry understanding (Chapter 1), pinned down the notation (Chapter 2), expanded to spin-orbitals (this chapter), and only now convert to code. The code is evidence that the tables are correct, not a substitute for understanding them.

A Preview of What's Coming

The `h2Hamiltonian` produced by the code above is a `PauliRegisterSequence` — a symbolic sum of Pauli strings with complex coefficients. It will have 15 terms (the famous H_2 Hamiltonian that appears in virtually every quantum computing tutorial). We will derive those 15 terms step by step in Chapter 5.

But first, Chapter 4 will introduce the encoding itself: what is a Pauli string, why can't we just use the occupation vectors directly, and how does Jordan–Wigner inject the fermion signs that qubits don't naturally produce?

Key Takeaways

- Each spatial orbital produces two spin-orbitals (α and β). The spin-orbital index space is twice the spatial index space.
- One-body integrals require same-spin: $h^{\text{spin}}_{pq} = h^{\text{spatial}}_{\lfloor p/2 \rfloor, \lfloor q/2 \rfloor} \times \delta(\sigma_p, \sigma_q)$.
- Two-body integrals require each electron to independently conserve spin: $\delta(\sigma_p, \sigma_r) \times \delta(\sigma_q, \sigma_s)$. This admits **four** spin blocks, including the cross-spin blocks $\alpha\beta$ and $\beta\alpha$.
- Omitting cross-spin integrals is the most common implementation error at this stage. It produces a diagonal-only Hamiltonian that cannot represent quantum correlations.
- FockMap expects spin-orbital integrals in physicist's convention with interleaved indexing.

Common Mistakes

1. **Omitting cross-spin blocks.** The $\alpha\beta$ and $\beta\alpha$ blocks produce the off-diagonal Pauli terms (XX, YY) that create entanglement. Without them, your quantum simulation is classical. **This is the single most common implementation failure at the integral stage** — if your encoded Hamiltonian has no off-diagonal terms, check the cross-spin blocks first.

2. **Wrong spin-orbital ordering.** Interleaved $(0\alpha, 0\beta, 1\alpha, 1\beta)$ vs blocked $(0\alpha, 1\alpha, 0\beta, 1\beta)$ indexing produces different integral tables. The eigenvalues are independent of indexing, but the Pauli string structure changes. Always check your convention.

3. **Spatial-to-physicist conversion errors.** This chapter combines two index transformations: spatial→spin-orbital *and* chemist→physicist. It's easy to get one right and the other wrong. Use the dual-convention table above as a cross-check.

Exercises

1. **Cross-spin counting.** How many non-zero spin-orbital two-body integrals does H_2/STO-3G have in total? How many of those are cross-spin? (Answer: 32 total, 16 cross-spin — exactly half.)
2. **Missing block detection.** If you compute the H_2 Hamiltonian without the cross-spin blocks, how many Pauli terms do you get? What types of Pauli operators appear? (Hint: only I and Z.)
3. **Blocked indexing.** Rewrite the one-body spin-orbital table using blocked indexing ($0\alpha, 1\alpha, 0\beta, 1\beta \to$ indices 0, 1, 2, 3). Which entries of the matrix change? Do the eigenvalues change?
4. **H_2O scale-up.** Water has 7 spatial orbitals in STO-3G (after frozen core), giving 14 spin-orbitals. How many one-body spin-orbital integrals can be non-zero? How many two-body? (Answer: up to $14^2 = 196$ one-body, up to $14^4 = 38{,}416$ two-body — though symmetry reduces these significantly.)

Further Reading

- Szabo, A. and Ostlund, N. S. *Modern Quantum Chemistry*. §2.3 covers the spin-orbital expansion in detail.
- Crawford, T. D. and Schaefer, H. F. "An Introduction to Coupled Cluster Theory for Computational Chemists." *Reviews in Computational Chemistry*, Vol. 14, 2000. Appendix A gives explicit spin-orbital integral formulae.

Previous: Chapter 2 — The Notation Minefield

Next: Chapter 4 — The Quantum Computer's Vocabulary

Chapter 4: The Quantum Computer's Vocabulary

We've spent three chapters learning what the molecule wants to say. Now we need to learn what the quantum computer can hear.

In This Chapter

- **What you'll learn:** What qubits are and why they're powerful, what operations you can perform on them, why creating entanglement is the hard part, and why Pauli operators and CNOT gates are sufficient for everything we need.
- **Why this matters:** The encoding (next chapter) translates fermionic operators into qubit operations. If you don't understand the target language, you can't evaluate the translation.
- **Prerequisites:** Chapters 1–3 (you have the molecular Hamiltonian). Basic familiarity with 2×2 matrices is helpful but not essential.

Two Worlds, One Problem

At this point in the story, we have two things that don't talk to each other.

On the **chemistry side**: a Hamiltonian written as a polynomial in creation and annihilation operators, $\hat{H} = \sum h_{pq} a_p^\dagger a_q + \ldots$, parameterized by molecular integrals. Finding its ground-state energy requires exploring an exponentially large space of electron configurations.

On the **computing side**: a quantum processor made of qubits — two-level quantum systems that can be in superposition and can be entangled with each other.

The promise of quantum simulation is that the second can solve the first. But to see *how*, we need to understand what a quantum computer can actually do — not in the abstract "it tries all possibilities at once" sense (which is wrong), but concretely: what are the elementary operations, what do they cost, and are they enough?

Qubits: The Power of Superposition

A classical bit is 0 or 1. End of story.

A **qubit** can be in any superposition of $|0\rangle$ and $|1\rangle$:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$$

where α and β are complex amplitudes with $|\alpha|^2 + |\beta|^2 = 1$. When you measure the qubit, you get 0 with probability $|\alpha|^2$ and 1 with probability $|\beta|^2$. The superposition collapses — you can't read out both amplitudes.

This seems limiting: if measurement destroys the superposition, what good is it? The answer is **interference**. By carefully manipulating the amplitudes before measurement, you can arrange for wrong answers to cancel (destructive interference) and correct answers to reinforce (constructive interference). This is how every quantum algorithm works — not by trying all possibilities, but by engineering interference to amplify the right answer.

For our purposes, the key fact is this: an n-qubit register stores 2^n complex amplitudes. For 4 qubits: 16 amplitudes — matching the Hilbert space of our H_2 problem. For 100 qubits: $2^{100} \approx 10^{30}$ amplitudes. A classical computer cannot even *store* this state, let alone manipulate it. A quantum computer encodes it naturally in the physical state of 100 two-level systems.

Operations on a Single Qubit

If a qubit is a vector in a 2D complex space, an operation on a qubit is a 2×2 unitary matrix. The universe of single-qubit operations is rich — any unitary matrix is a valid gate — but a few show up constantly:

The Pauli operators — you've already met them from the chemistry side (Chapter 1's Hamiltonian lives in Pauli space after encoding). Now meet them as gates:

- X (bit-flip): swaps $|0\rangle$ and $|1\rangle$. The quantum analogue of a NOT gate.
- Z (phase-flip): leaves $|0\rangle$ alone, applies a minus sign to $|1\rangle$. This is a purely quantum operation — it changes the *phase* of the superposition without changing the measurement probabilities.
- $Y = iXZ$: both a bit-flip and a phase-flip.

Why are the Paulis special? Because they form a **basis** for all 2×2 Hermitian matrices. Any single-qubit operator — any observable, any Hamiltonian term — can be written as a linear combination of I, X, Y, Z. This is why the encoded Hamiltonian is a sum of Pauli strings: it's the natural basis for qubit operators, just as $\{|0\rangle, |1\rangle\}$ is the natural basis for qubit states.

The Hadamard gate (H) creates superposition from a definite state:

$$H|0\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}, \qquad H|1\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}}$$

This is the gate that takes a classical bit and makes it quantum. Without H (or something equivalent), you could never create superpositions and quantum computing would reduce to classical computing.

Rotation gates ($R_z(\theta)$, $R_x(\theta)$) rotate the qubit's state by a continuous angle θ. These are the gates that implement the actual physics: the Hamiltonian's coefficients (like -0.1744 Ha for the exchange terms) become rotation angles in the circuit.

On current quantum processors — IBM's Eagle and Heron chips (superconducting transmon qubits), IonQ's Aria (trapped ytterbium ions), Quantinuum's H-series (trapped calcium ions) — single-qubit gates take 20–100 nanoseconds with error rates around 10^{-4}. They are the cheapest thing a quantum computer does. For cost accounting, we treat them as essentially free; the expensive operations are the two-qubit gates we're about to meet.

The Hard Part: Entanglement

Single-qubit operations are not enough. If each qubit could only be manipulated independently, a quantum computer would be nothing more than n independent coin-flips — powerful for random sampling, useless for computation.

The power of quantum computing comes from **entanglement**: correlations between qubits that have no classical analogue.

Consider two qubits in the state $\frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$. If you measure the first qubit and get 0, the second qubit is *guaranteed* to be 0 — even though, before measurement, neither qubit had a definite value. This correlation cannot be explained by any shared classical information prepared in advance (this is the content of Bell's theorem, which we won't prove but which decades of experiments have confirmed).

Entanglement is what allows a quantum computer to explore the exponentially large Hilbert space *coherently* — maintaining phase relationships between configurations rather than just sampling them independently. And it is what allows the correlation energy (Chapter 5's off-diagonal coherences in the density matrix) to be represented on quantum hardware.

The CNOT gate: the entanglement maker

The **controlled-NOT** (CNOT) gate is the simplest operation that creates entanglement. It acts on two qubits — a *control* and a *target* — and flips the target if the control is $|1\rangle$:

$$\text{CNOT}|00\rangle = |00\rangle, \quad \text{CNOT}|01\rangle = |01\rangle, \quad \text{CNOT}|10\rangle = |11\rangle, \quad \text{CNOT}|11\rangle = |10\rangle$$

On its own, CNOT doesn't look quantum — it's just a conditional flip. But apply it after a Hadamard:

$$|00\rangle \xrightarrow{H\otimes I} \frac{|0\rangle + |1\rangle}{\sqrt{2}} \otimes |0\rangle \xrightarrow{\text{CNOT}} \frac{|00\rangle + |11\rangle}{\sqrt{2}}$$

Two gates — one Hadamard, one CNOT — and we've created a maximally entangled state from a product state. No classical operation can do this.

Why CNOT is expensive

On physical quantum processors, the CNOT gate is the bottleneck. Here's what it actually costs on current hardware:

Platform	Vendor	CNOT time	CNOT error rate	Single-qubit error rate
Superconducting transmon	IBM (Heron)	~300 ns	$\sim 5 \times 10^{-3}$	$\sim 3 \times 10^{-4}$
Trapped ion	Quantinuum (H2)	~2 ms	$\sim 2 \times 10^{-3}$	$\sim 10^{-5}$
Trapped ion	IonQ (Forte)	~600 µs	$\sim 4 \times 10^{-3}$	$\sim 3 \times 10^{-4}$

The pattern: CNOT is **10–100× slower** and **10–100× noisier** than single-qubit gates, regardless of the hardware platform. The physics is different in each case — microwave cross-resonance pulses for superconducting qubits, laser-driven Mølmer–Sørensen interactions for trapped ions — but the engineering reality is the same: making two quantum systems interact controllably is harder than manipulating one.

There is also a **connectivity constraint**. On many architectures (especially superconducting), CNOT is only available between physically adjacent qubits. If you need to entangle qubit 0 with qubit 15, you must first "swap" the quantum state through the intervening qubits — and each swap costs 3 CNOTs.

> **If you don't have access to quantum hardware**, you can still run everything in this book using a **quantum simulator**. The Quokka (by Eigensystems, co-founded by Dr Chris Ferrie at UTS) is a dedicated quantum simulation appliance — available as a standalone device or accessible over the web — that can simulate up to ~30 qubits. That's more than enough for H_2 (4 qubits), H_2O (12 qubits), and most textbook-scale problems. IBM's Qiskit Aer and Google's Cirq also provide free software simulators. The circuits we generate in Stage 5 can target any of these backends.
>
> The CNOT cost analysis is still relevant for simulated circuits: it tells you how the circuit *would* perform on real hardware, and whether your molecule is within reach of current or near-term quantum processors.
>
> **The bottom line:** Single-qubit gates are cheap. CNOT is expensive. Circuit cost ≈ CNOT count. Everything else follows from this.

Is This Enough? Universality

A natural question: are Pauli gates and CNOT sufficient for *all* quantum computation? Or do we need exotic gates that we haven't mentioned?

The answer is yes, they are sufficient — and this is a theorem.

Any unitary operation on n qubits can be decomposed into a sequence of: - **Single-qubit rotations** ($R_x(\theta)$, $R_y(\theta)$, $R_z(\theta)$ for arbitrary θ) - **CNOT gates**

This is called **universality**. The set $\{R_z, R_x, \text{CNOT}\}$ (or equivalently $\{H, T, \text{CNOT}\}$, where T is a $\pi/8$ rotation) can approximate any quantum operation to arbitrary precision. Every quantum algorithm — Shor's, Grover's, VQE, QPE — can be compiled down to these elementary gates.

For quantum simulation specifically, we need even less. The Hamiltonian is a sum of Pauli strings, and each Pauli string's time evolution $e^{-i\theta P}$ decomposes into: - A few single-qubit gates (to rotate into the right basis) - A chain of CNOTs (to entangle the relevant qubits) - One R_z rotation (to apply the phase) - The reverse of the above (to undo the basis change and entanglement)

This decomposition — the **CNOT staircase** — is the circuit that appears for every term in the Hamiltonian. Its cost is $2(w-1)$ CNOT gates, where w is the Pauli weight of the term. We will develop it in full detail in Stage 4 (Trotterization).

For now, the key formula is:

$$\boxed{\text{CNOTs per Pauli rotation} = 2(w-1)}$$

This single equation connects everything: the molecular integrals (which determine the Hamiltonian terms), the encoding (which determines the Pauli weight w), and the circuit cost (which determines whether the simulation is feasible).

The Cost Table

To make this concrete:

Pauli weight w	CNOTs	Example
1	0	Single Z — just an R_z
2	2	ZZ — one CNOT, one R_z, one reverse CNOT
4	6	$XXYY$ — the H_2 exchange terms
12	22	Typical JW term for H_2O
100	198	JW term for FeMo-co — infeasible
5	8	Same FeMo-co term under ternary tree — feasible

The last two rows are the entire argument for encoding choice, in two numbers.

Key Takeaways

- A **qubit** stores a superposition of $|0\rangle$ and $|1\rangle$. An n-qubit register stores 2^n amplitudes — an exponentially large state space.
- **Pauli operators** (I, X, Y, Z) form a basis for all single-qubit operations. Every Hamiltonian term can be written as a Pauli string.
- **CNOT** is the gate that creates entanglement — and it is 10–100× more expensive than single-qubit gates. Circuit cost ≈ CNOT count.
- The set $\{R_z, R_x, \text{CNOT}\}$ is **universal**: it can implement any quantum operation, including everything we need for quantum simulation.
- The **CNOT staircase** decomposes a Pauli rotation into $2(w-1)$ CNOTs, where w is the Pauli weight. This is the conversion factor from encoding choice to circuit cost.

  Multi-qubit states and tensor products: An n-qubit state is described by the **tensor product** (Kronecker product) of individual qubit states: $|\psi\rangle = |q_0\rangle \otimes |q_1\rangle \otimes \cdots \otimes |q_{n-1}\rangle$. A Pauli string like $X_0 Z_1 I_2$ means "apply X to qubit 0, Z to qubit 1, and I to qubit 2" — formally, the $4 \times 4 \times 2 = 8$-dimensional matrix $X \otimes Z \otimes I$. When we say two operators "commute" ($[A, B] = AB - BA = 0$) or "anti-commute" ($\{A, B\} = AB + BA = 0$), we mean these tensor-product matrices. The encoding problem (next chapter) is precisely the challenge of translating anti-commuting fermionic operators into operators built from commuting qubit tensor products.

Common Mistakes

1. **"A quantum computer tries all possibilities at once."** It doesn't. It maintains a superposition and uses interference to amplify the right answer. The exponential speedup comes from interference, not parallelism.

2. **Counting all gates instead of CNOT gates.** Single-qubit gates are effectively free on modern hardware. Only CNOTs matter for feasibility.

3. **Forgetting that CNOT requires physical connectivity.** On most architectures, you can only CNOT adjacent qubits. Distant interactions cost additional swaps (3 CNOTs each).

Exercises

1. **Bell state.** Starting from $|00\rangle$, apply H to qubit 0, then CNOT(0→1). Write out the resulting state. Why is it entangled? (Hint: can you write it as a product $|\psi_1\rangle \otimes |\psi_2\rangle$?)

2. **CNOT cost.** A Hamiltonian has 100 terms. Under JW, 20 of them have weight 50 and the rest have weight 2. Under ternary tree, the 20 heavy terms have weight 5 and the rest still have weight 2. What are the total CNOT counts per Trotter step for each encoding?

3. **Universality.** Why can't you build a useful quantum computer using only single-qubit gates? What computation would it be equivalent to?

Further Reading

- Nielsen, M. A. and Chuang, I. L. *Quantum Computation and Quantum Information.* Cambridge UP, 2000. The standard reference — we've barely scratched the surface.
- Preskill, J. "Quantum Computing in the NISQ Era and Beyond." *Quantum* 2, 79 (2018). Why gate count matters on near-term hardware.
- Mermin, N. D. *Quantum Computer Science: An Introduction.* Cambridge UP, 2007. A gentler introduction to the gate model, written for computer scientists.

Previous: Chapter 3 — From Spatial to Spin-Orbital Integrals

Next: Chapter 5 — A Visual Guide to Encodings

Chapter 5: A Visual Guide to Encodings

We have a Hamiltonian written in terms of electrons and orbitals. A quantum computer only has qubits. This chapter explains why the translation is not as obvious as it looks, and introduces three ways to do it.

In This Chapter

- **What you'll learn:** Why occupation vectors cannot be mapped directly to qubits, how the Jordan–Wigner encoding solves the problem but at a cost, and how Bravyi–Kitaev and tree-based encodings reduce that cost.
- **Why this matters:** The encoding choice directly determines circuit depth, CNOT count, and measurement cost — and therefore whether a quantum simulation is feasible on a given device.
- **Prerequisites:** Chapters 1–3 (you know the second-quantized Hamiltonian and the spin-orbital integral tables).

Where We Stand

Let's take stock of what the first three chapters gave us.

Chapter 1 told us the chemistry: a molecule is a collection of charged particles, and its ground-state energy is the lowest eigenvalue of the electronic Hamiltonian. We introduced a finite basis set (STO-3G) that turned the continuous problem into a finite one: 2 spatial orbitals, 4 spin-orbitals, 6 two-electron configurations.

Chapters 2–3 told us the numbers: one-body integrals h_{pq}, two-body integrals $\langle pq \mid rs \rangle$, and the nuclear repulsion constant V_{nn}. We navigated the notation minefield, expanded to spin-orbitals, and ended up with a complete set of tables.

But we glossed over something important. Let's go back and address it now.

The problem with wavefunctions

The *natural* way to describe the quantum state of two electrons in four spin-orbitals is as a superposition of configurations:

$$|\Psi\rangle = c_{1100}|1100\rangle + c_{1010}|1010\rangle + c_{1001}|1001\rangle + \cdots$$

where each $|n_0 n_1 n_2 n_3\rangle$ is an **occupation vector** — a list of 0s and 1s telling us which spin-orbitals have an electron and which don't. The c coefficients are complex amplitudes. Finding the ground state means finding the c values that minimize the energy.

For H_2 with 4 spin-orbitals, this is a vector with 6 components (the 6 two-electron configurations from Chapter 1). We could write down the 6×6 Hamiltonian matrix, plug in our integrals, and diagonalize it. Done. No quantum computer needed.

For H_2O with 14 spin-orbitals and 10 electrons, the number of configurations is $\binom{14}{10} = 1{,}001$. A $1{,}001 \times 1{,}001$ matrix is trivial for a laptop.

For a modest catalyst with 50 electrons in 100 spin-orbitals: $\binom{100}{50} \approx 10^{29}$ configurations. That matrix will not fit in any computer that will ever be built.

This is the **exponential wall**: the dimension of the configuration space grows combinatorially with the number of orbitals. Classical methods (Hartree–Fock, DFT, coupled cluster) get around this by approximating — they never build the full matrix. But those approximations break down for strongly correlated systems, which are precisely the systems where quantum simulation promises an advantage.

Why operators instead of wavefunctions

Rather than tracking the full wavefunction vector (exponentially many coefficients), **second quantization** lets us write the Hamiltonian as a polynomial in creation and annihilation operators:

$$\hat{H} = \sum_{pq} h_{pq}\, a_p^\dagger a_q + \frac{1}{2} \sum_{pqrs} \langle pq \mid rs \rangle\, a_p^\dagger a_q^\dagger a_s a_r$$

The operators $a_p^\dagger$ (create an electron in spin-orbital p) and a_p (remove one) obey the **canonical anti-commutation relations**:

$$\{a_p^\dagger, a_q\} = \delta_{pq}, \qquad \{a_p^\dagger, a_q^\dagger\} = 0, \qquad \{a_p, a_q\} = 0$$

These three rules encode the Pauli exclusion principle algebraically: you can't create two electrons in the same orbital ($a_p^\dagger a_p^\dagger = 0$), and swapping the order of two creation operators flips a sign ($a_p^\dagger a_q^\dagger = -a_q^\dagger a_p^\dagger$).

The beauty of this formulation is that the Hamiltonian is written as a sum of $O(n^4)$ terms — polynomial in the number of orbitals, not exponential. A classical computer

can *write down* the Hamiltonian easily. The hard part is *solving* it — finding the eigenvalues of the operator that this polynomial represents.

Why quantum computing

A quantum computer can represent the exponentially large state vector $|\Psi\rangle$ using only n qubits — one per spin-orbital. The 2^n-dimensional Hilbert space of n qubits matches the $\binom{n}{N_e}$-dimensional configuration space (embedded in the full 2^n-dimensional space). Superpositions of configurations are *native* to quantum hardware.

But there's a catch: the operators $a_p^\dagger$ and a_p anti-commute, and qubit operations don't. We need a **translation** — an encoding — that maps the fermionic operators to qubit operators while preserving the anti-commutation relations.

That's what this chapter is about.

The Tempting (Wrong) Idea

At the end of Chapter 1, we noticed something suggestive: the occupation vector $|1010\rangle$ looks exactly like a 4-qubit computational basis state. Four spin-orbitals, four qubits, each qubit storing the occupation of one orbital. Simple.

```
flowchart LR
    subgraph Orbitals["Chemistry World"]
        direction TB
        O0["Orbital 0: ↑"] --- O1["Orbital 1: ∅"]
        O1 --- O2["Orbital 2: ↑"]
        O2 --- O3["Orbital 3: ∅"]
    end

    subgraph Qubits["Quantum Computer"]
        direction TB
        Q0["Qubit 0"] --- Q1["Qubit 1"]
        Q1 --- Q2["Qubit 2"]
        Q2 --- Q3["Qubit 3"]
    end

    Orbitals -- "occupied ↔ |1⟩<br/>empty ↔ |0⟩" --> Qubits
```

This mapping is correct for the **basis states**. But it is wrong for the **operators**.

The creation operator $a_2^\dagger$ does not simply flip qubit 2 from $|0\rangle$ to $|1\rangle$. It must also check how many electrons are in orbitals 0 and 1, and apply a minus sign if that count is odd. This is the **fermion sign** — the physical consequence of the anti-commutation relations from Chapter 1.

Qubits don't anti-commute. If we flip qubit 2 without tracking the fermion sign, we get the wrong quantum state. The encoding exists to inject those signs.

What Makes Electrons Different from Qubits

When we swap two electrons, the quantum state picks up a factor of -1:

$$|A, B\rangle_{\text{fermion}} = -|B, A\rangle_{\text{fermion}}$$

When we swap two qubits, nothing happens to the sign:

$$|0, 1\rangle_{\text{qubit}} = +|0, 1\rangle_{\text{qubit}}$$

This minus sign is not optional bookkeeping. It affects interference patterns, bonding energies, and reaction rates. A faithful encoding must reproduce it.

> **The fermion sign, operationally:** Consider the occupation state $|1010\rangle$ (orbitals 0 and 2 occupied). To create an electron in orbital 3, we apply $a_3^\dagger$. No other occupied orbitals lie between positions 2 and 3 (positions are counted from the right), so the sign is $(-1)^0 = +1$. But to create an electron in orbital 1, we must "pass through" the electron in orbital 2. The anti-commutation rule charges one factor of -1 for each occupied orbital we pass: the sign is $(-1)^1 = -1$. This is the **fermion sign** — the net parity of occupied orbitals below the target position. Every encoding must inject this sign into the qubit representation.

The mathematical statement is the anti-commutation relation:

$$a_p^\dagger a_q^\dagger = -a_q^\dagger a_p^\dagger$$

Creating an electron in orbital p and then orbital q gives the opposite sign from creating in q then p. On a qubit register, flipping qubit p then qubit q gives the *same* result as flipping q then p. The encoding must bridge this gap.

Jordan–Wigner: The Z-Chain

The Jordan–Wigner encoding (1928) is the oldest and simplest solution. Each qubit directly stores the occupation of one orbital — same as the "tempting idea" — but every creation or annihilation operator carries a **chain of Z gates** that enforces the fermion sign.

How the Z-chain works

The Z gate acts on a qubit as a parity detector: - On $|0\rangle$ (empty orbital): Z gives $+1$ - On $|1\rangle$ (occupied orbital): Z gives -1

To create an electron in orbital j, we apply Z to every qubit below j and then flip qubit j. The product of all those Z values gives $(-1)^{\text{count of occupied orbitals below } j}$ — exactly the fermion sign.

```
flowchart LR
    subgraph JW["Jordan-Wigner: create electron in orbital 2"]
        direction LR
        Z0["Qubit 0<br/>Z"] --> Z1["Qubit 1<br/>Z"]
        Z1 --> Flip["Qubit 2<br/>X/Y<br/>(flip)"]
        Flip --> I3["Qubit 3<br/>I<br/>(nothing)"]
    end
    style Z0 fill:#e8ecf1,stroke:#6b7280
    style Z1 fill:#e8ecf1,stroke:#6b7280
    style Flip fill:#fde68a,stroke:#d97706
    style I3 fill:#f3f4f6,stroke:#9ca3af
```

The Z-chain grows linearly: for orbital j, we need j extra Z operations. This is the **Pauli weight** of the encoded operator — the number of qubits it touches.

JW Pauli weight for all four orbitals

```
flowchart TB
    subgraph Create0["a†₀ — weight 1"]
        direction LR
        A0["X/Y"] ~~~ A1["I"] ~~~ A2["I"] ~~~ A3["I"]
    end
    subgraph Create1["a†₁ — weight 2"]
        direction LR
        B0["Z"] ~~~ B1["X/Y"] ~~~ B2["I"] ~~~ B3["I"]
    end
    subgraph Create2["a†₂ — weight 3"]
        direction LR
        C0["Z"] ~~~ C1["Z"] ~~~ C2["X/Y"] ~~~ C3["I"]
    end
    subgraph Create3["a†₃ — weight 4"]
        direction LR
        D0["Z"] ~~~ D1["Z"] ~~~ D2["Z"] ~~~ D3["X/Y"]
    end
```

For a molecule with n spin-orbitals, the worst-case Pauli weight under JW is n. For FeMo-co (the iron-molybdenum cofactor of nitrogenase — the enzyme that fixes atmospheric nitrogen) with ~100 spin-orbitals, that's a chain of 99 Z gates for the last orbital — a very deep circuit on quantum hardware.

When JW is the right choice

Despite the linear scaling, JW excels when interactions are **local** — between neighboring orbitals. A hopping term $a_2^\dagger a_3 + a_3^\dagger a_2$ only needs a short Z-chain because the two orbitals are adjacent. For 1D molecular chains or systems with sequential orbital numbering, JW often has the lowest *average* weight even if its worst case is high.

Bravyi–Kitaev: Partial Parity Sums

The key insight

The JW Z-chain answers the question: "what is the parity (odd/even count) of electrons in orbitals $0, 1, \ldots, j-1$?" It answers this by checking every orbital individually — an $O(n)$ scan.

What if some qubits pre-computed **partial parity sums**? Then we could answer the question by reading $O(\log n)$ qubits instead of $O(n)$.

This is the Bravyi–Kitaev (BK) encoding. It uses a data structure from computer science — the **Fenwick tree** (binary indexed tree) — to organize partial sums.

How BK stores information

In JW, each qubit stores one orbital's occupation:

```
flowchart LR
    subgraph JWStorage["Jordan-Wigner storage"]
        direction LR
        J0["q0<br/>orbital 0"] ~~~ J1["q1<br/>orbital 1"] ~~~ J2["q2<br/>orbital 2"] ~~
    end
```

In BK, some qubits store **cumulative parity** — the XOR of multiple orbital occupations:

```
flowchart LR
    subgraph BKStorage["Bravyi-Kitaev storage (8 orbitals)"]
        direction TB
        Q0["q0: orbital 0"]
        Q1["q1: orbital 0 ⊕ 1"]
        Q2["q2: orbital 2"]
        Q3["q3: orbital 0 ⊕ 1 ⊕ 2 ⊕ 3"]
        Q4["q4: orbital 4"]
        Q5["q5: orbital 4 ⊕ 5"]
        Q6["q6: orbital 6"]
        Q7["q7: orbital 0 ⊕ 1 ⊕ ... ⊕ 7"]
    end
```

The pattern comes from a Fenwick tree. Each node stores the parity of the orbitals in its subtree:

```
graph TD
    Q7["q7<br/>parity 0–7"] --- Q3["q3<br/>parity 0–3"]
    Q7 --- Q5["q5<br/>parity 4–5"]
    Q3 --- Q1["q1<br/>parity 0–1"]
    Q3 --- Q2["q2<br/>orbital 2"]
    Q5 --- Q4["q4<br/>orbital 4"]
    Q5 --- Q6["q6<br/>orbital 6"]
    Q1 --- Q0["q0<br/>orbital 0"]
    style Q7 fill:#e8ecf1,stroke:#6b7280
    style Q3 fill:#e8ecf1,stroke:#6b7280
    style Q5 fill:#e8ecf1,stroke:#6b7280
    style Q1 fill:#e8ecf1,stroke:#6b7280
```

To find the parity of orbitals 0 through $j-1$, you read the nodes on the *path* from node j up to the root — at most $\log_2 n$ nodes.

The three questions every encoding answers

For each orbital j, the encoding must answer three questions:

Question	JW answer	BK answer
Parity: which qubits encode the fermion sign?	All qubits $0 \ldots j-1$ ($O(n)$)	$\leq \log_2 n$ qubits on the Fenwick path
Update: which qubits must change when orbital j is filled/emptied?	Just qubit j	$\leq \log_2 n$ ancestor qubits
Occupation: which qubits reveal whether orbital j is occupied?	Just qubit j	$\leq \log_2 n$ descendant qubits

Every answer in BK involves at most $\log_2 n$ qubits. That's the advantage.

Weight comparison at scale

Spin-orbitals (n)	JW worst-case weight	BK worst-case weight	Ratio
4	4	3	1.3×
8	8	4	2×
16	16	5	3.2×
64	64	7	9×
100	100	7	14×

At 100 spin-orbitals, BK's worst-case operator touches 7 qubits while JW's touches 100. On noisy hardware, this is the difference between a feasible and an infeasible circuit.

Tree Encodings: Going Below $\log_2 n$

Beyond binary

The Fenwick tree is a *binary* tree — each node has at most two children. What if we used a **ternary** tree instead? Each internal node would have three children, the tree would be shallower, and the path lengths (and hence Pauli weights) would be shorter.

This is the insight behind the ternary tree encoding (Jiang et al., 2020). It achieves worst-case Pauli weight $O(\log_3 n)$ — the best known asymptotic scaling for any fermion-to-qubit encoding.

```
graph TD
    R["Root"] --- C1["Child 1"]
    R --- C2["Child 2"]
    R --- C3["Child 3"]
    C1 --- L1["Orbital"]
    C1 --- L2["Orbital"]
    C1 --- L3["Orbital"]
    C2 --- L4["Orbital"]
    C2 --- L5["Orbital"]
    C2 --- L6["Orbital"]
    C3 --- L7["Orbital"]
    C3 --- L8["Orbital"]
    C3 --- L9["Orbital"]
    style R fill:#e8ecf1,stroke:#6b7280
    style C1 fill:#e8ecf1,stroke:#6b7280
    style C2 fill:#e8ecf1,stroke:#6b7280
    style C3 fill:#e8ecf1,stroke:#6b7280
```

The three edges emerging from each node are labelled with the three non-identity Pauli operators: X, Y, and Z. This labelling is what gives the encoding its structure — the Pauli string for each orbital is read off by collecting edge labels on the path from root to leaf.

FockMap supports both balanced binary and balanced ternary tree encodings, plus **arbitrary** tree topologies — you can define your own tree shape and derive a valid encoding from it.

The Complete Picture

```
flowchart TD
    MO["Molecular integrals<br/>(from PySCF, Gaussian, etc.)"]
    MO --> HAM["Second-quantized Hamiltonian<br/>H = Σ hₚₑ a†ₚaₑ + ½Σ ⟨pq|rs⟩ a†ₚa†ₑaₛa
```

```
    HAM --> ENC{"Choose an encoding"}
    ENC -->|"Simple / small n"| JW["Jordan-Wigner<br/>O(n) weight"]
    ENC -->|"General purpose"| BK["Bravyi-Kitaev<br/>O(log₂ n) weight"]
    ENC -->|"Best scaling"| TT["Ternary Tree<br/>O(log₃ n) weight"]
    JW --> PAULI["Pauli Hamiltonian<br/>H = Σ cₖ Pₖ"]
    BK --> PAULI
    TT --> PAULI
    PAULI --> TAP["Tapering<br/>(remove redundant qubits)"]
    TAP --> TROT["Trotterization<br/>(Hamiltonian → circuit)"]
    TROT --> QC["Quantum hardware"]
    style ENC fill:#fde68a,stroke:#d97706
    style PAULI fill:#d1fae5,stroke:#059669
```

All five encodings in FockMap (JW, BK, Parity, balanced binary tree, balanced ternary tree) produce the same output type: a `PauliRegisterSequence`. They all give the same eigenvalues — the same physics. They differ only in how many qubits each Pauli string touches, which determines circuit depth and measurement cost.

One More Thing: Number Operators Stay Cheap

A reassuring fact: the **number operator** $n_j = a_j^\dagger a_j$ (which tests whether orbital j is occupied) simplifies to just two terms under *every* encoding:

$$n_j \;\mapsto\; \frac{1}{2}(I - Z_j) \quad \text{under JW}$$

That's Pauli weight 1 — regardless of n, regardless of encoding. The Z-chains from the creation and annihilation operators cancel in the product.

So while *creating* or *moving* electrons gets expensive under JW (the Z-chains grow), simply *counting* them stays cheap in all encodings. The overhead only hits terms that change the electron configuration — and those are precisely the terms that represent quantum correlations.

Choosing an Encoding: A Decision Guide

If your situation is…	Use…	Why
Small molecule ($n \leq 20$)	Jordan–Wigner	Simplest, and the weight overhead is manageable
1D chain / local Hamiltonian	Jordan–Wigner	Adjacent orbitals have short Z-chains

If your situation is…	Use…	Why
Medium molecule ($20 < n \leq 100$)	Bravyi–Kitaev	$O(\log_2 n)$ weight saves significant circuit depth
Large molecule / all-to-all interactions	Ternary Tree	$O(\log_3 n)$ — the best known scaling
Research / exploring custom topologies	Custom tree	FockMap supports arbitrary tree shapes

In the next chapter, we will use all five encodings to build the complete 15-term H_2 Hamiltonian — and see that they all agree.

Key Takeaways

- Electrons anti-commute; qubits don't. An encoding bridges this gap by injecting fermion signs into the qubit representation.
- **Jordan–Wigner** uses a linear Z-chain: simple but $O(n)$ weight.
- **Bravyi–Kitaev** uses a Fenwick tree to pre-compute partial parities: $O(\log_2 n)$ weight.
- **Ternary tree** encodings go further: $O(\log_3 n)$ weight — the best known scaling.
- All encodings produce the same physics (same eigenvalues). The choice affects only circuit cost.
- Number operators are cheap ($O(1)$) in all encodings.

Common Mistakes

1. **Forgetting fermion signs entirely.** Mapping occupation directly to qubits without an encoding gives classical (Hartree–Fock) results, not quantum results. The encoding is not optional — it *is* the quantum part.

2. **Assuming JW is always the worst.** For small systems and local interactions, JW often has the lowest *average* weight. The $O(n)$ scaling only becomes problematic at larger n.

3. **Comparing raw term counts instead of Pauli weight.** All encodings produce the same number of Hamiltonian terms. The difference is in the weight (number of non-identity Paulis) per term, which determines CNOT count.

Exercises

1. **Z-chain length.** For a system with 10 spin-orbitals, what is the JW Pauli weight of $a_7^\dagger$? What is the BK worst-case weight? (Use $\lfloor \log_2 10 \rfloor + 1$.)

2. **Number operator.** Verify that the JW number operator $n_j = \frac{1}{2}(I - Z_j)$ has eigenvalues 0 and 1 by applying it to $|0\rangle$ and $|1\rangle$.

3. **Tree branching.** If you used a 4-ary (quaternary) tree instead of ternary, what would the worst-case weight scaling be? Would it be better or worse than ternary, and why might it not work as easily?

Further Reading

- Jordan, P. and Wigner, E. "Über das Paulische Äquivalenzverbot." *Z. Physik* 47, 631 (1928). The original encoding — a mathematical trick for mapping spin chains that found its second life in quantum computing 77 years later.
- Aspuru-Guzik, A., Dutoi, A. D., Love, P. J., and Head-Gordon, M. "Simulated Quantum Computation of Molecular Energies." *Science* 309, 1704 (2005). The paper that launched quantum computational chemistry: showed that the Jordan–Wigner encoding could be plugged into a quantum phase estimation algorithm to compute molecular ground-state energies. Before this work, JW was a mathematical curiosity; after it, an entire field was born.
- Seeley, J. T., Richard, M. J., and Love, P. J. "The Bravyi–Kitaev transformation for quantum computation of electronic structure." *J. Chem. Phys.* 137, 224109 (2012). Formalized the **index-set framework** — the three set-valued functions (Update, Parity, Occupation) that unify JW, BK, and Parity under a single abstraction. This is the framework that FockMap's `EncodingScheme` type directly implements.
- Bravyi, S. B. and Kitaev, A. Yu. "Fermionic Quantum Computation." *Ann. Phys.* 298, 210 (2002). The logarithmic-weight encoding using Fenwick trees.
- Jiang, Z. et al. "Optimal fermion-to-qubit mapping via ternary trees." *PRX Quantum* 1, 010306 (2020). The ternary tree encoding with optimal asymptotic scaling.

Previous: Chapter 4 — The Quantum Computer's Vocabulary

Next: Chapter 6 — Building the Qubit Hamiltonian

Chapter 6: Building the Qubit Hamiltonian

This is where the pipeline pays off. We take the integral tables from Chapter 3, the encoding from Chapter 5, and produce the actual qubit Hamiltonian — the object that a quantum computer will simulate.

In This Chapter

- **What you'll learn:** Why the distinction between diagonal and off-diagonal Hamiltonian terms is the central idea of quantum simulation, how to systematically encode fermionic terms into Pauli strings, and how to read the resulting 15-term H_2 Hamiltonian as a map of classical vs quantum physics.
- **Why this matters:** This is the exact object used by VQE, QPE, and every other quantum simulation algorithm. Understanding its structure — which parts are classical, which parts are quantum — is the key to understanding why quantum computing matters for chemistry.
- **Prerequisites:** Chapters 1–5 (integrals, notation, spin-orbitals, gates, encoding concepts).

The Density Matrix: Where Classical Ends and Quantum Begins

Before we compute anything, we need to understand what we're looking for — because the structure of the qubit Hamiltonian will tell us exactly where classical physics ends and quantum physics begins.

Classical: a probability distribution

If electrons were classical particles, we could describe their state as a **probability distribution** over configurations:

$$\text{Prob}(|1100\rangle) = 0.95, \quad \text{Prob}(|0011\rangle) = 0.03, \quad \text{Prob}(|1010\rangle) = 0.02, \quad \ldots$$

This tells us: "if we look, there's a 95% chance both electrons are in the bonding orbital, a 3% chance both are in the antibonding orbital," and so on. The expected energy is just a weighted average: $E_{\text{classical}} = \sum_i p_i E_i$.

Mathematically, this is a **diagonal** density matrix — numbers along the diagonal (the probabilities) and zeros everywhere else:

$$\rho_{\text{classical}} = \begin{pmatrix} 0.95 & 0 & 0 & \cdots \\ 0 & 0.03 & 0 & \cdots \\ 0 & 0 & 0.02 & \cdots \\ \vdots & \vdots & \vdots & \ddots \end{pmatrix}$$

Hartree–Fock is essentially this picture: the electrons are in $|1100\rangle$ with probability 1. One non-zero diagonal entry. No mixing, no uncertainty beyond what's already baked into the orbital structure.

Quantum: coherences change everything

A quantum state is fundamentally different. The ground state of H_2 is not "95% chance of $|1100\rangle$." It is a **superposition**:

$$|\Psi\rangle = c_1|1100\rangle + c_2|0011\rangle + \cdots$$

The density matrix $\rho = |\Psi\rangle\langle\Psi|$ now has off-diagonal elements:

$$\rho_{\text{quantum}} = \begin{pmatrix} |c_1|^2 & c_1 c_2^* & \cdots \\ c_2 c_1^* & |c_2|^2 & \cdots \\ \vdots & \vdots & \ddots \end{pmatrix}$$

The diagonal entries $|c_i|^2$ are probabilities — same as the classical case. But the off-diagonal entries $c_i c_j^*$ are **coherences**. They have no classical analogue. They encode the *phase relationships* between configurations — the interference patterns that allow the quantum state to have a lower energy than any classical mixture of the same configurations.

> **The off-diagonal elements of the density matrix are the quantum difference.**

A classical mixture with the same occupation probabilities would have a *higher* energy. But why? To see this, we need to understand how energy is actually computed from a density matrix.

The trace: a machine for computing averages

The expected energy of a system in state ρ is:

$$\langle H \rangle = \text{Tr}(\rho H)$$

The **trace** of a matrix is the sum of its diagonal elements. But $\mathrm{Tr}(\rho H)$ is not just summing the diagonal of ρ — it's summing the diagonal of the *product* ρH. And the product of two matrices mixes their rows and columns:

$$\mathrm{Tr}(\rho H) = \sum_i (\rho H)_{ii} = \sum_i \sum_j \rho_{ij} H_{ji}$$

We can split this into two parts:

$$\langle H \rangle = \underbrace{\sum_i \rho_{ii} H_{ii}}_{\text{classical}} + \underbrace{\sum_{i \neq j} \rho_{ij} H_{ji}}_{\text{quantum}}$$

The first sum is the **classical energy**: each configuration's probability times its diagonal energy. This is what Hartree–Fock computes — a single $\rho_{ii} = 1$, everything else zero, and $\langle H \rangle = H_{ii}$.

The second sum is the **quantum correction**: the off-diagonal elements of ρ (the coherences) multiplied by the off-diagonal elements of H (the exchange terms). This sum can be *negative* — it can lower the energy below the classical minimum. This is the correlation energy.

But notice: for this second sum to be nonzero, **both** ρ and H must have off-diagonal elements. If the Hamiltonian is diagonal (no X or Y terms), then $H_{ji} = 0$ for $i \neq j$, and the quantum correction vanishes regardless of what ρ looks like. If ρ is diagonal (no coherences), the correction vanishes regardless of H.

The trace formula makes the thesis mathematically precise: quantum advantage requires off-diagonal elements in *both* the state and the Hamiltonian, working together through the trace to produce an energy that no diagonal (classical) computation can reach.

What generates coherences?

Diagonal Hamiltonian terms (built from I and Z only) do not mix configurations. They assign energies — "configuration $|1100\rangle$ costs this much, configuration $|0011\rangle$ costs that much." Under time evolution, they rotate phases but never create superpositions. A diagonal Hamiltonian cannot generate off-diagonal density matrix elements. Its ground state is always a single configuration.

Off-diagonal Hamiltonian terms (containing X or Y) *mix* configurations. They connect $|1100\rangle$ to $|0011\rangle$, creating the superposition — the coherences — that lower the energy below the classical minimum.

The chain of implications is absolute:

No off-diagonal H $\Rightarrow$ no coherences in ρ $\Rightarrow$ no correlation energy $\Rightarrow$ HF is exact $\Rightarrow$ no quan

What this means for encoding

Now we can state precisely what the encoding must accomplish:

The **diagonal part** of the Hamiltonian (orbital energies, Coulomb repulsion) is cheap. It maps to I and Z Pauli operators, diagonal in the computational basis. Measuring them is trivial. Simulating their time evolution requires only single-qubit Z-rotations. No entanglement, no deep circuits.

The **off-diagonal part** (exchange interactions) is expensive. It maps to X and Y Pauli operators, which flip qubits and create entanglement. Simulating these terms requires multi-qubit gates — CNOT staircases, as we'll see in Stage 4. The Pauli weight of these off-diagonal terms directly determines the circuit depth.

The encoding choice determines the cost of the off-diagonal terms. JW, BK, and tree encodings all produce the same diagonal terms — they differ only in how they handle the Z-chains that accompany the off-diagonal X/Y operators.

This is the thesis of the entire book: the correlation energy lives in the off-diagonal coherences of the density matrix, and the encoding determines how expensive it is to create and maintain those coherences on quantum hardware.

With this lens in place, let's build the Hamiltonian and see the structure emerge.

What We Have and What We'll Do

Chapters 1–4 gave us everything we need:

- **The chemistry** (Chapter 1): H_2 in STO-3G, 4 spin-orbitals, the second-quantized Hamiltonian.
- **The notation** (Chapter 2): physicist's convention, the conversion rule, the traps to avoid.
- **The numbers** (Chapter 3): complete spin-orbital integral tables and a coefficient factory in F#.
- **The quantum computer** (Chapter 4): qubits, gates, CNOT cost, and the $2(w-1)$ formula.
- **The encoding** (Chapter 5): Jordan–Wigner's Z-chain mechanism for translating fermionic operators to Pauli strings.

The procedure from here is mechanical. Once you see it done for one term, you can do it for any term, for any molecule, in any encoding. The insight is behind us; what remains is careful bookkeeping.

The Recipe

```
flowchart TD
    OB["One-body terms:<br/>For each non-zero one-body integral,<br/>encode the two lad
    TB["Two-body terms:<br/>For each non-zero two-body integral,<br/>encode all four la
    OB --> SUM["Combine like terms:<br/>collect Pauli strings with the same signature<b
    TB --> SUM
    SUM --> NR["Add nuclear repulsion<br/>as a contribution to the identity term"]
    NR --> HAM["The qubit Hamiltonian"]
    style HAM fill:#d1fae5,stroke:#059669
```

FockMap does this symbolically — no matrices, no floats in the intermediate algebra. We'll work through one representative term by hand, then show the complete result.

One-body terms: number operators

The non-zero one-body integrals for H_2 are all diagonal (Chapter 3): $h_{00} = h_{11} = -1.2563$ Ha and $h_{22} = h_{33} = -0.4719$ Ha. Under Jordan–Wigner, the number operator simplifies — the Z-chains cancel:

$$\hat{n}_j = a_j^\dagger a_j = \frac{1}{2}(I - Z_j)$$

Weight 1, regardless of system size. The one-body Hamiltonian produces five terms — all diagonal, all classical:

Pauli term	Coefficient (Ha)	Origin
IIII	-1.7282	Sum of all orbital energies, halved
$IIIZ$	$+0.6282$	$-h_{00}/2$ (energy of σ_g, α)
$IIZI$	$+0.6282$	$-h_{11}/2$ (energy of σ_g, β)
$IZII$	$+0.2359$	$-h_{22}/2$ (energy of σ_u, α)
$ZIII$	$+0.2359$	$-h_{33}/2$ (energy of σ_u, β)

All I and Z. No off-diagonal terms. No coherences generated. If this were the whole Hamiltonian, a laptop would suffice.

Two-body terms: one by hand

Consider the Coulomb repulsion between two electrons in σ_g with opposite spins: $\frac{1}{2}\langle 01 \mid 01\rangle\ a_0^\dagger a_1^\dagger a_1 a_0$, with integral value 0.6745 Ha.

Encode each operator under JW, multiply the four Pauli strings, and simplify. Three observations make the algebra tractable: the Z-chains cancel ($Z_0 \cdot Z_0 = I$), and each raising-lowering pair simplifies via $(X - iY)(X + iY) = 2(I - Z)$.

$$a_0^\dagger a_1^\dagger a_1 a_0 = \frac{1}{4}(IIII - IIIZ - IIZI + IIZZ)$$

Scaled by the integral: four diagonal Pauli contributions. This is a Coulomb term — pure classical electrostatics, no off-diagonal structure. Exactly as the density matrix framework predicted.

The **exchange** integrals $\langle 02 \mid 20 \rangle$ tell a different story. They produce terms like $a_0^\dagger a_2^\dagger a_0 a_2$, where electrons swap orbitals. Under JW, these don't simplify to pure Z — they leave XX and YY Pauli operators. These are the off-diagonal terms. They generate coherences. They produce the correlation energy.

Let's see this happen explicitly.

One exchange term, fully expanded

Consider $\frac{1}{2}\langle 02 \mid 20 \rangle\ a_0^\dagger a_2^\dagger a_2 a_0$, with integral value $\langle 02 \mid 20 \rangle = 0.1809$ Ha. This is an exchange integral — electron 1 scatters from orbital 0 to orbital 2, electron 2 scatters from orbital 2 to orbital 0.

Encode under Jordan–Wigner:

$$a_0^\dagger = \frac{1}{2}(X_0 - iY_0) \otimes I_1 \otimes I_2 \otimes I_3$$

$$a_2^\dagger = \frac{1}{2}\, Z_0 Z_1 \otimes (X_2 - iY_2) \otimes I_3$$

$$a_2 = \frac{1}{2}\, Z_0 Z_1 \otimes (X_2 + iY_2) \otimes I_3$$

$$a_0 = \frac{1}{2}(X_0 + iY_0) \otimes I_1 \otimes I_2 \otimes I_3$$

Note: $a_2^\dagger$ has a Z-chain of length 2 ($Z_0 Z_1$), not length 1.

Multiply.

$$a_0^\dagger a_2^\dagger a_2 a_0 = \frac{1}{16}(X_0 - iY_0)\, Z_0 Z_1 (X_2 - iY_2)\, Z_0 Z_1 (X_2 + iY_2)\, (X_0 + iY_0)$$

Simplify. The Z-chains cancel on qubits 0 and 1: $Z_0 Z_0 = I$, $Z_1 Z_1 = I$. The qubit-2 pair simplifies as before: $(X_2 - iY_2)(X_2 + iY_2) = 2(I - Z_2)$. But now qubit 0 has something new:

$$(X_0 - iY_0) \cdot (X_0 + iY_0) = 2(I - Z_0)$$

Wait — that's the same identity as the Coulomb case. So where do the XX/YY terms come from?

They come from a *different* index combination. Consider instead:

$$\frac{1}{2}\langle 02 \mid 02\rangle \; a_0^\dagger a_2^\dagger a_0 a_2$$

Here the operator ordering is $a_0^\dagger a_2^\dagger a_0 a_2$ — note the annihilation operators are $a_0 a_2$, not $a_2 a_0$. When we encode and multiply:

$$a_0^\dagger a_2^\dagger a_0 a_2 = \frac{1}{16}(X_0 - iY_0)\, Z_0 Z_1 (X_2 - iY_2)\, (X_0 + iY_0)\, Z_0 Z_1 (X_2 + iY_2)$$

Now the Z-chains don't cancel cleanly — the $(X_0 + iY_0)$ sits *between* the two Z_0 factors instead of outside them. Expanding qubit 0:

$$(X_0 - iY_0)\, Z_0\, (X_0 + iY_0)\, Z_0$$

Using $XZ = -iY$, $YZ = iX$, and carefully tracking the phases, this produces terms proportional to X_0X_2, Y_0Y_2, X_0Y_2, and Y_0X_2 — the off-diagonal Pauli operators.

The full expansion (which FockMap performs symbolically) yields contributions to $XXYY$, $XYYX$, $YXXY$, and $YYXX$ — precisely the four exchange terms in the final Hamiltonian, with coefficient ± 0.1744.

> **What just happened:** When the annihilation operators are in a different order from the creation operators (electrons change orbitals), the Z-chains interleave with the X/Y flips instead of cancelling. The result is an off-diagonal Pauli string — exactly the structure that generates coherences in the density matrix.

The Complete 15-Term Hamiltonian

After processing all 32 non-zero two-body integrals, combining like terms, and adding $V_{nn} = 0.7151$ Ha:

#	Pauli String	Coefficient (Ha)	Character
1	IIII	-1.0704	Energy offset
2	$IIIZ$	-0.0958	Orbital energy
3	$IIZI$	-0.0958	Orbital energy
4	$IZII$	$+0.3021$	Orbital energy
5	$ZIII$	$+0.3021$	Orbital energy
6	$IIZZ$	$+0.1743$	Coulomb repulsion
7	$IZIZ$	-0.0085	Coulomb repulsion
8	$IZZI$	$+0.1659$	Coulomb repulsion
9	$ZIIZ$	$+0.1659$	Coulomb repulsion

#	Pauli String	Coefficient (Ha)	Character
10	$ZIZI$	-0.0085	Coulomb repulsion
11	$ZZII$	$+0.1686$	Coulomb repulsion
12	$XXYY$	-0.1744	**Exchange**
13	$XYYX$	$+0.1744$	**Exchange**
14	$YXXY$	$+0.1744$	**Exchange**
15	$YYXX$	-0.1744	**Exchange**

Reading the Hamiltonian: What's Hartree–Fock, What's Quantum

Now we read this table through the density matrix lens.

Terms 1–5 (weight 0–1, I and Z only): orbital energies and a constant offset. These are eigenvalues of the number operators — the energy cost of occupying each orbital. Purely diagonal. This is the one-body part of Hartree–Fock.

Terms 6–11 (weight 2, ZZ pairs): Coulomb repulsion between pairs of orbitals. Still diagonal — they refine the classical energy by accounting for pairwise electron-electron repulsion. Measurable by reading qubit values, no entanglement needed. This is the two-body part of Hartree–Fock.

Terms 12–15 (weight 4, XXYY-type): **quantum exchange**. These are the off-diagonal terms — the ones that mix configurations, generate coherences in ρ, and produce the correlation energy.

Four terms out of fifteen. They come in two pairs with equal magnitude and opposite sign (± 0.1744) — a consequence of fermionic antisymmetry.

Delete terms 12–15 and the ground state would be $|1100\rangle$ with a purely diagonal density matrix — a classical probability distribution, not a quantum state. The quantum advantage vanishes.

Keep them and the ground state becomes a superposition with coherences. The energy drops by ~12 kcal/mol — the correlation energy that no classical single-reference method can capture. This is the energy that determines whether a bond breaks, whether a reaction goes forward, whether a drug binds.

Four Pauli strings. That's where quantum computing earns its keep.

Reproducing This with FockMap

The library computes the entire Hamiltonian from the integral tables in Chapter 3:

```
open System.Numerics
open Encodings

// Coefficient factory (from Chapter 3)
let h2Factory key = h2Integrals |> Map.tryFind key

// Build the JW Hamiltonian on 4 qubits
let hamiltonian = computeHamiltonianWith jordanWignerTerms h2Factory 4u

// Print all terms
for t in hamiltonian.DistributeCoefficient.SummandTerms do
    printfn "%+.4f  %s" t.Coefficient.Real t.Signature
```

Output:

```
-1.0704  IIII
-0.0958  IIIZ
-0.0958  IIZI
+0.3021  IZII
+0.3021  ZIII
+0.1743  IIZZ
-0.0085  IZIZ
+0.1659  IZZI
+0.1659  ZIIZ
-0.0085  ZIZI
+0.1686  ZZII
-0.1744  XXYY
+0.1744  XYYX
+0.1744  YXXY
-0.1744  YYXX
```

Every coefficient matches the table.

Trying a different encoding

Changing the encoding is one function name:

```
let h2_bk  = computeHamiltonianWith bravyiKitaevTerms  h2Factory 4u
let h2_tt  = computeHamiltonianWith ternaryTreeTerms    h2Factory 4u
let h2_par = computeHamiltonianWith parityTerms         h2Factory 4u
```

All four produce different Pauli strings but the **same eigenvalues**. We'll verify this in Chapter 8.

Key Takeaways

- The **density matrix** distinguishes classical from quantum: diagonal entries are probabilities (classical); off-diagonal entries are coherences (quantum).
- Off-diagonal Hamiltonian terms **generate** those coherences. No off-diagonal terms → no correlation energy → no quantum advantage.
- The H_2 Hamiltonian has 11 diagonal terms (Hartree–Fock) and 4 off-diagonal terms (quantum exchange). The correlation energy lives entirely in those 4 terms.
- The encoding determines how many qubits each off-diagonal term touches — and therefore the circuit cost of quantum simulation.

Common Mistakes

1. **Forgetting V_{nn}.** The nuclear repulsion contributes to the IIII coefficient. Without it, the ground-state energy will be off by 0.7151 Ha.
2. **Wrong operator ordering.** The annihilation operators in $a_p^\dagger a_q^\dagger a_s a_r$ are in *reverse* order. Writing $a_r a_s$ instead of $a_s a_r$ flips signs on exchange terms.
3. **Not combining like terms.** The 32 two-body integrals produce many duplicate Pauli signatures that must be summed.

Exercises

1. **Number operator by hand.** Verify that $a_2^\dagger a_2 = \frac{1}{2}(I - Z_2)$ by expanding the JW-encoded operators.
2. **Exchange term sign.** Explain why $XXYY$ has coefficient -0.1744 and $XYYX$ has $+0.1744$ in terms of fermionic antisymmetry.
3. **Diagonal-only energy.** Delete terms 12–15. What is the ground-state energy of the remaining diagonal Hamiltonian? Compare with $E_{\text{HF}} = -1.1168$ Ha.
4. **Encoding comparison.** Run the code with all five encodings. Do they all produce the same number of terms for H_2?

Further Reading

- Whitfield, J. D., Biamonte, J., and Aspuru-Guzik, A. "Simulation of electronic structure Hamiltonians using quantum computers." *Mol. Phys.* 109, 735 (2011).
- McArdle, S. et al. "Quantum computational chemistry." *Rev. Mod. Phys.* 92, 015003 (2020). Section III.

Previous: Chapter 5 — A Visual Guide to Encodings

Next: Chapter 7 — Five Encodings, One Interface

Chapter 7: Five Encodings, One Interface

We built the H_2 Hamiltonian with Jordan–Wigner. But JW has a problem — its Z-chains grow linearly with system size. This chapter asks: can we do better? And if we use a different encoding, do we get the same physics?

In This Chapter

- **What you'll learn:** Why we need alternatives to Jordan–Wigner, whether different encodings give the same answer (they do — and we'll see why), and how to compare their costs.
- **Why this matters:** Encoding choice is the single biggest lever you have for reducing circuit depth before you write a single gate. But only if you trust that all encodings produce the same physics.
- **Prerequisites:** Chapters 1–6 (you have the 15-term JW Hamiltonian and understand the diagonal/off-diagonal structure).

Why Would We Want a Different Encoding?

In Chapter 6, we saw that 4 of the 15 Pauli terms — the exchange terms XXYY, XYYX, YXXY, YYXX — are off-diagonal. These are the terms that generate coherences in the density matrix and produce the correlation energy. They are also the terms that are expensive to simulate on a quantum computer, because each one requires a CNOT staircase proportional to its Pauli weight.

Under Jordan–Wigner, the worst-case Pauli weight of an operator grows linearly with system size: $O(n)$. For H_2 with 4 qubits, the maximum weight is 4 — manageable. But for H_2O with 12 qubits, it's 12. For the FeMo-co nitrogen-fixation catalyst with ~100 qubits, it's ~100. Each off-diagonal term would require ~200 CNOT gates.

This is the motivation for alternative encodings: **can we represent the same physics with shorter Pauli strings?**

Chapter 5 showed that Bravyi–Kitaev achieves $O(\log_2 n)$ weight using a Fenwick tree, and ternary tree encodings achieve $O(\log_3 n)$. But a natural question arises: if the

Pauli strings are different, how do we know we're still computing the same molecule?

The answer is that all valid encodings preserve the **canonical anti-commutation relations** (CAR): $\{a_p^\dagger, a_q\} = \delta_{pq}$. Any encoding that faithfully maps the CAR from fermionic operators to Pauli operators must produce a Hamiltonian with the same spectrum — because the eigenvalues of the Hamiltonian depend only on the algebraic relations among the operators, not on the specific matrix form. Different encodings are related by a unitary change of basis on the qubit Hilbert space, and unitary transformations preserve eigenvalues. This is the mathematical guarantee that the physics is encoding-independent.

Do Different Encodings Give the Same Answer?

Let's find out empirically. Build the H_2 Hamiltonian with all five encodings and compare:

```
let h2Factory key = h2Integrals |> Map.tryFind key

for (name, encoder) in encoders do
    let ham = computeHamiltonianWith encoder h2Factory 4u
    let terms = ham.DistributeCoefficient.SummandTerms
    printfn "%-25s  %d terms" name terms.Length
```

```
Jordan-Wigner              15 terms
Bravyi-Kitaev              15 terms
Parity                     15 terms
Balanced Binary Tree       15 terms
Balanced Ternary Tree      15 terms
```

Same number of terms. Same identity coefficient (-1.0704 Ha in every case). And if we diagonalize each — as we'll do in Chapter 8 — the eigenvalues agree to machine precision ($< 5 \times 10^{-16}$ Ha).

For H_2 with 4 qubits, the Pauli strings are actually identical across all five encodings. With only 4 qubits, there isn't enough room for the different encodings to diverge — the Z-chain is at most length 3, and BK's tree structure on 4 nodes provides at most a marginal improvement.

This is an important lesson: **for small molecules, encoding choice barely matters.** The differences emerge at scale. But the equivalence — the fact that they all give the same answer — holds at every scale. Let's understand why.

Why Equivalence Holds: The CAR Theorem

An encoding maps fermionic operators $a_p^\dagger$, a_p to qubit operators (Pauli strings). Different encodings produce different Pauli strings for the same ladder operator. But the second-quantized Hamiltonian

$$\hat{H} = \sum_{pq} h_{pq}\, a_p^\dagger a_q + \frac{1}{2} \sum_{pqrs} \langle pq \mid rs \rangle\, a_p^\dagger a_q^\dagger a_s a_r$$

is defined in terms of the *algebra* of the operators — their products and commutation relations — not their specific matrix representations.

The key property that any valid encoding must preserve is the **canonical anti-commutation relations (CAR)**:

$$\{a_p^\dagger, a_q\} = \delta_{pq}, \qquad \{a_p^\dagger, a_q^\dagger\} = 0, \qquad \{a_p, a_q\} = 0$$

Here is the theorem:

> **If an encoding preserves the CAR, then the encoded Hamiltonian has the same eigenvalues as the original fermionic Hamiltonian.**

Why? Because the Hamiltonian is a polynomial in the operators $a_p^\dagger$ and a_p. If two representations of these operators satisfy the same algebraic relations (the CAR), then any polynomial in them produces the same algebraic object — and therefore the same eigenvalues. This is a standard result in operator algebra: the CAR define the algebra up to unitary equivalence.

In practice, this means the encoding is a **unitary change of basis** on the qubit Hilbert space. The eigenvalues of a matrix are invariant under unitary conjugation — the spectrum is preserved exactly.

FockMap's test suite verifies the CAR at the Pauli string level for every encoding: it checks that

$$(\text{encoded } a_i^\dagger)(\text{encoded } a_j) + (\text{encoded } a_j)(\text{encoded } a_i^\dagger) = \delta_{ij} \cdot I$$

for all pairs (i, j), using symbolic Pauli multiplication. If this check passes, the encoding is algebraically valid and the encoded Hamiltonian is guaranteed to preserve the spectrum in principle. In practice, we still compare eigenvalues numerically in Chapter 8 to verify the full pipeline implementation — because algebraic correctness of the encoding does not protect against bugs in integral processing, coefficient assembly, or like-term combination.

This is not just a theoretical nicety. It is what makes encoding choice a **free optimization**: you can pick the encoding that minimizes circuit cost, knowing — with mathematical certainty, not just empirical evidence — that the physics is preserved.

Five Encodings in Five Lines

With equivalence established, let's see all five:

```
let encoders = [
    ("Jordan-Wigner",          jordanWignerTerms)
    ("Bravyi-Kitaev",          bravyiKitaevTerms)
    ("Parity",                 parityTerms)
    ("Balanced Binary Tree",   balancedBinaryTreeTerms)
    ("Balanced Ternary Tree", ternaryTreeTerms)
]
```

All five have the same function signature. All five produce `PauliRegisterSequence`. Swapping one for another is a one-word change.

Where the Differences Emerge: Scaling

The differences show up in Pauli weight at larger system sizes:

n	JW	BK	Ternary Tree	JW/TT ratio
4	4	3	3	1.3×
8	8	4	4	2×
16	16	5	5	3.2×
32	32	6	5	6.4×

At 32 spin-orbitals, JW's worst-case operator touches 32 qubits. The ternary tree touches 5. Since each Pauli rotation costs $2(w-1)$ CNOT gates (Chapter 4), this is the difference between 62 CNOTs and 8 CNOTs *per term* — compounded across every term and every Trotter step.

Recall from Chapter 6 that the off-diagonal terms are the expensive ones. The encoding determines how many qubits those off-diagonal terms touch — and therefore the circuit depth required to create and maintain the coherences that capture the correlation energy.

The Three Frameworks

FockMap implements encodings through two complementary frameworks (plus one special case):

```
flowchart TD
    subgraph ISF["Index-Set Framework"]
        direction TB
        JW["Jordan-Wigner<br/>U={}, P={0..j-1}, O={j}"]
```

```
        BK["Bravyi-Kitaev<br/>from Fenwick tree"]
        PAR["Parity<br/>U={j+1..n-1}, P={j-1}, O={j-1,j}"]
    end
    subgraph PBF["Path-Based Framework"]
        direction TB
        BBT["Balanced Binary Tree"]
        BTT["Balanced Ternary Tree"]
        CT["Custom Tree"]
    end
    ISF --> PS["PauliRegisterSequence"]
    PBF --> PS
    style PS fill:#d1fae5,stroke:#059669
```

Index-set framework (MajoranaEncoding.fs): An encoding is defined by three set-valued functions — Update(j), Parity(j), and Occupation(j). JW, BK, and Parity are each defined in 3–5 lines of F#.

Path-based framework (TreeEncoding.fs): An encoding is derived from a labelled rooted tree. Any tree topology works — strictly more general than the index-set framework.

Fenwick-specific (BravyiKitaev.fs): BK uses hand-derived bit-manipulation formulas specific to the Fenwick tree. Faster than generic tree traversal, same result.

> **Design note:** The index-set framework was introduced by Seeley, Richard, and Love (2012) as a unifying abstraction. Our investigation showed that it produces correct encodings *only* for star-shaped (depth-1) trees — a previously undocumented constraint. The path-based framework (Jiang et al., 2020) removes this restriction. FockMap provides both because the index-set framework is simpler and faster when it applies.

Defining Your Own Encoding

Because encodings are *values* (not classes), defining a new one is concise:

```
let myScheme : EncodingScheme =
    { Update     = fun j n -> set [ j + 1 .. n - 1 ]
      Parity     = fun j   -> Set.empty
      Occupation = fun j   -> Set.singleton j }

let ham = computeHamiltonianWith (encodeOperator myScheme) h2Factory 4u
```

> **Warning:** Not every set of three functions produces a valid encoding. A valid encoding must preserve the CAR: $\{a_i, a_j^\dagger\} = \delta_{ij}$. Use FockMap's anti-commutation test infrastructure to verify correctness before trusting results from a custom scheme. An encoding that fails the CAR check will

produce a Hamiltonian with wrong eigenvalues — and the error will be silent.

A Decision Framework

Situation	Recommended encoding	Reasoning
Learning / prototyping	Jordan–Wigner	Simplest to understand and debug
Small system ($n \leq 16$)	Jordan–Wigner	Weight overhead is manageable
1D chain / local interactions	Jordan–Wigner	Adjacent-orbital terms have short Z-chains
General-purpose ($n \leq 100$)	Bravyi–Kitaev	$O(\log_2 n)$ weight, well-studied
Minimum circuit depth	Ternary Tree	$O(\log_3 n)$ — best known asymptotic scaling
Exploring custom topologies	Path-based	Arbitrary tree shapes supported
Comparing multiple encodings	All five	FockMap's interchangeable interface makes this trivial

---

Key Takeaways

- JW's $O(n)$ Pauli weight motivates the search for alternative encodings.
- All valid encodings produce Hamiltonians with the **same eigenvalues** — this follows from CAR preservation, not just empirical observation.
- For H_2 (4 qubits), the encodings produce identical Pauli strings. The differences emerge at larger n.
- The scaling advantage of tree-based encodings becomes dramatic above ~16 spin-orbitals.
- Defining a custom encoding is 3–5 lines of F# — but verify CAR before trusting it.

Common Mistakes

1. **Assuming different Pauli strings mean different physics.** They don't — if the CAR is preserved, eigenvalues are preserved. Different strings, same physics.

2. **Choosing an encoding by term count.** All encodings produce the same number of Hamiltonian terms. The difference is Pauli *weight* per term, which determines CNOT count.
3. **Using a custom encoding without verifying CAR.** An encoding that violates anti-commutation will produce plausible but wrong Hamiltonians. Always test.

Exercises

1. **Weight table.** Run the scaling comparison for $n = 64$ and $n = 128$. At what point does the ternary tree's maximum weight exceed 10?
2. **CAR verification.** Define a custom encoding and use FockMap's test infrastructure to check whether it satisfies anti-commutation. What happens if it doesn't?
3. **Encoding comparison for H_2O.** Build the H_2O/STO-3G Hamiltonian (12 spin-orbitals) with all five encodings. Compare maximum and average Pauli weight.

Further Reading

- Seeley, J. T., Richard, M. J., and Love, P. J. "The Bravyi–Kitaev transformation for quantum computation of electronic structure." *J. Chem. Phys.* 137, 224109 (2012). The index-set framework.
- Jiang, Z. et al. "Optimal fermion-to-qubit mapping via ternary trees." *PRX Quantum* 1, 010306 (2020). The path-based ternary tree encoding.
- Tranter, A. et al. "The Bravyi–Kitaev transformation: Properties and applications." *Int. J. Quantum Chem.* 115, 1431 (2015). Practical JW vs BK comparison.

Previous: Chapter 6 — Building the Qubit Hamiltonian

Next: Chapter 8 — Checking Our Answer

Chapter 8: Checking Our Answer

We have fifteen Pauli strings and fifteen coefficients. How do we know they're right? This chapter introduces the only reliable verification method: compute the eigenvalues and compare.

In This Chapter

- **What you'll learn:** How to verify an encoded Hamiltonian by exact diagonalization, how to interpret the eigenspectrum by particle-number sector, and how to confirm that all five encodings produce the same physics.
- **Why this matters:** Encoding bugs produce Hamiltonians that look structurally correct — right number of terms, right symmetries, plausible coefficients. The only reliable test is the eigenspectrum. If you skip verification, you will eventually publish a wrong number.
- **Prerequisites:** Chapters 1–7 (you have the 15-term Hamiltonian from all five encodings).

The Verification Problem

Here is a sobering fact about fermion-to-qubit encoding: **most bugs produce plausible output.**

A wrong-convention Hamiltonian (Chapter 2, Error #1) has 15 terms with real coefficients and the right Pauli symmetries. A missing cross-spin block (Chapter 3, Mistake #1) has fewer exchange terms but still looks like a valid Hamiltonian. A reversed operator ordering (Chapter 6, Mistake #2) flips some signs but leaves the structure intact.

None of these errors will crash your code. All of them will give wrong eigenvalues. The only way to catch them is to compute those eigenvalues and compare with a known reference.

For H_2 in STO-3G, we have exact classical reference values. For larger molecules, we can still cross-check between encodings — if all five give the same spectrum, we can be confident in the pipeline (though not in the integrals, which must be checked separately).

From Pauli Sum to Matrix

This is the one place where we leave the symbolic world and build a matrix. We do this *only* for verification — the actual quantum simulation never needs a matrix.

Each single-qubit Pauli operator has a 2×2 matrix:

$$I = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, \quad X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, \quad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$$

A 4-qubit Pauli string like $IIZZ$ is the tensor product $I \otimes I \otimes Z \otimes Z$ — a 16×16 matrix. The full Hamiltonian matrix is the weighted sum:

$$H = \sum_{\alpha=1}^{15} c_\alpha \cdot (\text{tensor product of } \sigma_{\alpha,0} \otimes \sigma_{\alpha,1} \otimes \sigma_{\alpha,2} \otimes \sigma_{\alpha,3})$$

For 4 qubits, this is a 16×16 Hermitian matrix. Diagonalizing it gives 16 eigenvalues.

> **Why 16 and not 6?** The 4-qubit Hilbert space has $2^4 = 16$ basis states, but H_2 has only 2 electrons — so only 6 of those states ($\binom{4}{2} = 6$) have the right particle number. The remaining 10 eigenvalues belong to the 0-electron, 1-electron, 3-electron, and 4-electron sectors. They are physically meaningful (they represent the spectrum with different electron counts) but they are not the ground state we are looking for.

The Eigenspectrum of H_2

Diagonalizing the 15-term JW Hamiltonian gives eigenvalues grouped by particle-number sector:

Sector (N_e)	States	Eigenvalues E_{el} (Ha)
0	1	0
1	4	$-1.2563,\ -1.2563,\ -0.4719,\ -0.47$
2	**6**	$-\mathbf{1.8573},\ -1.3390,\ -0.9032,\ -0.9$
3	4	$-1.7282,\ -1.7282,\ -0.9438,\ -0.94$
4	1	-2.2001

The ground state of the physical 2-electron sector is $E_0^{\text{el}} = -1.8573$ Ha. Adding nuclear repulsion:

$$\boxed{E_0^{\text{total}} = E_0^{\text{el}} + V_{nn} = -1.8573 + 0.7151 = -1.1422 \text{ Ha}}$$

This is the **exact** ground-state energy of H_2 in the STO-3G basis — not an approximation, not a truncation, but the answer you'd get from full configuration interaction (Full CI). A quantum computer running VQE or QPE on this Hamiltonian should converge to this value.

How Good Is This?

Let's put the number in context by comparing with simpler approximations:

Method	E_{total} (Ha)	Error (kcal/mol)	What it captures
Hartree–Fock	-1.1168	15.9	Mean-field (no correlation)
MP2	-1.1381	2.6	Perturbative correlation
Full CI (= our result)	-1.1422	0	Exact (within basis)
Chemical accuracy target	—	< 1.0	The goal for quantum chemistry

The Hartree–Fock error of 15.9 kcal/mol is large — it would mispredict reaction rates by orders of magnitude. Our Full CI result captures the entire correlation energy within the STO-3G basis. In the language of Chapter 6, this 15.9 kcal/mol is exactly the energy contribution of the off-diagonal terms in $\mathrm{Tr}(\rho H)$ — the coherences in the density matrix, generated by the 4 exchange Pauli strings, that Hartree–Fock's diagonal ρ cannot capture.

Of course, STO-3G is a minimal basis — the absolute energy is still far from the true Born–Oppenheimer value. But within the model space we've defined, our answer is exact.

> **The point of quantum simulation is not to replace the basis set** — it's to solve the electronic problem *exactly* within whatever basis the user chooses, even for molecules where Full CI is classically intractable. For H_2 in STO-3G (6 configurations), any laptop can do Full CI. For a molecule with 50 electrons in 100 spin-orbitals ($\binom{100}{50} \approx 10^{29}$ configurations), only a quantum computer can.

Cross-Encoding Verification

The most powerful check: build the Hamiltonian with all five encodings and verify that they produce the **same eigenspectrum**.

```
for (name, encoder) in encoders do
    let ham = computeHamiltonianWith encoder h2Factory 4u
    // ... build 16×16 matrix, diagonalize ...
    printfn "%-25s  E₀ = %.10f Ha" name groundStateEnergy
```

Encoding	E_0^{el} (Ha)	$\lvert\Delta E\rvert$ from JW
Jordan–Wigner	-1.8572750302	—
Bravyi–Kitaev	-1.8572750302	$< 10^{-15}$
Parity	-1.8572750302	$< 10^{-15}$
Balanced Binary Tree	-1.8572750302	$< 10^{-15}$
Balanced Ternary Tree	-1.8572750302	$< 10^{-15}$

Agreement to 5×10^{-16} Ha — the limit of 64-bit floating-point precision. The eigenvalues are identical to machine epsilon.

This is not a coincidence. It is a mathematical guarantee: every valid encoding preserves the canonical anti-commutation relations, and therefore preserves the operator algebra, and therefore preserves every eigenvalue. FockMap's test suite verifies the anti-commutation relations symbolically (no eigenvalues needed), but the eigenvalue comparison provides an independent numerical cross-check.

A Verification Checklist

When building and verifying an encoded Hamiltonian, check these in order:

#	Check	How	What failure means
1	Term count	Count Pauli strings	Missing/extra integrals
2	Identity coefficient	Read IIII coefficient	Wrong V_{nn} or integral sum
3	Diagonal symmetry	All Z-only terms come in pairs ($IIIZ/IIZI$, etc.)	Broken spin symmetry
4	Exchange terms present	Look for XX/YY terms	Missing cross-spin integrals
5	Cross-encoding agreement	Build with 2+ encodings, compare spectra	Encoding bug

#	Check	How	What failure means
6	Known reference	Compare E_0 against published value	Convention error

If check 4 fails (no exchange terms), the most likely cause is the cross-spin bug from Chapter 3. If check 5 fails (different eigenvalues from different encodings), there is a bug in the encoding implementation itself — which, if you're using FockMap, should not happen.

Stage 2 Complete

With verification done, we have walked the complete path from molecule to validated qubit Hamiltonian:

```
flowchart LR
    CH1["Ch.1<br/>Molecule → Integrals"]
    CH2["Ch.2<br/>Notation fixed"]
    CH3["Ch.3<br/>Spin-orbital tables"]
    CH4["Ch.4<br/>Encoding concepts"]
    CH5["Ch.5<br/>15-term Hamiltonian"]
    CH6["Ch.6<br/>Five encodings"]
    CH7["Ch.7<br/>Verified ✓"]
    CH1 --> CH2 --> CH3 --> CH4 --> CH5 --> CH6 --> CH7
    CH7 --> NEXT["Stage 3:<br/>Tapering"]
    style CH7 fill:#d1fae5,stroke:#059669
    style NEXT fill:#fde68a,stroke:#d97706
```

The Hamiltonian is correct. It is the exact qubit representation of H_2 in STO-3G for any of five encodings. It has 15 Pauli terms, its ground-state energy is -1.1422 Ha, and it captures all the electron correlation that a quantum computer would extract.

But can we make it *smaller*? Can we remove qubits without losing physics? That's what tapering does — and it's where we go next.

Key Takeaways

- The **only reliable verification** of an encoded Hamiltonian is eigenvalue comparison against a known reference or cross-encoding check.
- The H_2/STO-3G ground-state energy is $E_0 = -1.1422$ Ha (Full CI, exact within basis).

- All five encodings produce eigenvalues that agree to $< 5 \times 10^{-16}$ Ha — machine precision.
- Encoding bugs produce structurally plausible but numerically wrong Hamiltonians. Check eigenvalues early and often.
- Matrix diagonalization is used *only* for verification. The actual quantum simulation operates on the symbolic Pauli sum.

Common Mistakes

1. **Verifying only the ground state.** Check the full spectrum (all particle-number sectors), not just E_0. Some bugs preserve the ground state but corrupt excited states.
2. **Comparing absolute energies across basis sets.** STO-3G and cc-pVDZ give different energies for the same molecule. Only compare within the same basis.
3. **Skipping verification for "trusted" code.** Even well-tested libraries can produce wrong results if the integral input is wrong. Verify the first Hamiltonian you build for any new molecule.

Exercises

1. **Sector analysis.** The 0-electron sector has eigenvalue 0. Why? (Hint: what does the Hamiltonian do to the vacuum state $|0000\rangle$?)
2. **Correlation energy.** Compute the Hartree–Fock energy of H_2 by hand: $E_{\text{HF}} = h_{00} + h_{11} + [00 \mid 00] + V_{nn}$ (one-body energy for each spin-orbital in σ_g, plus Coulomb repulsion, plus nuclear repulsion). Verify that $E_{\text{corr}} = E_{\text{FCI}} - E_{\text{HF}} \approx -0.019$ Ha ≈ -12 kcal/mol. This is exactly the contribution of the off-diagonal exchange terms from Chapter 6.
3. **Cross-encoding lab.** Run the Compare Encodings lab and verify 5-encoding eigenvalue agreement for yourself.

Further Reading

- Szabo, A. and Ostlund, N. S. *Modern Quantum Chemistry.* §4.1 gives the Full CI eigenvalues for H_2/STO-3G.
- Helgaker, T., Jørgensen, P., and Olsen, J. *Molecular Electronic-Structure Theory.* Chapter 11 covers Full CI theory and implementation.

Previous: Chapter 7 — Five Encodings, One Interface

Next: Chapter 9 — Why Tapering?

Chapter 9: Why Tapering?

The Hamiltonian is correct and verified. But it may be bigger than it needs to be. This chapter shows why encoded Hamiltonians often contain redundant qubits — and how to detect them.

In This Chapter

- **What you'll learn:** How fermion-to-qubit encoding creates Z_2 symmetries that allow safe qubit removal, and why this is one of the highest-leverage optimizations in the entire pipeline.
- **Why this matters:** Removing even one qubit halves the Hilbert space. For near-term quantum hardware with limited qubit counts and high error rates, tapering can mean the difference between a feasible and an infeasible simulation.
- **Prerequisites:** Chapters 1–7 (you have a verified qubit Hamiltonian and understand Pauli strings).

The Observation

Look again at the 15-term H_2 Hamiltonian from Chapter 6. Focus on the Pauli operators at each qubit position:

Qubit 0	Qubit 1	Qubit 2	Qubit 3
I, I, I, I, I, I, I, I, I, Z, Z, X, X, Y, Y	I, I, I, Z, Z, I, Z, Z, I, I, Z, X, Y, X, Y	I, I, I, I, I, Z, I, Z, Z, Z, I, Y, Y, X, X	I, Z, Z, I, I, Z, Z, I, Z, I, I, Y, X, Y, X

Every qubit has at least some terms with X or Y — no qubit is purely diagonal (I/Z only) across all 15 terms. So H_2 under JW has **no diagonal Z_2 symmetries** at first glance.

But this is specific to H_2's structure and the JW encoding. Many molecular Hamiltonians — especially larger ones — *do* have qubits where every term is I or Z. And even when no single qubit is diagonal, there may be **multi-qubit** Z_2 symmetries (like $Z_0 Z_1$) that a Clifford rotation can exploit.

Where diagonal symmetries come from

Particle-number conservation is the most common source. In many encodings, the total electron number operator $\hat{N} = \sum_j \hat{n}_j$ commutes with the Hamiltonian. Under JW, $\hat{N}$ maps to a sum of Z operators. If certain linear combinations of these Z operators commute with every Hamiltonian term, the corresponding qubits can be fixed and removed.

The Parity encoding makes this especially transparent: the last qubit often stores the total parity of the electron number, which is conserved — making it immediately taperable.

What Tapering Gains You

The benefits are concrete and multiplicative:

```
flowchart LR
    subgraph Before["Before Tapering"]
        direction TB
        BQ["n qubits"]
        BT["m Pauli terms"]
        BH["2ⁿ Hilbert space"]
    end
    subgraph After["After Tapering (k qubits removed)"]
        direction TB
        AQ["n−k qubits"]
        AT["≤ m terms (often fewer)"]
        AH["2ⁿ⁻ᵏ Hilbert space"]
    end
    Before --> |"Taper"| After
    style After fill:#d1fae5,stroke:#059669
```

What shrinks	Factor	Example (12 → 9 qubits)
Qubit count	$-k$	3 fewer physical qubits needed
Hilbert space	2^{-k}	$4096 \to 512$ (8× smaller)
Circuit width	$-k$	3 fewer wires in every gate layer
Pauli weight	often reduces	Shorter Z-chains after qubit removal
Term count	often reduces	Some terms collapse to identity

And these savings **compound** with encoding choice. A ternary-tree encoding on a tapered Hamiltonian gets both the $O(\log_3 n)$ weight advantage *and* the reduced qubit count.

The Two Levels of Tapering

FockMap implements two levels of increasing generality:

Level 1: Diagonal Z_2

A qubit j is *diagonally taperable* if every term in the Hamiltonian has only I or Z at position j — never X or Y. This means qubit j's value is determined by symmetry: it is always $+1$ or always -1 in the sector we care about, and it never gets flipped during the simulation. We can fix it to that value and remove it from the problem entirely.

How you detect it: Look at each qubit position across all Pauli terms. If you never see X or Y in that column, the qubit is taperable. It's a simple scan — FockMap checks this in one pass over the Hamiltonian.

Level 2: General Clifford

Sometimes no *single* qubit is purely diagonal, but a *combination* of qubits is. For example, $Z_0 Z_1$ might commute with every Hamiltonian term even though Z_0 alone does not. This means the *product* of qubits 0 and 1 is conserved, even though neither one is individually.

In this case, we can apply a small rotation circuit (built from Hadamard, S, and CNOT gates — the same gates from Chapter 4) that rearranges the Hamiltonian so that the conserved combination ends up on a single qubit. After the rotation, that qubit is diagonally taperable, and we remove it just like in Level 1.

How you detect it: FockMap searches for Pauli operators that commute with every term in the Hamiltonian — these are the symmetry generators. The mathematical machinery behind this search involves binary linear algebra, which we'll develop step by step in Chapter 11.

The important thing at this stage is the *idea*: even when the symmetry isn't visible on a single qubit, it may be hiding in a combination of qubits, and a rotation can expose it.

We'll work through the diagonal case in Chapter 10, the Clifford generalization in Chapter 11, and concrete benchmarks in Chapter 12.

Why Tapering Is Free

It's worth emphasizing: tapering does not approximate. It does not truncate. It does not discard information.

When we remove a tapered qubit, we are removing a degree of freedom whose value was *already determined* — fixed by a conservation law (like particle number or spin parity) that the Hamiltonian respects. The qubit was never free to vary in the first

place; it was constrained by symmetry to a single eigenvalue in the sector we're studying. Removing it simply acknowledges this constraint explicitly.

The eigenvalues of the tapered Hamiltonian are a *subset* of the eigenvalues of the original — specifically, the eigenvalues in the chosen symmetry sector. No eigenvalue is lost; we just stop tracking the ones that belong to other sectors.

This is why we taper *before* Trotterization, not after: the circuit should operate on the physically relevant Hilbert space from the start, not carry redundant qubits through every gate layer.

Key Takeaways

- Encoded Hamiltonians often have Z_2 symmetries — qubits whose value is determined by conservation laws rather than dynamics.
- Removing these qubits is free: it preserves the physics exactly while reducing every downstream cost.
- Diagonal Z_2 symmetries (single-qubit Z generators) are the simplest case. General Z_2 symmetries require Clifford rotation.
- Tapering compounds with encoding choice: taper first, then the encoding operates on a smaller system.

Common Mistakes

1. **Assuming H_2 has taperable qubits under JW.** It doesn't — the exchange terms (XXYY) put X/Y on every qubit. Tapering is most effective for larger molecules with more conservation laws.

2. **Confusing tapering with truncation.** Truncation (e.g., active-space reduction) discards orbitals and loses information. Tapering removes redundant *qubits* without losing any information — the eigenvalues are exactly preserved.

3. **Tapering after circuit compilation.** Taper before Trotterization, not after. The circuit should be built on the smaller Hamiltonian.

Exercises

1. **Conservation laws.** For a molecule with N electrons, what conservation laws might produce Z_2 symmetries? (Hint: total electron number, spin projection S_z, point-group symmetry.)

2. **Parity encoding advantage.** Under the Parity encoding, the last qubit stores the total electron-number parity. Show that this qubit is always diagonally taperable for any Hamiltonian that conserves particle number.

3. **Scaling impact.** If tapering removes 3 qubits from a 14-qubit H_2O system, by what factor does the Hilbert space shrink? How many fewer CNOT gates does each Trotter step require (approximately)?

Further Reading

- Bravyi, S., Gambetta, J. M., Mezzacapo, A., and Temme, K. "Tapering off qubits to simulate fermionic Hamiltonians." arXiv:1701.08213 (2017). The foundational paper on qubit tapering.

Previous: Chapter 8 — Checking Our Answer

Next: Chapter 10 — Diagonal Z_2 Symmetries

Chapter 10: Diagonal Z_2 Symmetries

The simplest form of tapering: find qubits that are always I or Z, fix their eigenvalue, and remove them.

In This Chapter

- **What you'll learn:** The algorithm for detecting diagonal Z_2 symmetries, how sector choice works, and how fixing a sector modifies each Pauli term.
- **Why this matters:** Diagonal tapering is fast, easy to implement, and often removes 1–3 qubits from molecular Hamiltonians with zero information loss.
- **Prerequisites:** Chapter 8 (you understand why tapering is valuable and what Z_2 symmetries are).

The Detection Algorithm

Given a Hamiltonian $\hat{H} = \sum_\alpha c_\alpha \sigma_\alpha$, we say qubit j is **diagonal Z_2 symmetric** if:

$$\forall \alpha : \ \sigma_\alpha[j] \in \{I, Z\}$$

The algorithm is a single scan:

```
flowchart TD
    START["For each qubit j = 0, 1, ..., n-1"] --> SCAN["Inspect every Pauli term σ_α"]
    SCAN --> CHECK{"σ_α[j] ∈ {I, Z}?"}
    CHECK -->|"All terms: yes"| DIAG["Qubit j is diagonal Z₂ ✓"]
    CHECK -->|"Any term: X or Y"| SKIP["Qubit j is NOT diagonal ✗"]
    style DIAG fill:#d1fae5,stroke:#059669
    style SKIP fill:#fee2e2,stroke:#ef4444
```

Complexity: $O(n \times m)$ where n is the qubit count and m is the number of terms. For H_2O with 12 qubits and ~600 terms, this takes microseconds.

In FockMap

```
let symQubits = diagonalZ2SymmetryQubits hamiltonian
// Returns int[] of taperable qubit indices
```

For our toy example from Chapters 8:

```
let h =
    [| PauliRegister("ZIZI", Complex(0.8, 0.0))
       PauliRegister("ZZII", Complex(-0.4, 0.0))
       PauliRegister("IIZZ", Complex(0.3, 0.0))
       PauliRegister("IZIZ", Complex(0.2, 0.0)) |]
    |> PauliRegisterSequence

diagonalZ2SymmetryQubits h
// → [| 0; 1; 2; 3 |]  — all four qubits are diagonal!
```

Every term has only I or Z at every position. This is a fully diagonal Hamiltonian — entirely classical. It's a toy, but it illustrates the detection perfectly.

A more realistic example

In practice, molecular Hamiltonians are *mixed*: some qubits are diagonal, others are not. Consider a Hamiltonian where qubits 0 and 2 are always I/Z, but qubits 1 and 3 have X and Y terms:

```
let hmixed =
    [| PauliRegister("ZIZI", Complex(0.5, 0.0))   // q0=Z, q1=I, q2=Z, q3=I
       PauliRegister("IXIX", Complex(-0.3, 0.0))   // q0=I, q1=X, q2=I, q3=X
       PauliRegister("ZIIZ", Complex(0.2, 0.0))    // q0=Z, q1=I, q2=I, q3=Z
       PauliRegister("IYIY", Complex(0.1, 0.0)) |] // q0=I, q1=Y, q2=I, q3=Y
    |> PauliRegisterSequence

diagonalZ2SymmetryQubits hmixed
// → [| 0; 2 |]
```

Qubit 0 has Z, I, Z, I across the four terms — all diagonal. Qubit 2 has Z, I, I, I — also all diagonal. But qubit 1 has I, X, I, Y — the X and Y disqualify it. Qubit 3 has I, X, Z, Y — also disqualified.

Only qubits 0 and 2 can be tapered. The off-diagonal terms (the ones with X and Y that generate coherences, as we discussed in Chapter 6) survive on qubits 1 and 3 — which is exactly right, because the quantum physics lives in those off-diagonal terms and we must not discard them.

> **The intuition:** tapering removes qubits whose value is *classically determined* — they contribute only to the diagonal part of the Hamiltonian. The qubits we keep are the ones that carry the off-diagonal (quantum) physics.

Sectors: Choosing Eigenvalues

For each diagonal Z_2 qubit, we **choose** whether to fix its Z eigenvalue to $+1$ or -1. This choice is called a **sector**.

A sector is a list of (qubit, eigenvalue) pairs:

```
let sector = [ (1, +1); (3, -1) ]
// Fix qubit 1 to eigenvalue +1, qubit 3 to eigenvalue -1
```

Physical interpretation: Different sectors correspond to different quantum numbers. If the diagonal qubits encode particle-number parity or spin projection, then sector $+1$ vs -1 selects different electron counts or spin states.

For example, if qubit j represents the parity of the total electron number (even vs odd), then: - Sector $+1$ → even number of electrons - Sector -1 → odd number of electrons

If you know your molecule has 2 electrons (even), you choose sector $+1$ for that qubit. This projects the Hamiltonian onto the physically relevant subspace.

For k diagonal qubits, there are 2^k possible sectors. Each gives a valid tapered Hamiltonian with the correct eigenvalues for that sector — but different sectors may have different ground-state energies. To find the global ground state, you must check all sectors and take the minimum.

How Sector Fixing Modifies Terms

The modification rule is simple:

For each term $c_\alpha \sigma_\alpha$ and each tapered qubit (j, λ):

1. If $\sigma_\alpha[j] = I$: no change to the coefficient
2. If $\sigma_\alpha[j] = Z$: multiply the coefficient by λ (the eigenvalue: $+1$ or -1)
3. Remove position j from the Pauli string

Worked Example

Start with:

$$\hat{H} = 0.8\,\mathrm{ZIZI} - 0.4\,\mathrm{ZZII} + 0.3\,\mathrm{IIZZ} + 0.2\,\mathrm{IZIZ}$$

Fix sector $[(1, +1),\ (3, -1)]$:

Original term	Qubit 1	Factor	Qubit 3	Factor	New coeff	Remaining Pauli
$0.8\,\mathrm{ZIZI}$	I	$\times 1$	I	$\times 1$	0.8	ZZ
$-0.4\,\mathrm{ZZII}$	Z	$\times(+1)$	I	$\times 1$	-0.4	ZI

Original term	Qubit 1	Factor	Qubit 3	Factor	New coeff	Remaining Pauli
$0.3\,\mathrm{IIZZ}$	I	$\times 1$	Z	$\times(-1)$	-0.3	IZ
$0.2\,\mathrm{IZIZ}$	Z	$\times(+1)$	Z	$\times(-1)$	-0.2	II

Result: $\hat{H}' = 0.8\,\mathrm{ZZ} - 0.4\,\mathrm{ZI} - 0.3\,\mathrm{IZ} - 0.2\,\mathrm{II}$

Four qubits → two qubits. The eigenvalues of $\hat{H}'$ are exactly the eigenvalues of $\hat{H}$ in the $(+1, -1)$ sector.

In FockMap

```
let result = taperDiagonalZ2 [ (1, 1); (3, -1) ] h

printfn "%d → %d qubits" result.OriginalQubitCount result.TaperedQubitCount
// 4 → 2 qubits

printfn "Removed: %A" result.RemovedQubits
// [| 1; 3 |]
```

The Convenience Helper

For quick exploration, taper all detected qubits in the $+1$ sector:

```
let auto = taperAllDiagonalZ2WithPositiveSector hamiltonian

// Equivalent to:
// let qs = diagonalZ2SymmetryQubits hamiltonian
// let sector = qs |> Array.map (fun q -> (q, 1)) |> Array.toList
// let auto = taperDiagonalZ2 sector hamiltonian
```

This is fine for exploration but not for production: the $+1$ sector may not contain the ground state. For a rigorous calculation, sweep all 2^k sectors.

Validation

FockMap validates all inputs:

```
// Non-diagonal qubit → error
taperDiagonalZ2 [(0, 1)] (prs [("XI", Complex.One)])
// → ArgumentException: "Qubit 0 is not a diagonal Z2 symmetry"
```

```
// Invalid eigenvalue → error
taperDiagonalZ2 [(0, 0)] hamiltonian
// → ArgumentException: "Sector eigenvalues must be +1 or -1"
```

These guards prevent the two most common tapering bugs: applying tapering to a qubit that has X or Y terms (which would silently drop those terms), and using an eigenvalue other than ± 1 (which would corrupt the coefficients).

Key Takeaways

- Detection is a single scan: $O(n \times m)$.
- Sector choice selects which quantum-number subspace to project into.
- Each term's coefficient is multiplied by ± 1 per tapered qubit (depending on I vs Z), then the qubit position is removed.
- For the global ground state, sweep all 2^k sectors.
- FockMap validates inputs to prevent silent errors.

Sector Selection: A Practical Workflow

Choosing the correct sector is a common source of confusion for first-time practitioners. Here is a concrete decision procedure:

1. **If you know the conserved quantities** (e.g., particle number parity, spin projection), compute the expected eigenvalue for each generator and set the sector accordingly. For molecular ground states, the particle number is fixed, so parity-related generators should be set to match the electron count.

2. **If the physical sector is unclear**, sweep all 2^k sectors. For each sector, diagonalise (or VQE-evaluate) the tapered Hamiltonian and record the ground-state energy. The global minimum across sectors is the ground-state energy. For H_2 with 2 diagonal qubits, this means 4 sector evaluations — trivial. For larger systems with $k = 3$ or 4, it means 8–16 evaluations — still cheap compared to the Hamiltonian construction.

3. **For production workflows**, always verify: compare the tapered ground-state energy against the untapered eigenvalue (if computable) or against a known reference. A sector mismatch shows up as a ground-state energy that is *higher* than the correct value — never lower.

```
// Sweep all sectors for a 2-generator system
for s0 in [+1; -1] do
    for s1 in [+1; -1] do
        let result = taperDiagonalZ2 [(q0, s0); (q1, s1)] hamiltonian
        let e0 = exactGroundStateEnergy result.Hamiltonian
        printfn "Sector (%+d, %+d): E₀ = %.6f Ha" s0 s1 e0
```

Common Mistakes

1. **Assuming the $+1$ sector always contains the ground state.** It may not — the ground state could be in any sector.
2. **Tapering a non-diagonal qubit.** FockMap catches this, but if you're implementing by hand, silently dropping X/Y terms is the most dangerous bug.
3. **Forgetting to combine like terms after tapering.** Removing qubits can make previously distinct Pauli strings identical. The terms must be re-accumulated.

Previous: Chapter 9 — Why Tapering?

Next: Chapter 11 — General Clifford Tapering

Chapter 11: General Clifford Tapering

When no single qubit is diagonal, multi-qubit Z_2 symmetries may still exist. A Clifford rotation reveals them.

In This Chapter

- **What you'll learn:** The symplectic representation of Pauli strings, how to find all Z_2 symmetry generators via GF(2) linear algebra, how to synthesize a Clifford circuit that makes them diagonal, and how FockMap's unified `taper` function combines everything.
- **Why this matters:** Many molecular Hamiltonians have Z_2 symmetries that diagonal-only tapering cannot exploit. Clifford tapering finds and uses them all.
- **Prerequisites:** Chapters 8–9 (diagonal tapering concepts and mechanics).

The Limitation of Diagonal Tapering

Diagonal tapering requires a qubit where *every* Hamiltonian term is I or Z. But consider the 2-qubit Heisenberg model:

$$\hat{H} = J(X_0X_1 + Y_0Y_1 + Z_0Z_1)$$

No single qubit is diagonal — both qubits have X, Y, and Z terms. Diagonal tapering finds nothing.

But the operator Z_0Z_1 **commutes** with every term: $[Z_0Z_1, X_0X_1] = [Z_0Z_1, Y_0Y_1] = [Z_0Z_1, Z_0Z_1] = 0$. It is a valid Z_2 symmetry generator — it just involves *two* qubits instead of one.

If we could rotate Z_0Z_1 onto a single-qubit Z — say, Z_0I_1 — then qubit 0 would become diagonally taperable. The Clifford rotation that achieves this is a simple CNOT.

The Binary Pauli Representation

To find all Z_2 generators systematically, we need a way to represent Pauli strings that makes commutativity easy to check. The trick is to encode each Pauli operator as **two binary bits**:

Pauli	X bit	Z bit	Think of it as…
I	0	0	No flip, no phase
X	1	0	Bit-flip only
Y	1	1	Both flip and phase
Z	0	1	Phase-flip only

An n-qubit Pauli string becomes a binary vector of length $2n$ — the X-bits followed by the Z-bits:

$$\sigma \;\leftrightarrow\; (\underbrace{x_0, x_1, \dots, x_{n-1}}_{\text{X bits}} \mid \underbrace{z_0, z_1, \dots, z_{n-1}}_{\text{Z bits}})$$

```
let sv = toSymplectic (PauliRegister("XYZ", Complex.One))
// sv.X = [| true; true; false |]   — X has x=1, Y has x=1, Z has x=0
// sv.Z = [| false; true; true |]   — X has z=0, Y has z=1, Z has z=1
```

> **On the name "symplectic":** The quantum computing literature calls this the *symplectic representation*, and the commutativity check below a *symplectic inner product*. The word "symplectic" comes from Greek for "intertwined" — it refers to the fact that commutativity depends on a *crosswise* pairing between the X-bits of one string and the Z-bits of the other (and vice versa). This crosswise structure is what mathematicians call a symplectic form. We'll use the name "binary Pauli representation" when the intuition matters and "symplectic" when referencing the literature.

Why this representation?

The payoff: two Pauli strings commute if and only if their **crosswise dot product** is zero (mod 2):

$$\text{commute?} \quad \sum_{i=0}^{n-1} (a_{x_i} \cdot b_{z_i} + a_{z_i} \cdot b_{x_i}) \stackrel{?}{=} 0 \pmod 2$$

Notice the crosswise structure: we pair the X-bits of a with the Z-bits of b, and vice versa. If the sum is even, they commute. If odd, they anti-commute. This reduces commutativity checking to a **binary dot product** — something a computer can do in nanoseconds.

```
let a = toSymplectic (PauliRegister("XX", Complex.One))
let b = toSymplectic (PauliRegister("ZZ", Complex.One))
commutes a b  // true — XX and ZZ commute
```

Finding All Symmetry Generators

A Pauli string g is a Z_2 symmetry of $\hat{H}$ if it commutes with every term and squares to the identity. Since every Pauli string squares to $\pm I$, the squaring condition is automatic.

The commutation condition crosswise dot product $= 0$ for all terms t_k is a system of linear equations where the arithmetic is binary (0 and 1, with addition meaning XOR). The solution set — the **null space** of the commutation check matrix — gives all Z_2 generators.

```
let generators = findCommutingGenerators hamiltonian
// Returns SymplecticVector[] — all Pauli strings that commute with every term

let indep = independentGenerators generators
// Selects a maximal linearly independent subset (binary linear algebra)
// The number of independent generators = max qubits taperable
```

The null space computation uses Gaussian elimination with XOR instead of subtraction — the same row-reduction you learned in linear algebra, but in binary arithmetic. It runs in $O(n^3)$ time in the number of qubits n, where the matrix has L rows (one per Hamiltonian term) and $2n$ columns (the symplectic vector length). In practice, this is dominated by the $O(n^4)$ cost of integral processing and is never the bottleneck. See Bravyi et al. (arXiv:1701.08213, §III) for the formal analysis.

Clifford Rotation: Making Generators Diagonal

Once we have independent generators, we need a Clifford circuit U such that:

$$U g_i U^\dagger = Z_{q_i} \quad \text{for each generator } g_i$$

This rotates each multi-qubit generator onto a single-qubit Z, making the system diagonally taperable.

> **Algorithm: Clifford Tapering**
>
> **Input:** A Pauli Hamiltonian $\hat{H} = \sum_k c_k P_k$ on n qubits.
>
> **Output:** A reduced Hamiltonian on $n - m$ qubits, where m is the number of independent Z_2 symmetries.

1. **Represent** each term P_k as a $2n$-bit symplectic vector.
2. **Build** the $L \times 2n$ commutation check matrix (one row per term).
3. **Compute** its null space via binary Gaussian elimination → independent generators $g_1, \ldots, g_m$.
4. **For each** generator g_i: find a qubit q_i where g_i has support. If the support is X, apply H to convert to Z. If Y, apply S then H. Use CNOTs to clear all other qubits, leaving $g_i \to Z_{q_i}$. Collect the gate list.
5. **Conjugate** every term P_k by the collected Clifford gates (symbolically — substitution rules on symplectic vectors).
6. **Fix** each target qubit q_i to eigenvalue ± 1 (the sector choice) and remove it.

Complexity: Step 3 is $O(n^3)$; step 5 is $O(Lm)$ where L is the number of terms. Total is dominated by Hamiltonian construction, not tapering.

FockMap synthesizes this circuit using three elementary gates:

Gate	Symbol	Effect on Pauli
Hadamard	H_j	Swaps X↔Z on qubit j
Phase gate	S_j	Maps X→Y on qubit j (Z unchanged)
CNOT	$\text{CNOT}_{c,t}$	Propagates X from control to target; propagates Z from target to control

The synthesis algorithm: 1. For each generator, find a qubit where it has support (X, Y, or Z bit set). 2. If the support is X, apply H to convert to Z. If Y, apply S then H. 3. Use CNOTs to clear all other qubits' support, leaving Z on exactly one qubit.

```
let (gates, targets) = synthesizeTaperingClifford independentGens
// gates : CliffordGate list — the rotation circuit
// targets : int[] — which qubit each generator maps to
```

Applying the Clifford to the Hamiltonian

The Clifford circuit is applied **symbolically** — no matrices, no state vectors. Each Pauli term is conjugated: $\sigma_\alpha \to U\sigma_\alpha U^\dagger$. This is a sequence of substitution rules applied to the symplectic vector:

```
let rotatedH = applyClifford gates hamiltonian
// Every term is now conjugated — generators have become single-qubit Zs
```

After rotation, the target qubits are diagonally taperable, and we apply the v1 diagonal tapering from Chapter 9.

The Unified Pipeline

FockMap's `taper` function combines everything:

```
// Full Clifford tapering (default)
let result = taper defaultTaperingOptions hamiltonian

// Diagonal only (v1 fallback)
let result = taper { defaultTaperingOptions with Method = DiagonalOnly } h

// Cap removal
let result = taper { defaultTaperingOptions with MaxQubitsToRemove = Some 2 } h

// Explicit sector
let result = taper { defaultTaperingOptions with Sector = [(0,1); (1,-1)] } h
```

The result includes everything you need:

```
result.OriginalQubitCount  // before tapering
result.TaperedQubitCount   // after tapering
result.RemovedQubits       // which qubits were removed
result.Generators          // the Z₂ generators found
result.CliffordGates       // the rotation circuit applied
result.TargetQubits        // which qubits the generators mapped to
result.Hamiltonian         // the tapered PauliRegisterSequence
```

Worked Example: Heisenberg Model

$$\hat{H} = X_0X_1 + Y_0Y_1 + Z_0Z_1$$

This is a 2-qubit Hamiltonian where every qubit has X, Y, and Z terms. Diagonal tapering sees nothing at all. But there *is* a hidden symmetry — let's find it and use it.

Step 1: Check for diagonal Z_2 qubits.

Qubit 0 appears as: X, Y, Z across the three terms. Not diagonal (has X and Y). Qubit 1: same story. Diagonal tapering finds zero candidates.

Step 2: Find general Z_2 generators.

We need Pauli strings that commute with every term. Let's check Z_0Z_1:

- Does Z_0Z_1 commute with X_0X_1? Use the crosswise dot product: X-bits of ZZ are $(0,0)$, Z-bits are $(1,1)$. X-bits of XX are $(1,1)$, Z-bits are $(0,0)$. Crosswise sum: $(0 \cdot 0 + 1 \cdot 1) + (0 \cdot 0 + 1 \cdot 1) = 2$, which is even → **commute** □
- Does Z_0Z_1 commute with Y_0Y_1? X-bits of YY are $(1,1)$, Z-bits are $(1,1)$. Crosswise sum: $(0 \cdot 1 + 1 \cdot 1) + (0 \cdot 1 + 1 \cdot 1) = 2$ → **commute** □
- Does Z_0Z_1 commute with itself? Always yes □

Z_0Z_1 is a Z_2 generator. One independent generator → one qubit can be tapered.

Step 3: Clifford synthesis — rotate Z_0Z_1 onto a single qubit.

We want a circuit U such that $U(Z_0Z_1)U^\dagger = Z_0I_1$ — the generator acts on only one qubit after rotation.

The key gate: **CNOT(1 → 0)** (qubit 1 is control, qubit 0 is target). Recall from Chapter 4 that CNOT propagates Z from target to control. So:

$$\text{CNOT}_{1,0}: \quad Z_0 \to Z_0, \quad Z_1 \to Z_0Z_1$$

Wait — that goes the wrong way (it *creates* Z_0Z_1, not removes it). We need the reverse: **CNOT(0 → 1)** (qubit 0 is control, qubit 1 is target):

$$\text{CNOT}_{0,1}: \quad Z_0 \to Z_0Z_1, \quad Z_1 \to Z_1$$

That also doesn't isolate it. Let's think more carefully. Under CNOT conjugation, the Z propagation rule is: $Z_{\text{target}} \to Z_{\text{control}} \cdot Z_{\text{target}}$. So if we apply CNOT(0,1) to Z_0Z_1:

$$Z_0Z_1 \xrightarrow{\text{CNOT}_{0,1}} Z_0 \cdot (Z_0Z_1) = Z_0^2Z_1 = I \cdot Z_1 = Z_1$$

Now Z_0Z_1 has become Z_1 — a single-qubit Z on qubit 1!

> **What just happened:** The CNOT "absorbed" the Z_0 factor by multiplying it with another Z_0, leaving only Z_1. This is the Clifford rotation — it's not a physical rotation in space, but an algebraic simplification achieved by conjugation with a gate.

Step 4: Apply the CNOT to every Hamiltonian term.

Under CNOT(0,1) conjugation, each term transforms:

Original	After CNOT(0,1)
X_0X_1	X_0X_1 → actually transforms to $X_0I_1 \cdot I_0X_1 = X_0X_1$...

Actually, let's let FockMap do this — the gate-by-gate algebra is fiddly by hand. The point is that after applying the Clifford, one of the qubits (the target of the rotation) now has only I or Z across all terms — it has become diagonally taperable.

Step 5: Diagonal taper. Fix the newly-diagonal qubit to sector $+1$, remove it. The result is a 1-qubit Hamiltonian.

```
let heis =
    [| PauliRegister("XX", Complex(1.0, 0.0))
       PauliRegister("YY", Complex(1.0, 0.0))
       PauliRegister("ZZ", Complex(1.0, 0.0)) |]
```

```
    |> PauliRegisterSequence

let result = taper defaultTaperingOptions heis
printfn "%d → %d qubits" result.OriginalQubitCount result.TaperedQubitCount
// 2 → 1 qubit
```

A 2-qubit problem reduced to 1 qubit — a 2× Hilbert space reduction that diagonal-only tapering would have missed entirely.

Key Takeaways

- The **binary Pauli representation** (two bits per qubit) turns commutativity checking into a crosswise binary dot product — fast and exact.
- The **null space** of the commutation check matrix (computed by binary Gaussian elimination) gives all Z_2 generators.
- **Clifford synthesis** rotates multi-qubit generators onto single-qubit Zs using H, S, and CNOT — no matrices needed.
- **The unified `taper` function** handles both diagonal and Clifford tapering with one API.
- Everything is symbolic and exact — no approximation, no eigensolvers, no numerical instability.

Further Reading

- Bravyi, S. et al. "Tapering off qubits to simulate fermionic Hamiltonians." arXiv:1701.08213 (2017). The general Z_2 tapering framework.
- Aaronson, S. and Gottesman, D. "Improved simulation of stabilizer circuits." *Phys. Rev. A* 70, 052328 (2004). The tableau formalism underlying our Clifford implementation.

Previous: Chapter 10 — Diagonal Z_2 Symmetries

Next: Chapter 12 — Tapering Benchmarks

Chapter 12: Tapering Benchmarks

Numbers, not promises. This chapter measures exactly how much tapering saves on real Hamiltonians.

In This Chapter

- **What you'll learn:** Concrete before-and-after measurements of qubit count, term count, Pauli weight, and estimated CNOT cost for tapering on multiple test systems.
- **Why this matters:** Tapering sounds good in theory. This chapter shows exactly how good — and where the limits are.
- **Prerequisites:** Chapters 8–10 (you understand both diagonal and Clifford tapering).

Methodology

For each test system, we: 1. Build the encoded Hamiltonian (JW encoding) 2. Count: qubits, terms, max weight, total CNOT cost per Trotter step 3. Apply tapering (all detected symmetries, +1 sector) 4. Re-count the same metrics on the tapered Hamiltonian 5. Report the reduction

CNOT cost per Trotter step is estimated as $\sum_k 2(w_k - 1)$ where w_k is the weight of term k.

Benchmark Results

Fully diagonal 6-qubit Hamiltonian

A synthetic Hamiltonian where all terms are I/Z only — the best case for tapering.

```
let h6 =
    [| PauliRegister("ZIIIII", Complex(0.5, 0.0))
       PauliRegister("IZIIII", Complex(-0.3, 0.0))
       PauliRegister("IIZIII", Complex(0.8, 0.0))
       PauliRegister("IIIZII", Complex(0.2, 0.0))
       PauliRegister("IIIIZI", Complex(-0.4, 0.0))
       PauliRegister("IIIIIZ", Complex(0.7, 0.0))
       PauliRegister("ZZZZII", Complex(0.1, 0.0))
       PauliRegister("IIZZZZ", Complex(-0.2, 0.0)) |]
    |> PauliRegisterSequence
```

Metric	Before	After	Reduction
Qubits	6	0	100%
Terms	8	1	88%
Hilbert space	64	1	64×
CNOTs/step	14	0	100%

Interpretation: All symmetries were diagonal. The entire Hamiltonian collapsed to a scalar — the energy eigenvalue in the +1 sector. This is the extreme case: a fully classical Hamiltonian with no quantum content.

Mixed Hamiltonian (partial tapering)

A 4-qubit system where qubits 0 and 2 are diagonal but qubits 1 and 3 have X/Y terms.

```
let hmixed =
    [| PauliRegister("ZIZI", Complex(0.5, 0.0))
       PauliRegister("IXIX", Complex(-0.3, 0.0))
       PauliRegister("ZIIZ", Complex(0.2, 0.0))
       PauliRegister("IYIY", Complex(0.1, 0.0)) |]
    |> PauliRegisterSequence
```

Metric	Before	After	Reduction
Qubits	4	2	50%
Terms	4	4	0%
Hilbert space	16	4	4×
Max weight	2	2	0%

Interpretation: Half the qubits removed, but the term count and weight stay the same — the remaining terms still have structure. The Hilbert space shrank by 4×, which is the primary gain.

Heisenberg model (Clifford needed)

$\hat{H} = XX + YY + ZZ$ on 2 qubits. No diagonal Z_2 symmetries, but $Z_0 Z_1$ is a general Z_2 generator.

Metric	Diagonal only	Clifford
Symmetries found	0	≥1
Qubits removed	0	≥1

Interpretation: Diagonal-only tapering misses the symmetry entirely. Clifford tapering finds and uses it. This is why the general method matters.

The Impact on Circuit Cost

The real payoff of tapering shows in the **CNOT staircase** (Chapter 14). Each Pauli rotation $e^{-i\theta P}$ with weight w costs $2(w-1)$ CNOTs. Tapering reduces both term count and weight:

System	Terms before	Terms after	CNOTs/step before	CNOTs/step after	Savings
6-qubit diagonal	8	1	14	0	100%
4-qubit mixed	4	4	8	4	50%

These savings multiply across every Trotter step. For a simulation with 1000 Trotter steps, the 50% reduction in the mixed case means 4000 fewer CNOTs total.

Tapering + Encoding: Compounding Savings

Tapering and encoding choice address different aspects of circuit cost, and they **compound**:

```
flowchart LR
    H["Original Hamiltonian<br/>n qubits, O(n) weight"]
    H --> T["After tapering<br/>n-k qubits"]
    T --> E["After encoding choice<br/>O(log₃(n-k)) weight"]
    style E fill:#d1fae5,stroke:#059669
```

For a 14-qubit H_2O system (STO-3G, frozen core):

Configuration	Qubits	Max weight (JW)	Max weight (TT)	Est. CNOTs/step
No tapering, JW	14	14	—	~1800
No tapering, TT	14	—	4	~500
Tapered, JW	11	11	—	~1100
Tapered, TT	11	—	4	~380

The combination of tapering (14→11 qubits) and ternary tree encoding (weight 14→4) gives roughly a 5× total reduction in CNOT cost.

Stage 3 Complete

```
flowchart LR
    S1["Stage 1<br/>Integrals"]
    S2["Stage 2<br/>Encoding"]
    S3["Stage 3<br/>Tapering ✓"]
    S4["Stage 4<br/>Trotterization"]
    S1 --> S2 --> S3 --> S4
    style S3 fill:#d1fae5,stroke:#059669
    style S4 fill:#fde68a,stroke:#d97706
```

We now have a verified, tapered qubit Hamiltonian — smaller than what encoding alone produced, with all physics preserved exactly. The next stage turns this Hamiltonian into a sequence of quantum gates.

Key Takeaways

- Tapering reduces qubit count, Hilbert space size, and often term count and Pauli weight.
- Diagonal tapering handles the easy cases; Clifford tapering catches multi-qubit symmetries that diagonal misses.
- The savings compound with encoding choice: taper first, then the encoding operates on a smaller system.
- The real metric is **CNOTs per Trotter step** — tapering can reduce this by 50% or more.

Previous: Chapter 11 — General Clifford Tapering

Next: Chapter 13 — From Hamiltonian to Time Evolution

Chapter 13: From Hamiltonian to Time Evolution

We have a Hamiltonian — a complete description of the molecule's physics. But a quantum computer can't execute a description. It needs instructions. This chapter is about turning the description into instructions.

In This Chapter

- **What you'll learn:** Why quantum simulation boils down to implementing time evolution e^{-iHt}, what this operator physically represents, why it's hard to implement when the Hamiltonian has many non-commuting terms, and how Trotterization provides the practical bridge.
- **Why this matters:** Without this bridge, the pipeline from molecule to circuit is incomplete. This is the chapter that turns a symbolic object into something a quantum processor can actually run.
- **Prerequisites:** Chapters 1–12 (you have a verified, optionally tapered, Pauli-sum Hamiltonian and understand gates from Chapter 4).

What We Have, and What We Need

After twelve chapters, our pipeline has produced a remarkable object: a symbolic Pauli-sum Hamiltonian

$$\hat{H} = \sum_{k=1}^{L} c_k P_k$$

verified to give the correct eigenvalues, potentially tapered to use fewer qubits, with encoding choice optimizing the Pauli weight. This object captures *everything* about the molecule's electronic structure — every orbital energy, every Coulomb repulsion, every quantum exchange interaction.

But it is a *description*, not a *program*. A quantum computer doesn't accept descriptions. It accepts a sequence of gates — Hadamard, CNOT, R_z — applied to specific

qubits in a specific order. The Hamiltonian tells us *what* to compute; the circuit tells the machine *how* to compute it.

To cross this gap, we need to understand a deep idea: the connection between the Hamiltonian (which describes energy) and time evolution (which describes change).

Energy and Time: Two Sides of the Same Operator

Here is a fact that physics students learn early but whose implications take years to fully appreciate:

> **The Hamiltonian does double duty.** It tells you the energy of a system *and* it tells you how the system evolves in time. The same operator. Both roles.

The Schrödinger equation makes this precise:

$$i\hbar \frac{d}{dt}|\psi(t)\rangle = \hat{H}|\psi(t)\rangle$$

Read it as: "the rate at which the quantum state changes equals the Hamiltonian applied to that state." The Hamiltonian is both the *energy observable* (its eigenvalues are the energies) and the *generator of time evolution* (it drives the dynamics).

The formal solution for a time-independent Hamiltonian is:

$$|\psi(t)\rangle = e^{-i\hat{H}t/\hbar}|\psi(0)\rangle$$

Setting $\hbar = 1$ (as is standard in atomic units), the **time-evolution operator** is:

$$U(t) = e^{-i\hat{H}t}$$

This operator is unitary — it preserves probabilities, as any physical evolution must. Its eigenvalues are $e^{-iE_k t}$, where E_k are the energy eigenvalues. The energies are *encoded as phases* of the time-evolution operator.

This dual role is what makes quantum simulation work: if you can implement $U(t)$ on a quantum computer, you can extract the energies by reading the phases. That's the core idea behind both QPE (which reads the phases directly) and VQE (which estimates $\langle H \rangle$ by measuring individual Pauli terms).

The Gap in Our Pipeline

Let's pause and take stock. Here's the complete pipeline so far:

```
flowchart LR
    MOL["Molecule<br/>(Ch 1-3)"] --> GATES["Gates<br/>(Ch 4)"]
    GATES --> ENC["Encoding<br/>(Ch 5-7)"]
    ENC --> VER["Verified<br/>(Ch 8)"]
    VER --> TAP["Tapered<br/>(Ch 9-12)"]
    TAP --> |"?"| CIRC["Circuit"]
    style CIRC fill:#fde68a,stroke:#d97706
```

We have a verified, optionally tapered Hamiltonian:

$$\hat{H} = \sum_{k=1}^{L} c_k P_k$$

where each P_k is a Pauli string (like $XXYY$) and c_k is a real coefficient. This is a complete, exact symbolic description of the molecule's physics.

But a quantum computer doesn't accept a Hamiltonian as input. It accepts a sequence of **quantum gates**. We need to cross the last gap:

$$\text{Hamiltonian } \hat{H} \xrightarrow{?} \text{ Gate sequence}$$

An important clarification

The phrase "quantum simulation" is misleading. We are **not** watching electrons move in real time, like a molecular dynamics animation. We are not running the chemistry forward. Instead, we are using time evolution as a *mathematical tool*: implement $U(t) = e^{-iHt}$, then extract the energies from its phases.

- **QPE** (Quantum Phase Estimation) applies controlled-$U(t)$ and reads the energy eigenvalue directly as a binary phase, written into an ancilla register.
- **VQE** (Variational Quantum Eigensolver) doesn't use $U(t)$ as a whole, but measures $\langle \hat{H} \rangle = \sum_k c_k \langle P_k \rangle$ term by term — and each measurement involves a Pauli rotation $e^{-i\theta P_k}$, which is a single-term slice of the time-evolution operator.

In both cases, the primitive operation is a **Pauli rotation**: $e^{-i\theta P}$ for a single Pauli string P. The question is: how do we implement the full $e^{-i\hat{H}t}$ when $\hat{H}$ is a sum of many such terms that don't commute with each other?

The Problem: Non-Commuting Terms

If the Hamiltonian had a single term, $\hat{H} = cP$, then:

$$e^{-i\hat{H}t} = e^{-ictP}$$

This is a single Pauli rotation — easy to implement (Chapter 14 will show exactly how). But our Hamiltonian has L terms:

$$\hat{H} = c_1 P_1 + c_2 P_2 + \cdots + c_L P_L$$

and the terms generally **do not commute**: $P_j P_k \neq P_k P_j$. This means:

$$e^{-i(c_1 P_1 + c_2 P_2)t} \neq e^{-ic_1 P_1 t} \cdot e^{-ic_2 P_2 t}$$

The exponential of a sum is *not* the product of exponentials for non-commuting operators. This is where Trotterization enters.

The Trotter Idea

The Trotter–Suzuki product formula says that for small Δt:

$$e^{-i(A+B)\Delta t} \approx e^{-iA\Delta t} \cdot e^{-iB\Delta t} + O(\Delta t^2)$$

The error is proportional to Δt^2 — so if we break the total time t into N small steps of size $\Delta t = t/N$:

$$e^{-i\hat{H}t} = \left(e^{-i\hat{H}\Delta t}\right)^N \approx \left(\prod_{k=1}^{L} e^{-ic_k P_k \Delta t}\right)^N$$

Each factor $e^{-ic_k P_k \Delta t}$ is a single Pauli rotation — implementable as a gate sequence. The full circuit is just N repetitions of L rotations.

```
flowchart LR
    subgraph Step["One Trotter Step"]
        direction LR
        R1["e^{-ic₁P₁Δt}"] --> R2["e^{-ic₂P₂Δt}"] --> R3["..."] --> RL["e^{-icₗPₗΔt}"]
    end
    Step --> |"Repeat N times"| OUT["≈ e^{-iHt}"]
```

Trade-off: More Trotter steps (N) → better approximation but deeper circuit. Fewer steps → shallower circuit but larger error. The choice of N depends on the target precision and the commutator structure of $\hat{H}$.

First vs Second Order

The formula above is **first-order Trotter**: error $O(\Delta t^2)$ per step, $O(t^2/N)$ total.

Second-order Trotter (Suzuki) cuts the error to $O(\Delta t^3)$ by symmetrizing:

$$e^{-i\hat{H}\Delta t} \approx \prod_{k=1}^{L} e^{-ic_k P_k \Delta t/2} \cdot \prod_{k=L}^{1} e^{-ic_k P_k \Delta t/2}$$

The forward pass uses half-angles, the reverse pass mirrors the sequence. The cost is $2L$ rotations per step instead of L, but the error decreases faster, so you need fewer steps for the same precision.

Order	Rotations per step	Error per step	Error for N steps
First	L	$O(\Delta t^2)$	$O(t^2/N)$
Second	$2L$	$O(\Delta t^3)$	$O(t^3/N^2)$

For most molecular simulations, second-order Trotter with a moderate N is the standard choice.

Beyond Trotterization: Qubitization

Trotterization is the workhorse of Hamiltonian simulation — simple, well-understood, and the approach we develop in this book. But it is not the only method, and intellectual honesty requires us to mention the alternative.

In 2016, Dr Guang Hao Low and Isaac Chuang introduced **qubitization** — a fundamentally different approach to Hamiltonian simulation that achieves optimal query complexity. Where Trotterization approximates e^{-iHt} as a product of easy rotations (with error that shrinks as you add more steps), qubitization encodes the Hamiltonian directly into a quantum walk operator using a technique called the **Linear Combination of Unitaries (LCU)**. The result: instead of error scaling as $O(t^2/N)$ or $O(t^3/N^2)$, qubitization achieves error that scales *linearly* in the number of queries to the Hamiltonian — provably optimal.

The catch: qubitization requires additional ancilla qubits and a more complex circuit structure (the "PREPARE" and "SELECT" oracles). It is harder to implement and harder to optimize for near-term hardware. For the molecules in this book (H_2, H_2O), Trotterization is more than adequate. For the grand-challenge molecules (FeMo-co, cytochrome P450), qubitization may be the only method that achieves chemical accuracy within a reasonable circuit depth.

We will not develop qubitization in this book — it deserves its own treatment — but we mention it here so that the reader understands where Trotterization sits in the landscape:

Method	Error scaling	Circuit structure	Best for
First-order Trotter	$O(t^2/N)$	Simple: L rotations per step	Learning, small systems
Second-order Trotter	$O(t^3/N^2)$	Symmetric: $2L$ rotations per step	Most molecular simulations
Qubitization (LCU)	$O(\log(1/\epsilon))$	Complex: ancilla + walk operator	Large systems, optimal scaling

The qubitization paper — G. H. Low and I. L. Chuang, "Hamiltonian Simulation by Qubitization," *Quantum* 3, 163 (2019); original arXiv:1610.06546 (2016) — is one of the foundational results of quantum algorithms for chemistry. It is dedicated, with gratitude, as part of the intellectual lineage that inspired this book.

What Comes Next

The Trotter decomposition converts our Hamiltonian into a list of Pauli rotations:

$$\text{Hamiltonian } \hat{H} \xrightarrow{\text{Trotter}} [e^{-i\theta_1 P_1},\ e^{-i\theta_2 P_2},\ \ldots]$$

Each rotation $e^{-i\theta P}$ must then be decomposed into elementary gates (H, CNOT, Rz). That's the **CNOT staircase** — Chapter 14. But first, Chapter 13 will show how FockMap computes the rotation list.

Key Takeaways

- Quantum computers execute gates, not Hamiltonians. The bridge is time evolution $e^{-i\hat{H}t}$.
- Non-commuting Pauli terms prevent direct exponentiation. The Trotter–Suzuki formula approximates $e^{-i(A+B)t}$ as a product of individual rotations.
- First-order Trotter: L rotations, $O(t^2/N)$ error. Second-order: $2L$ rotations, $O(t^3/N^2)$ error.
- The quality of the Trotter approximation depends on the time step size and the commutator norm $\|[P_j, P_k]\|$ — smaller commutators mean smaller errors.

Further Reading

- Trotter, H. F. "On the product of semi-groups of operators." *Proc. Am. Math. Soc.* 10, 545 (1959). The original product formula.

- Suzuki, M. "General theory of fractal path integrals with applications to many-body theories and statistical physics." *J. Math. Phys.* 32, 400 (1991). Higher-order product formulas.
- Childs, A. M. and Su, Y. "Nearly optimal lattice simulation by product formulas." *Phys. Rev. Lett.* 123, 050503 (2019). Modern error bounds for Trotter formulas.
- Low, G. H. and Chuang, I. L. "Hamiltonian Simulation by Qubitization." *Quantum* 3, 163 (2019); arXiv:1610.06546 (2016). The optimal-complexity alternative to Trotterization, encoding the Hamiltonian directly into a quantum walk operator.

Previous: Chapter 12 — Tapering Benchmarks

Next: Chapter 14 — Trotterization in Practice

Chapter 14: Trotterization in Practice

Chapter 13 explained why we need to break the time-evolution operator into small, implementable pieces. This chapter actually does it — and reveals something satisfying: the structure of the Hamiltonian we've been studying since Chapter 6 determines the structure of the circuit.

In This Chapter

- **What you'll learn:** How to apply first-order and second-order Trotter decomposition to the H_2 Hamiltonian, what the rotation list looks like, how to choose a time step, and how to estimate circuit cost before generating a single gate.
- **Why this matters:** The rotation list is the bridge between the symbolic Hamiltonian (the physicist's object) and the physical circuit (the engineer's object). This is where the two worlds meet.
- **Prerequisites:** Chapter 13 (you understand the Trotter–Suzuki formula and why it's needed).

What Happens When Theory Meets H_2

In Chapter 13, we developed the Trotter formula in general terms: break $e^{-i\hat{H}t}$ into a product of single-term rotations. The formula looked clean and abstract. Now let's apply it to the actual 15-term Hamiltonian we've been carrying since Chapter 6 — the one we built integral by integral, verified against eigenvalues, and (optionally) tapered.

What comes out the other end is a **rotation list** — an ordered sequence of Pauli rotations, each with: - a **Pauli string** (the "axis" of rotation: $XXYY$, $IIZZ$, etc.) - a **rotation angle** (the coefficient times the time step)

This list is the intermediate representation between the symbolic world (where the Hamiltonian lives) and the gate world (where the quantum computer lives). Chapter 15 will decompose each rotation into elementary gates; here, we focus on producing the list and understanding its structure.

Remember what we learned in Chapter 6: the Hamiltonian's terms split into diagonal (classical) and off-diagonal (quantum). That split carries through to the rotation list — the diagonal rotations are cheap (Z-only, no CNOTs), and the off-diagonal rotations are expensive ($XXYY$-type, 6 CNOTs each). Trotterization doesn't change this fundamental economics; it just makes it executable.

The H_2 Hamiltonian, One Last Time

Here are our 15 terms, grouped by character:

Group	Pauli Strings	Coefficients (Ha)	Weight	Character
1	IIII	-1.0704	0	Global phase (drop)
2–5	$IIIZ$, $IIZI$, $IZII$, $ZIII$	± 0.09 to ± 0.30	1	Orbital energies
6–11	$IIZZ$, $IZIZ$, $IZZI$, $ZIIZ$, $ZIZI$, $ZZII$	± 0.01 to ± 0.17	2	Coulomb
12–15	$XXYY$, $XYYX$, $YXXY$, $YYXX$	± 0.1744	4	Exchange (quantum)

The identity term (IIII) contributes a global phase $e^{-i(-1.0704)t}$ which has no observable effect — it shifts all eigenvalues equally. We drop it from the circuit, leaving **14 non-identity terms** to Trotterize.

First-order Trotter with $\Delta t = 0.1$

Each term produces one Pauli rotation. The angle is $\theta_k = c_k \times \Delta t$:

```
open Encodings

let step = firstOrderTrotter 0.1 hamiltonian

printfn "Rotations: %d" step.Rotations.Length
for r in step.Rotations do
    printfn "  angle=%+.6f  Pauli=%s  weight=%d"
        r.Angle
        r.Operator.Signature
        (r.Operator.Signature |> Seq.filter (fun c -> c <> 'I') |> Seq.length)
```

The output will show 14 rotations — one per non-identity Hamiltonian term. The angles are small (all less than 0.04 in magnitude) because we chose $\Delta t = 0.1$ and the

coefficients are all less than 0.35 Ha.

What the rotation list tells us

Each entry says: "rotate the quantum state by angle θ around the axis defined by Pauli string P." In physical terms, this evolves the state under the influence of one term of the Hamiltonian for a short time.

The key observation from Chapter 6 carries through: the **diagonal rotations** (weights 1–2, Z-only terms) are cheap and classical. The **off-diagonal rotations** (weight 4, XXYY-type) are expensive and quantum. The Trotter decomposition separates them — each gets its own rotation — and the encoding determines how expensive each one is.

Second-Order Trotter: Symmetry for Accuracy

First-order Trotter applies the rotations in one direction:

$$e^{-ic_1 P_1 \Delta t} \cdot e^{-ic_2 P_2 \Delta t} \cdot \ldots \cdot e^{-ic_L P_L \Delta t}$$

Second-order Trotter applies them forward at half-angle, then backward at half-angle:

$$\underbrace{e^{-ic_1 P_1 \Delta t/2} \ldots e^{-ic_L P_L \Delta t/2}}_{\text{forward, half angle}} \cdot \underbrace{e^{-ic_L P_L \Delta t/2} \ldots e^{-ic_1 P_1 \Delta t/2}}_{\text{reverse, half angle}}$$

```
let step2 = secondOrderTrotter 0.1 hamiltonian

printfn "Rotations: %d (vs %d for first-order)"
    step2.Rotations.Length
    step.Rotations.Length
// → 28 rotations (2 × 14)
```

The symmetrization cancels the leading-order error terms. Intuitively: the forward pass over-approximates by a small amount, and the reverse pass under-approximates by the same amount. The errors cancel, leaving a much smaller residual.

Property	First-order	Second-order
Rotations per step	L (14 for H_2)	$2L$ (28 for H_2)
Rotation angles	$c_k \Delta t$	$c_k \Delta t/2$
Error per step	$O(\Delta t^2)$	$O(\Delta t^3)$
Total error for N steps	$O(t^2/N)$	$O(t^3/N^2)$

For the same target accuracy, second-order Trotter typically needs $\sqrt{N}$ fewer steps than first-order — which more than compensates for the doubled rotation count per step.

Choosing the Time Step

The time step Δt is the key knob in Trotterization. Too large → bad approximation. Too small → too many steps → too deep a circuit.

A practical rule of thumb: $\Delta t \leq 1/\|\hat{H}\|_1$, where the 1-norm is $\|\hat{H}\|_1 = \sum_k |c_k|$.

For H_2: $\|\hat{H}\|_1 \approx 3.7$ Ha, giving $\Delta t \lesssim 0.27$. Our choice of $\Delta t = 0.1$ is comfortably within this bound.

For H_2O (12 qubits): $\|\hat{H}\|_1$ is larger (~30 Ha), so Δt must be smaller — around 0.03. This means more Trotter steps per unit time, which means more CNOTs. This is another reason larger molecules are harder to simulate.

Quick CNOT Estimate (Before Gate Decomposition)

We can estimate the CNOT cost without actually building the gate circuit, using the formula from Chapter 4:

$$\text{CNOTs per Trotter step} = \sum_{k=1}^{L} 2(w_k - 1)$$

```
let cnots = trotterCnotCount step
printfn "Estimated CNOTs per first-order step: %d" cnots
```

For H_2 (14 non-identity terms):

Term type	Count	Weight	CNOTs each	Subtotal
Single-Z ($IIIZ$, etc.)	4	1	0	0
Double-Z ($IIZZ$, etc.)	6	2	2	12
Exchange ($XXYY$, etc.)	4	4	6	24
Total	**14**	—	—	**36**

36 CNOTs per first-order Trotter step. Second-order: 72. For a 100-step simulation: 3,600 (first-order) or 7,200 (second-order) CNOTs total.

These are small numbers — easily within reach of current hardware for H_2. For H_2O with ~600 terms and higher weights, the numbers grow into the thousands per step — which is where encoding choice and tapering make the difference between feasible and infeasible.

How Many Trotter Steps Do I Need?

The Trotter approximation introduces error. How much error, and how many steps are needed to keep it below a target ϵ?

The error bound

For first-order Trotter with N steps of size $\Delta t = t/N$, the total error is bounded by:

$$\epsilon_{\text{Trotter}} \leq \frac{t^2}{2N} \sum_{j<k} \|[c_j P_j,\ c_k P_k]\|$$

The commutator sum $\Lambda = \sum_{j<k} \|[c_j P_j, c_k P_k]\|$ measures how "badly" the terms fail to commute. If all terms commuted, $\Lambda = 0$ and there would be no Trotter error at all — the product formula would be exact.

For the second-order formula, the error bound improves to:

$$\epsilon_{\text{Trotter}} \leq \frac{t^3}{12N^2} \sum_{j,k,l} \|[[c_j P_j, c_k P_k], c_l P_l]\|$$

In practice, practitioners use a simpler (looser) bound: the 1-norm $\|H\|_1 = \sum_k |c_k|$ provides a conservative estimate:

$$N \geq \frac{t^2 \|H\|_1^2}{2\epsilon} \quad \text{(first-order)} \qquad N \geq t\sqrt{\frac{t\|H\|_1^3}{12\epsilon}} \quad \text{(second-order)}$$

Worked example: H_2 at chemical accuracy

For H_2: $\|H\|_1 \approx 3.7$ Ha. Target precision: $\epsilon = 1.6 \times 10^{-3}$ Ha (chemical accuracy). Total evolution time: $t = 1$ (one unit of atomic time).

First-order:

$$N \geq \frac{(1)^2 \times (3.7)^2}{2 \times 0.0016} = \frac{13.69}{0.0032} \approx 4{,}278 \text{ steps}$$

At 36 CNOTs per step: ~154,000 total CNOTs. That's conservative — the actual commutator-based bound is much tighter because many H_2 terms commute.

Second-order:

$$N \geq 1 \times \sqrt{\frac{1 \times (3.7)^3}{12 \times 0.0016}} = \sqrt{\frac{50.65}{0.0192}} \approx \sqrt{2{,}638} \approx 52 \text{ steps}$$

At 72 CNOTs per step: ~3,700 total CNOTs. This is why second-order Trotter is the standard choice — the step count drops by nearly 100×.

The practical message

For H_2, even the conservative estimate says ~52 second-order steps suffice for chemical accuracy — about 3,700 CNOTs. For H_2O ($\|H\|_1 \approx 30$ Ha), the step count grows and the per-step cost is higher, pushing the total to ~500,000 CNOTs. These numbers match the resource estimates in Chapter 19.

The tighter commutator-based bounds (Childs and Su, PRL 2019) typically improve on the 1-norm estimate by a factor of 5–50×, because real molecular Hamiltonians have significant commuting structure. But the 1-norm bound is safe, easy to compute, and gives the right order of magnitude.

Key Takeaways

- A Trotter step converts a Hamiltonian into a list of **Pauli rotations** — each with a Pauli string and an angle.
- First-order: L rotations. Second-order: $2L$ rotations at half angle, with quadratically better error scaling.
- The time step Δt is bounded by $1/\|H\|_1$ — larger Hamiltonians need smaller steps.
- CNOT cost is estimable from Pauli weights alone: $\sum_k 2(w_k - 1)$.
- For H_2: 36 CNOTs per first-order step. Feasible. For H_2O: thousands. That's where optimization matters.

Common Mistakes

1. **Including the identity term.** The IIII term contributes only a global phase — include it and you waste one rotation per step on something unobservable.
2. **Using first-order Trotter for production.** First-order is fine for learning but second-order is almost always better in practice — the error improvement outweighs the doubled rotation count.
3. **Choosing Δt too large.** If $\Delta t \cdot \|H\|_1 > 1$, the Trotter approximation breaks down and the circuit does not approximate the correct time evolution.

Exercises

1. **Rotation count.** For H_2O with 600 non-identity terms, how many rotations does a single second-order Trotter step have?
2. **Time step.** If $\|H\|_1 = 30$ Ha and you want $\Delta t = 0.01$, how many Trotter steps do you need for total evolution time $t = 1$?

3. **CNOT scaling.** If every term in a Hamiltonian has weight 5 (ternary tree on a 27-orbital system), and there are 200 non-identity terms, what is the CNOT count per first-order step?

Previous: Chapter 13 — From Hamiltonian to Time Evolution

Next: Chapter 15 — The CNOT Staircase

Chapter 15: The CNOT Staircase

Every Pauli rotation becomes a sequence of elementary gates. This chapter shows exactly how — and connects the dots from encoding choice to circuit cost.

In This Chapter

- **What you'll learn:** How $e^{-i\theta P}$ decomposes into basis-change gates, a CNOT chain, and an R_z rotation. Why the cost is exactly $2(w-1)$ CNOTs. How to trace a complete decomposition for the $XXYY$ exchange term.
- **Why this matters:** This is where everything converges. Encoding choice (Chapter 7), tapering (Chapters 9–12), and Trotterization (Chapters 13–14) all reduce to one question: how many times do we run this staircase, and how tall is it?
- **Prerequisites:** Chapters 4 (gates), 13–14 (Trotter decomposition).

The Big Picture

Recall from Chapter 4 the CNOT staircase formula: a Pauli rotation with weight w costs $2(w-1)$ CNOT gates. We stated this without proof. Now let's see *why* — by building the decomposition step by step.

The intuition: a weight-w Pauli rotation $e^{-i\theta P}$ applies a phase rotation that depends on the *collective parity* of w qubits. No single qubit carries enough information — we need to temporarily entangle the w qubits (using CNOTs) so that their joint parity lives on one qubit, apply the rotation there, and then disentangle. The "staircase" is the entangling chain.

The Decomposition Recipe

A Pauli rotation $e^{-i\theta P}$ for a weight-w Pauli string P is implemented in three phases:

```
flowchart LR
    BC["1. Basis change<br/>(single-qubit gates)"]
    CS["2. CNOT staircase<br/>(entangling)"]
```

```
    RZ["3. Rz(2θ)<br/>(rotation)"]
    UCS["4. Undo staircase"]
    UBC["5. Undo basis change"]
    BC --> CS --> RZ --> UCS --> UBC
```

Phase 1: Basis change

For each qubit position in P: - If $P_j = X$: apply Hadamard (H) to convert X-basis to Z-basis - If $P_j = Y$: apply $R_x(\pi/2)$ to convert Y-basis to Z-basis - If $P_j = Z$ or I: no gate needed

After this step, all non-identity Pauli operators have been converted to Z.

Phase 2: CNOT staircase

Chain CNOTs between the w non-identity qubits:

```
flowchart LR
    Q0["q₀"] --> |CNOT| Q1["q₁"]
    Q1 --> |CNOT| Q2["q₂"]
    Q2 --> |CNOT| Qw["q_{w-1}"]
```

This creates a parity computation: qubit q_{w-1} now holds the XOR of all w qubits' Z-values.

Phase 3: Rz rotation

Apply $R_z(2\theta)$ to the last qubit in the chain. This imprints the rotation angle, conditioned on the collective parity.

Phases 4–5: Uncompute

Reverse the CNOT staircase (same gates, reversed order), then reverse the basis-change gates. The total circuit is self-inverse up to the rotation.

Gate Counts

Component	Gates
Basis change	$\leq w$ single-qubit gates
Forward CNOT staircase	$w - 1$ CNOTs
R_z rotation	1 single-qubit gate
Reverse CNOT staircase	$w - 1$ CNOTs
Undo basis change	$\leq w$ single-qubit gates
Total CNOTs	$2(w - 1)$
Total single-qubit	$\leq 2w + 1$

Examples

Pauli string	Weight w	CNOTs	Single-qubit
Z (single qubit)	1	0	1 (R_z only)
ZZ	2	2	1
$XXYY$	4	6	9
$ZZZZZZZZZ$ (weight 9)	9	16	1

Key insight: CNOTs scale linearly with weight. This is why encoding choice matters — reducing the maximum Pauli weight from 100 (JW) to 5 (ternary tree) saves 190 CNOTs *per rotation*.

Worked Example: XXYY Rotation

Decompose $e^{-i\theta X_0 X_1 Y_2 Y_3}$:

Basis change: - Qubit 0: $X \to Z$ via Hadamard - Qubit 1: $X \to Z$ via Hadamard - Qubit 2: $Y \to Z$ via $R_x(\pi/2)$ - Qubit 3: $Y \to Z$ via $R_x(\pi/2)$

After basis change: the operation is now $e^{-i\theta Z_0 Z_1 Z_2 Z_3}$

CNOT staircase: CNOT(0→1), CNOT(1→2), CNOT(2→3)

Rz: $R_z(2\theta)$ on qubit 3

Reverse CNOT staircase: CNOT(2→3), CNOT(1→2), CNOT(0→1)

Undo basis change: $R_x(-\pi/2)$ on qubits 2,3; Hadamard on qubits 0,1

Total: 6 CNOTs + 9 single-qubit gates.

Why This Makes Encoding Choice Concrete

The CNOT staircase turns the abstract concept of "Pauli weight" into a concrete gate count:

$$\text{CNOTs per Trotter step} = \sum_{k=1}^{L} 2(w_k - 1)$$

For the same molecule encoded differently:

Encoding	Max w_k	Typical CNOT cost/step
Jordan–Wigner ($n = 32$)	32	~2000

Encoding	Max w_k	Typical CNOT cost/step
Bravyi–Kitaev ($n = 32$)	6	~500
Ternary Tree ($n = 32$)	5	~400
Tapered + TT ($n = 29$)	5	~340

The 6× ratio between JW and TT at $n = 32$ is entirely due to the CNOT staircase — the same formula, the same chemistry, but dramatically different circuit depth.

Key Takeaways

- Each Pauli rotation decomposes into basis change → CNOT staircase → R_z → reverse.
- The CNOT count is exactly $2(w - 1)$ per rotation, where w is the Pauli weight.
- Total CNOTs per Trotter step: $\sum_k 2(w_k - 1)$ — the single most important number for circuit feasibility.
- Encoding choice and tapering both reduce w_k and therefore CNOT cost.

Previous: Chapter 14 — Trotterization in Practice

Next: Chapter 16 — Cost Analysis Across Encodings

Chapter 16: Cost Analysis Across Encodings

Everything we've done — encoding, tapering, Trotterization — converges to one number: the CNOT count. This chapter computes it.

In This Chapter

- **What you'll learn:** The complete CNOT cost for H_2 and H_2O across all five encodings, with and without tapering, and how the optimization stack compounds.
- **Why this matters:** This answers the practical question: "which encoding should I use for my molecule?" The answer depends on the system size, and the numbers tell the story.
- **Prerequisites:** Chapters 13–15 (Trotter decomposition and CNOT staircase).

The Optimization Stack

Before we compute anything, here is the complete pipeline showing every optimization we've developed:

```
flowchart TD
    RAW["Raw JW Hamiltonian<br/>n qubits, O(n) weight"]
    RAW --> |"Tapering<br/>(Ch 9–12)"| TAP["Tapered Hamiltonian<br/>n-k qubits"]
    TAP --> |"Encoding choice<br/>(Ch 7)"| ENC["Ternary tree encoding<br/>O(log₃ n) wei
    ENC --> |"2nd-order Trotter<br/>(Ch 13–14)"| TROT["Gate sequence<br/>2L rotations ×
    style TROT fill:#d1fae5,stroke:#059669
```

Each stage contributes multiplicatively: - **Tapering** reduces qubit count by k, often reduces term count - **Encoding choice** reduces worst-case weight from $O(n)$ to $O(\log n)$ - **Trotter order** trades rotations per step for error convergence rate

The order matters: **taper first** (on any encoding), then choose the encoding for the tapered system, then Trotterize.

The Full Cost Formula

The total CNOT cost for one Trotter step is:

$$C_{\text{CNOT}} = \sum_{k=1}^{L} 2(w_k - 1)$$

where L is the number of non-identity terms and w_k is the Pauli weight of term k. For second-order Trotter, multiply by 2 (each term appears twice).

H_2 (4 qubits, 15 terms)

Encoding	Max weight	Avg weight	CNOTs/step (1st order)	CNOTs/step (2nd order)
Jordan–Wigner	4	2.1	36	72
Bravyi–Kitaev	4	2.4	40	80
Parity	4	2.3	38	76
Balanced Binary	4	2.3	38	76
Balanced Ternary	4	2.4	40	80

Observation: At 4 qubits, JW is actually the *cheapest*. The tree encodings have slightly higher average weight because the tree structure on 4 nodes doesn't provide enough room for the logarithmic advantage to manifest. This matches our observation from Chapter 6.

H_2O (12 qubits, frozen core, STO-3G)

This is where the differences become real. H_2O has 7 spatial orbitals (with frozen core), giving 14 spin-orbitals — but we freeze 2 core electrons, leaving 12 active spin-orbitals = 12 qubits.

Configuration	Qubits	Terms	Max weight	CNOTs/step
JW, no tapering	12	~630	12	~1800
BK, no tapering	12	~630	5	~750
TT, no tapering	12	~630	4	~600

Configuration	Qubits	Terms	Max weight	CNOTs/step
JW, tapered	9	~420	9	~900
TT, tapered	9	~420	4	~500

Key takeaway: The ternary tree encoding with tapering gives ~3.6× fewer CNOTs than JW without tapering. For a 100-step Trotter simulation, that's 130,000 fewer CNOTs — potentially the difference between a feasible and an infeasible experiment on near-term hardware.

Scaling to Larger Molecules

Molecule	Spin-orbitals	JW CNOTs/step	TT CNOTs/step	Ratio
H_2	4	36	40	0.9×
LiH	12	~1200	~500	2.4×
H_2O	14	~1800	~600	3×
N_2	20	~5000	~1200	4.2×
FeMo-co	~100	~500,000	~20,000	~25×

The ratio grows monotonically because JW's worst-case weight scales as $O(n)$ while TT's scales as $O(\log_3 n)$. At FeMo-co scale (~100 orbitals), the 25× CNOT reduction from encoding choice alone is transformative.

Key Takeaways

- For small molecules ($n \leq 6$), encoding choice barely matters. JW is often cheapest.
- For medium molecules ($n = 12$–20), ternary tree encoding saves 3–5× CNOTs over JW.
- For large molecules ($n \sim 100$), the savings are ~25× — potentially enabling experiments that would otherwise be infeasible.
- The complete optimization stack is: **taper → encode → Trotterize**. Each stage compounds multiplicatively.
- The CNOT count is the single most important feasibility metric for near-term quantum simulation.
- **Measurement cost matters too**: on near-term hardware running VQE, the total experimental cost includes both circuit depth (CNOT count per shot) and shot count (repetitions needed for statistical precision). The shot count scales as $N \sim (\sum_k |c_k|)^2/\epsilon^2$, where the 1-norm $\sum |c_k|$ depends on encoding and tapering. Tapering reduces the 1-norm, so it saves both gate cost *and* measurement cost. Chapter 19 covers this in detail.

Common Mistakes

1. **Encoding before tapering.** Taper first — the encoding then operates on a smaller qubit count, which can change which encoding is optimal.
2. **Comparing encodings at $n = 4$.** The differences are negligible at small n. Scale to $n \geq 12$ to see real savings.
3. **Ignoring the time step.** CNOT count per step is only half the story — the number of Trotter steps (determined by Δt and $\|H\|_1$) multiplies everything.

Exercises

1. **H_2 by hand.** Verify the 36-CNOT figure for H_2 first-order Trotter by summing $2(w_k - 1)$ over all 14 non-identity terms.
2. **Tapering impact.** If tapering removes 3 qubits from a 14-qubit system and reduces the term count from 630 to 420, estimate the CNOT savings assuming average weight drops from 4 to 3.
3. **Run it.** Use the companion script `book/code/ch07-five-encodings.fsx` to compute the weight scaling table for $n = 4, 8, 16, 32, 64$. At what n does the ternary tree first beat JW?

Previous: Chapter 15 — The CNOT Staircase

Next: Chapter 17 — The Complete Pipeline

Chapter 17: The Question We Can Now Answer

Sixteen chapters of theory and code. Time to run the whole thing.

In This Chapter

- **What you'll learn:** Every stage of the pipeline assembled into a single executable script — from molecular integrals to quantum circuit — for the hydrogen molecule.
- **Why this matters:** This is the capstone. Every concept from the preceding chapters appears here in its final, integrated form. If you can follow this script line by line and understand what each call does, you have mastered the pipeline.
- **Prerequisites:** All of Stages 1–5 (Chapters 1–16).

The Payoff

Let's take stock of what we've built.

In Chapter 1, we started with a molecule and asked: *what is its ground-state energy?* That question led us through electronic structure (the integrals), second quantization (the ladder operators), encoding (the Pauli strings), tapering (removing redundant qubits), Trotterization (turning operators into gates), and cost analysis (counting the CNOTs). At each stage we introduced one mathematical transformation, implemented it, and tested it on H_2.

Now we put the whole chain together. The script below takes the H_2 molecular integrals and produces a quantum circuit — ready to run on a simulator or real hardware. Five function calls. Under fifty lines of code. Under a second of runtime.

> **A note on energy evaluation:** In this chapter and the next, we use **exact diagonalisation** (classical matrix eigensolving) as the energy-evaluation backend. For H_2 (4 qubits, $2^4 = 16$-dimensional matrix) and H_2O in minimal basis (~11 qubits after tapering, $2^{11} = 2{,}048$-dimensional matrix), this is trivially feasible on a laptop. The pipeline constructs the *circuit*; exact diagonalisation is a stand-in for the quantum measurement step. Chapter

19 explains the quantum algorithms (VQE, QPE) that replace exact diagonalisation when the system is too large for classical solution.

The Script

```
open System.Numerics
open Encodings

// ═══════════════════════════════════════════════════════════════════
//  The integrals (from Chapter 3)
// ═══════════════════════════════════════════════════════════════════

let Vnn = 0.7151043391  // Nuclear repulsion energy (Hartrees)

let integrals = Map [
    // One-body: kinetic + electron-nuclear attraction
    ("0,0", Complex(-1.2563390730, 0.0))
    ("1,1", Complex(-1.2563390730, 0.0))
    ("2,2", Complex(-0.4718960244, 0.0))
    ("3,3", Complex(-0.4718960244, 0.0))
    // Two-body: electron-electron repulsion
    ("0,0,0,0", Complex(0.6744887663, 0.0))
    ("1,1,1,1", Complex(0.6744887663, 0.0))
    ("2,2,2,2", Complex(0.6973979495, 0.0))
    ("3,3,3,3", Complex(0.6973979495, 0.0))
    ("0,0,2,2", Complex(0.6636340479, 0.0))
    ("2,2,0,0", Complex(0.6636340479, 0.0))
    ("1,1,3,3", Complex(0.6636340479, 0.0))
    ("3,3,1,1", Complex(0.6636340479, 0.0))
    ("0,2,2,0", Complex(0.1809312700, 0.0))
    ("2,0,0,2", Complex(0.1809312700, 0.0))
    ("1,3,3,1", Complex(0.1809312700, 0.0))
    ("3,1,1,3", Complex(0.1809312700, 0.0))
]

let factory key = integrals |> Map.tryFind key

// ═══════════════════════════════════════════════════════════════════
//  All five encodings, one loop
// ═══════════════════════════════════════════════════════════════════

let encoders = [
    ("Jordan-Wigner",  jordanWignerTerms)
    ("Bravyi-Kitaev",  bravyiKitaevTerms)
    ("Parity",         parityTerms)
```

```
    ("Binary Tree",    balancedBinaryTreeTerms)
    ("Ternary Tree",   ternaryTreeTerms)
]

for (name, encoder) in encoders do
    // Stage 1: Build the qubit Hamiltonian
    let ham = computeHamiltonianWith encoder factory 4u

    // Stage 2: Taper — remove symmetry-redundant qubits
    let tapResult = taper defaultTaperingOptions ham

    // Stage 3: Trotterize — one first-order step at Δt = 0.1
    let step = firstOrderTrotter 0.1 tapResult.Hamiltonian
    let stats = trotterStepStats step

    // Stage 4: Export to OpenQASM
    let gates = decomposeTrotterStep step
    let qasm = toOpenQasm defaultOpenQasmOptions tapResult.TaperedQubitCount gates

    printfn "%-15s  %d → %d qubits  %d terms  %d CNOTs  %d total gates"
        name
        tapResult.OriginalQubitCount
        tapResult.TaperedQubitCount
        (tapResult.Hamiltonian.DistributeCoefficient.SummandTerms.Length)
        stats.CnotCount
        stats.TotalGates
```

That's it. One loop, five encodings, each going from integrals all the way to a quantum circuit.

What Each Line Does

If you've read the preceding chapters, every line should be familiar. But let's trace through one iteration — Jordan–Wigner — to see the full pipeline in action:

`computeHamiltonianWith encoder factory 4u` — Takes the encoder function and the integral lookup, builds a 4-qubit Pauli Hamiltonian. This is Chapter 6: the one-body and two-body integrals are paired with encoded ladder operators and summed into a `PauliRegisterSequence`. For JW, we get the 15-term Hamiltonian we first met in Chapter 3.

`taper defaultTaperingOptions ham` — Finds Z_2 symmetries (Chapter 10), synthesizes the Clifford rotation (Chapter 11), applies it, and removes the redundant qubits. The default options use the positive sector and Clifford-based tapering. For JW on H_2, this typically removes 2 qubits, leaving a 2-qubit Hamiltonian.

`firstOrderTrotter 0.1 tapResult.Hamiltonian` — Decomposes the tapered Hamil-

tonian into a sequence of Pauli rotations (Chapter 13). Each term $c_k P_k$ becomes a rotation $e^{-ic_k \Delta t P_k/2}$. The time step $\Delta t = 0.1$ controls the Trotter error.

`decomposeTrotterStep step` — Converts each Pauli rotation into concrete gates via the CNOT staircase (Chapter 15). A weight-w rotation becomes $2(w-1)$ CNOTs plus single-qubit gates.

`toOpenQasm defaultOpenQasmOptions numQubits gates` — Writes the gate sequence as a valid OpenQASM 3.0 program (Chapter 20 will explore this in detail).

`trotterStepStats step` — Counts everything: rotations, CNOTs, single-qubit gates, total gates (Chapter 16).

The Numbers

Run the script and you get a table like this:

Encoding	Qubits	Tapered	Terms	CNOTs/step	Total gates
Jordan–Wigner	4	2	5	12	52
Bravyi–Kitaev	4	2	5	12	52
Parity	4	2	5	12	52
Binary Tree	4	2	5	12	52
Ternary Tree	4	2	5	12	52

Wait — they're all the same?

Yes. At 4 qubits, every encoding produces the same cost after tapering. The Pauli strings are different, but the weights are the same. H_2 was always our *teacher*, not our *benchmark*. The differences appear at scale — we'll see them with H_2O in the next chapter.

But notice what we've achieved: **five independent encoding pipelines all converge to the same physical answer.** That's the verification from Chapter 8, the eigenvalue agreement from Chapter 7, now confirmed at the circuit level. The mathematics works.

A Closer Look at the Output

Let's examine what the OpenQASM looks like for one encoding. Taking the Jordan–Wigner tapered Hamiltonian:

```
let jwHam = computeHamiltonianWith jordanWignerTerms factory 4u
let jwTapered = taper defaultTaperingOptions jwHam
let jwStep = firstOrderTrotter 0.1 jwTapered.Hamiltonian
```

```
let jwGates = decomposeTrotterStep jwStep
let qasm = toOpenQasm defaultOpenQasmOptions jwTapered.TaperedQubitCount jwGates

printfn "%s" qasm
```

The output is a valid QASM 3.0 program — qubit declarations, followed by a sequence of `h`, `cx`, `rz`, `s`, and `sdg` instructions. You could paste this into IBM Quantum, run it on a simulator, measure the energy, and compare to −1.8572 Hartrees. Chapter 19 will explore the algorithms that do exactly that.

Extra Credit: The H_2 Dissociation Curve

The pipeline runs at one geometry — but there's nothing stopping us from running it at *many* geometries. If we vary the bond length R and plot the energy, we get the **dissociation curve**: the potential energy surface of H_2 as the two atoms are pulled apart.

This is a classic test of quantum chemistry methods. Hartree–Fock breaks down as the bond stretches (it can't describe the transition from a bonding pair to two independent atoms). Full configuration interaction (FCI) — which is what exact diagonalisation of our encoded Hamiltonian gives — handles the dissociation correctly.

The Skeleton API

At every bond length, the Pauli string *structure* is the same — same 4-qubit encoding, same term signatures. Only the integral *values* change. FockMap's skeleton API exploits this:

```
// Precompute the Pauli structure once
let skeleton = computeHamiltonianSkeleton jordanWignerTerms 4u

// Then, for each bond length, just plug in the integrals
let energyAt (integrals : Map<string, Complex>) =
    let factory key = integrals |> Map.tryFind key
    let ham = applyCoefficients skeleton factory
    let tapered = taper defaultTaperingOptions ham
    // Exact diagonalisation for the ground-state energy
    exactGroundStateEnergy tapered.Hamiltonian
```

`computeHamiltonianSkeleton` does the expensive encoding work once. `applyCoefficients` is cheap — it just multiplies precomputed Pauli strings by numbers. For a 20-point scan, this is 20× faster than rebuilding the Hamiltonian from scratch each time.

The Scan

Generate the integrals at each bond length using PySCF (or use the values from a precomputed table), then loop:

```
let bondLengths = [| 0.4; 0.5; 0.6; 0.7; 0.74; 0.8; 0.9; 1.0;
                     1.2; 1.4; 1.6; 1.8; 2.0; 2.5; 3.0; 4.0 |]

for R in bondLengths do
    let integrals = loadH2Integrals R   // from precomputed JSON (see below)
    let Vnn = 1.0 / (R * 1.8897259886)  // nuclear repulsion: 1/R in atomic units
    let Etotal = energyAt integrals + Vnn
    printfn "R = %.2f Å   E = %.6f Ha" R Etotal
```

> **Where do `loadH2Integrals` and `exactGroundStateEnergy` come from?** They are helper functions defined in the companion script `book/code/ch17-dissociation-scan.py` (which generates the integral JSON files) and `book/code/ch17-pipeline.fsx` (which reads them). `loadH2Integrals R` reads a JSON file mapping integral keys like `"0,1"` to complex values — the same `Map<string, Complex>` format our `factory` function expects. `exactGroundStateEnergy` constructs the $2^n \times 2^n$ Hamiltonian matrix from the Pauli terms and returns its smallest eigenvalue via standard linear algebra. Both are thin wrappers — the real work is done by the FockMap API calls shown in the text.

The Result

The output is a table of bond lengths and exact FCI energies — a slice through the potential energy surface:

R (Å)	E_{FCI} (Ha)	R (Å)	E_{FCI} (Ha)
0.30	−0.601804	1.20	−1.056741
0.40	−0.914150	1.40	−1.015468
0.50	−1.055160	1.60	−0.983473
0.60	−1.116286	1.80	−0.961817
0.70	−1.136189	2.00	−0.948641
0.74	**−1.137284**	2.50	−0.936055
0.80	−1.134148	3.00	−0.933632
0.90	−1.120560	4.00	−0.933171
1.00	−1.101150	5.00	−0.933164

The energy drops steeply as the atoms approach from infinity, reaches a minimum at **0.74 Å**, then rises sharply as nuclear repulsion takes over at close range. This is the classic Morse-like dissociation curve. At large separations the energy converges to **−0.933 Ha** — two isolated hydrogen atoms, each at −0.4666 Ha in the STO-3G basis.

Notice how Hartree–Fock and FCI agree near equilibrium but diverge at large R: HF wrongly forces the two electrons to stay paired (giving −0.599 Ha at $R = 5$ Å, far too

high), while FCI correctly dissociates into two independent atoms. This is the **static correlation** problem that motivated quantum simulation in the first place.

The companion script `book/code/ch17-dissociation-scan.py` generates the data and a publication-quality plot saved to `book/code/h2_dissociation.png`.

This is the same kind of scan we'll do for H_2O's bond angle in Chapter 18 — but the structural parameter will be an angle instead of a distance. The machinery is identical: skeleton precomputation, integral swap, energy evaluation.

What the Curve Tells You

The dissociation curve encodes several physical observables:

- **Equilibrium bond length** (R_e): the location of the minimum
- **Dissociation energy** (D_e): the depth of the well (minimum energy minus the asymptotic value)
- **Vibrational frequency** (ω_e): proportional to the square root of the curvature at the minimum

All three are experimentally measurable. All three emerge from our pipeline without empirical input. The R_e we compute (0.74 Å in STO-3G) matches the experimental value of 0.741 Å. The dissociation energy D_e = 1.137 − 0.933 = 0.204 Ha ≈ 5.6 eV, compared to the experimental 4.75 eV — STO-3G overbinds slightly, but the shape is correct. Our minimal basis set happens to be accurate for the equilibrium geometry because H_2 has no lone pairs or angular structure to distort.

From One Molecule to Any Molecule

The dissociation curve demonstrates the pipeline's real power: it's not a one-shot computation but a *reusable tool*. To scan a different molecule, change the integrals and qubit count. The skeleton API ensures the encoding work is done once, no matter how many geometries you scan.

In the next chapter, we'll do exactly that — but instead of scanning a bond length, we'll scan a bond angle, and the molecule will be water.

Key Takeaways

- The entire pipeline from integrals to quantum circuit is **five function calls**: `computeHamiltonianWith`, `taper`, `firstOrderTrotter`, `decomposeTrotterStep`, `toOpenQasm`.
- All five encodings produce the **same physics** (same eigenvalues, same energy) — confirmed at every stage from Hamiltonian to circuit.
- The **skeleton API** separates Pauli structure (computed once) from integral values (applied per geometry), making potential energy surface scans efficient.

- The H_2 dissociation curve — equilibrium bond length, dissociation energy, vibrational frequency — emerges from the pipeline without empirical input.
- The pipeline is **composable**: swap the integrals and qubit count to compute any molecule.

Previous: Chapter 16 — Cost Analysis

Next: Chapter 18 — The Water Bond Angle

Chapter 18: The Water Bond Angle

We built a pipeline. Now we use it to answer a question that every chemistry student learns but nobody derives from scratch: why does water bend at 104.5°?

In This Chapter

- **What you'll learn:** How to scan a potential energy surface by running the same pipeline at many molecular geometries — and how the energy minimum tells you the equilibrium bond angle.
- **Why this matters:** This is the payoff of the entire book. We built sixteen chapters of machinery. This chapter uses that machinery to compute a molecular property — the H–O–H bond angle — from first principles. No fitting. No empirical parameters. Just quantum mechanics and our pipeline.
- **Prerequisites:** Chapter 17 (the complete H_2 pipeline).

The Question

Every introductory chemistry textbook states the facts: water has a bond angle of 104.52°. The oxygen atom sits at the vertex, the two hydrogen atoms at the arms. VSEPR theory offers a qualitative explanation — two bonding pairs and two lone pairs around oxygen arrange themselves to minimise repulsion — but it doesn't predict the actual number. It says "less than 109.5°" and leaves it at that.

Quantum mechanics does better. The bond angle is the geometry that minimises the total electronic energy. If we can compute the energy as a function of angle, the minimum of that curve *is* the bond angle. No hand-waving about lone pair repulsion. No empirical fitting. Just energy versus geometry.

That computation is exactly what our pipeline was built for.

The Strategy: A Potential Energy Surface Scan

The approach is simple:

1. **Pick a range of angles.** We'll scan from 60° to 180° in 5° steps (25 geometries).
2. **At each angle, compute the molecular integrals.** PySCF generates the one-body and two-body integrals for that geometry.
3. **At each angle, build the encoded Hamiltonian.** Same pipeline as Chapter 17, but with different integrals each time.
4. **At each angle, compute the ground-state energy.** For now, we diagonalise the Hamiltonian matrix directly (exact diagonalisation). On a quantum computer, this would be a VQE or QPE run — Chapter 19 covers those algorithms.
5. **Find the angle with the lowest energy.** That's the bond angle.

Step 3 is where the pipeline shines — and where the skeleton API eliminates redundant work.

The Skeleton Trick

At every bond angle, we call `computeHamiltonianWith` with different integrals. But the *structure* of the Hamiltonian — which Pauli strings appear, how ladder operators map to qubits — depends only on the encoding and the number of qubits. It doesn't change when the molecule bends.

FockMap exploits this with the **skeleton** pattern:

```
// Precompute the structure once
let skeleton = computeHamiltonianSkeleton ternaryTreeTerms 14u

// Then, for each geometry, just plug in the integrals
let hamAtAngle angle =
    let factory = integralsAt angle
    applyCoefficients skeleton factory
```

`computeHamiltonianSkeleton` does the expensive work — encoding every ladder operator pair, combining like terms, building the Pauli string structure — exactly once. `applyCoefficients` then multiplies each precomputed Pauli string by the appropriate integral value. For a 25-point scan, this is 25× faster than calling `computeHamiltonianWith` 25 times, because the encoding step (which involves Pauli string multiplication and like-term combination) dominates the cost.

This is the same separation of concerns that makes compiled languages fast: separate the work that depends on program structure (compilation) from the work that depends on input data (execution).

The Integrals

For H_2 we had 4 spin-orbitals and 16 integrals. For H_2O in a minimal basis (STO-3G), we have 14 spin-orbitals and hundreds of integrals — too many to type by hand. In practice, you generate them with PySCF:

> **Active space and frozen core:** H_2O has 10 electrons, but the oxygen 1s core electrons are tightly bound and contribute negligibly to chemical bonding. In a minimal basis (STO-3G, 7 spatial orbitals → 14 spin-orbitals), all electrons are included because the basis is already small. In larger basis sets, practitioners typically "freeze" the core electrons — removing them from the active space and treating their contribution as a constant energy offset. This reduces the number of spin-orbitals (and therefore qubits) without significantly affecting the bond angle or other valence properties. The integrals generated below include all electrons because STO-3G is minimal; for larger-basis production calculations, the PySCF `mc.CASCI` or `mc.CASSCF` interface would be used to define the active space explicitly.

```
from pyscf import gto, scf
import numpy as np
import json

def h2o_integrals(angle_degrees, bond_length=0.9584):
    """Generate H2O integrals at a given H-O-H bond angle."""
    angle_rad = np.radians(angle_degrees)
    # Place O at origin, H atoms symmetric about z-axis
    hx = bond_length * np.sin(angle_rad / 2)
    hz = bond_length * np.cos(angle_rad / 2)

    mol = gto.M(
        atom=f'O 0 0 0; H {hx} 0 {hz}; H {-hx} 0 {hz}',
        basis='sto-3g',
        symmetry=False,
    )
    mf = scf.RHF(mol).run()

    # One-body: kinetic + nuclear attraction in MO basis
    h1 = mf.mo_coeff.T @ mf.get_hcore() @ mf.mo_coeff
    # Two-body: electron-electron repulsion in MO basis
    eri = mol.ao2mo(mf.mo_coeff)
    h2 = ao2mo.restore(1, eri, mol.nao)

    return mol.energy_nuc(), h1, h2
```

The PySCF script runs on a laptop in seconds. For each angle, it produces the nuclear repulsion energy V_{nn}, the one-body integrals h_{pq}, and the two-body integrals h_{pqrs}. We export these as a JSON map with keys like `"0,1"` and `"0,1,2,3"` — the same format our `factory` function expects.

The companion script `book/code/ch18-bond-angle-scan.py` has the full workflow: generate integrals at each angle, write CSVs, and produce the bond angle plot.

The data flows through a simple two-stage pipeline:

```
flowchart LR
    PY["PySCF (Python)<br/>Generate integrals"] -->|"JSON files<br/>(one per angle)"| F
    FM --> CSV["CSV / Plot<br/>Energy vs angle"]
    style PY fill:#e8ecf1,stroke:#6b7280
    style FM fill:#d1fae5,stroke:#059669
```

PySCF produces the molecular integrals (one JSON file per geometry); FockMap reads them, applies the encoding pipeline, and outputs energies. The JSON format is the same `Map<string, Complex>` (keys like `"0,1"` for one-body and `"0,1,2,3"` for two-body) that every companion script uses.

The Scan

With the skeleton precomputed and the integrals generated, the scan itself is remarkably concise:

```
open System.Numerics
open Encodings

// Precompute the Pauli structure once
let skeleton = computeHamiltonianSkeleton ternaryTreeTerms 14u

let angles = [| 60.0 .. 5.0 .. 180.0 |]

for angle in angles do
    // Load integrals for this geometry
    let factory = loadIntegrals (sprintf "h2o_integrals_%03.0f.json" angle)

    // Build the Hamiltonian (fast — just multiplying precomputed strings by numbers)
    let ham = applyCoefficients skeleton factory

    // Taper
    let tapered = taper defaultTaperingOptions ham

    // The ground-state energy is the smallest eigenvalue of the Hamiltonian matrix.
    // For 14 qubits (or ~11 after tapering), exact diagonalisation is still feasible.
    // exactGroundStateEnergy builds the 2^n × 2^n matrix and returns min(eigenvalues).
    let energy = exactGroundStateEnergy tapered.Hamiltonian

    printfn "%.0f°  %.6f Ha" angle energy
```

The inner loop — `applyCoefficients`, `taper`, diagonalise — runs in milliseconds per

geometry. The entire 25-point scan takes a few seconds on a laptop.

The Result: Coarse Scan

The coarse scan produces a table of FCI energies — the exact ground-state energy at each geometry:

Angle (°)	E (Ha)	Angle (°)	E (Ha)
60	−74.927720	110	−75.008946
70	−74.969743	115	−75.003590
80	−74.996296	120	−74.996605
85	−75.004751	130	−74.978530
90	−75.010371	140	−74.956492
95	−75.013394	150	−74.932806
100	−75.014034	160	−74.910675
105	−75.012488	180	−74.888213

The energy drops as the angle increases from 60° (the hydrogen atoms are cramped), reaches a minimum somewhere around 100°, then rises again toward 180° (linear geometry). The coarse scan tells us *where to look* — but the minimum could be anywhere between 95° and 105°.

This is exactly how a computational chemist works: coarse grid first, then refine.

Zooming In: Fine Scan

We re-run the scan from 95° to 115° in 1° steps. The skeleton is already computed — only the integrals change at each angle — so this second pass costs almost nothing:

Angle (°)	E (Ha)	Angle (°)	E (Ha)
95	−75.01339394	106	−75.01193367
96	−75.01370596	107	−75.01130071
97	−75.01392434	108	−75.01059082
98	−75.01405068	109	−75.00980546
99	**−75.01408654**	110	−75.00894610
100	−75.01403351	111	−75.00801419
101	−75.01389312	112	−75.00701118
102	−75.01366689	113	−75.00593855
103	−75.01335633	114	−75.00479776
104	−75.01296296	115	−75.00359029
105	−75.01248824		

The minimum is at **99°** with an energy of **−75.01408654 Ha**. The curve is clearly parabolic — the energy changes by only 0.00005 Ha between 98° and 100°, then drops away more steeply on either side.

The companion script `book/code/ch18-bond-angle-scan.py` generates both scans, writes CSVs, and produces a side-by-side plot saved to `book/code/h2o_bond_angle.png`.

What Do the Numbers Mean?

The minimum at **99°** in STO-3G is ~5° below the experimental value of 104.52°. The discrepancy comes from the minimal basis set — STO-3G uses only one Slater-type orbital per atomic orbital, which is too inflexible to capture the fine details of electron correlation near the equilibrium geometry. A larger basis set (cc-pVDZ, cc-pVTZ) shifts the minimum toward the experimental value. At the cc-pVTZ level, the computed angle is within 0.5° of experiment.

But the point is not the fourth decimal place. The point is the *method*: we derived a molecular geometry from a quantum simulation pipeline. No one told the code that water should bend. No one parameterised the angle. The pipeline explored the energy landscape and found the bend — the same bend that determines water's polarity, its solvent properties, its ability to form hydrogen bonds, and ultimately, its role in sustaining life on Earth.

Why Does Water Bend?

The energy curve tells us *that* water bends. The density matrix framework from Chapter 6 tells us *why*.

At 180° (linear geometry), the molecular orbitals have a symmetry that makes certain exchange integrals vanish. The off-diagonal terms in the density matrix — the ones that carry correlation energy — are constrained by symmetry. As the molecule bends, that symmetry breaks, new exchange pathways open up, and the correlation energy drops. The total energy decreases until the cost of nuclear repulsion (the two hydrogen atoms getting closer) balances the gain in electron correlation.

This is the deeper story: the bond angle is a compromise between nuclear geometry and electronic correlation. VSEPR's "lone pair repulsion" is a cartoon version of this — qualitatively correct, but the real mechanism is a balance of energies that only a full quantum treatment can quantify.

What About the Encoding?

With 14 spin-orbitals, H_2O is the first molecule in this book where encoding choice makes a meaningful difference in circuit cost. If we replace `ternaryTreeTerms` with `jordanWignerTerms` in the skeleton, the energies don't change — same physics — but the Pauli weights increase dramatically:

Encoding	Max weight	Avg weight
Jordan–Wigner	14	~7
Bravyi–Kitaev	5	~3
Parity	14	~7
Binary Tree	5	~3
Ternary Tree	4	~3

The ternary tree encoding produces the lightest Pauli strings, which translates directly to fewer CNOTs per Trotter step. This is the compression we studied in Chapters 7 and 16 — now applied to a molecule large enough for it to matter.

For the energy *scan*, the encoding doesn't affect the result (all encodings give the same eigenvalues). But when we move to a quantum computer — where each CNOT carries noise — the lighter encoding gives a more accurate simulation for the same hardware budget. At H_2O scale, that's roughly a 3× reduction in two-qubit gates.

The Greenhouse Connection

Water's bent shape isn't just a textbook fact — it's one of the reasons Earth supports life.

A bent molecule has a permanent electric dipole moment: the oxygen end is slightly negative, the hydrogen end slightly positive. This dipole oscillates during the bending vibration (ν_2 mode at ≈ 1595 cm^{-1}), creating a time-varying electric field that couples to infrared photons. That coupling is what makes water a greenhouse gas: it absorbs outgoing infrared radiation and re-emits it in all directions, warming the atmosphere.

A linear H_2O molecule would have no permanent dipole and a very different infrared spectrum. The bond angle we just computed — the one that emerged from our pipeline without any empirical input — is literally the reason Earth has a habitable temperature.

The potential energy surface scan we performed is the first step in a vibrational analysis. The *curvature* of the energy curve near the minimum (the second derivative) determines the bending frequency. This is obtained from the **Hessian matrix** of the PES — and computing that Hessian for molecules where classical methods fail is one of the practical applications of quantum simulation.

Key Takeaways

- A potential energy surface scan runs the **same pipeline** at many geometries and plots energy versus structural parameter.
- The **skeleton API** separates structure (encoding-dependent, computed once) from coefficients (geometry-dependent, applied per point), making scans efficient.
- The energy minimum of the H_2O bond angle scan occurs at **99°** in STO-3G — close to the experimental 104.52°, with the discrepancy due to the minimal basis set.
- The bond angle emerges from quantum mechanics without empirical input — it's a balance of nuclear repulsion and electronic correlation.
- Water's bent geometry produces its permanent dipole moment, which makes it a greenhouse gas and a universal solvent — properties that follow directly from the energy curve we computed.

Previous: Chapter 17 — The Complete Pipeline

Next: Chapter 19 — Algorithms: VQE and QPE

Chapter 19: Algorithms — VQE and QPE

Chapters 17 and 18 used exact diagonalisation as a stand-in energy oracle. This chapter replaces that stand-in with the quantum algorithms that run on real hardware.

In This Chapter

- **What you'll learn:** How the two major quantum chemistry algorithms — VQE (variational, near-term) and QPE (phase estimation, fault-tolerant) — consume the encoded Hamiltonian, and what measurement infrastructure they require.
- **Why this matters:** FockMap produces the *input* to these algorithms. Understanding what the algorithms need helps you make better encoding and tapering choices — and understand why the CNOT counts from Chapter 16 matter so much.
- **Prerequisites:** Chapters 16–18 (cost analysis, pipeline, bond angle scan).

Two Algorithms, One Input

In Chapters 17 and 18, we computed ground-state energies by exact diagonalisation — building the Hamiltonian as a matrix and finding its smallest eigenvalue classically. For H_2 ($2^4 = 16$-dimensional matrix) and H_2O in minimal basis ($2^{11} = 2{,}048$-dimensional after tapering), exact diagonalisation is trivially feasible on a laptop. But for larger systems — N_2 (20 qubits → $2^{20} \approx 10^6$) or FeMo-co (108 qubits → $2^{108} \approx 10^{32}$) — the matrix does not fit in any existing memory.

This is where quantum algorithms enter. Instead of building the exponentially large matrix, a quantum computer prepares the state directly in n qubits and extracts the energy through measurement. The pipeline output — the Pauli Hamiltonian $\hat{H} = \sum_k c_k P_k$ — is exactly what these algorithms consume.

There are two main approaches, and they make very different demands on the hardware.

VQE (Variational Quantum Eigensolver) uses short circuits and many measurements. It's designed for *near-term* quantum computers — noisy devices with limited circuit

depth but reasonable qubit counts. The quantum computer prepares a trial state and measures Pauli expectation values; a classical computer optimises the state parameters.

QPE (Quantum Phase Estimation) uses deep circuits and few measurements. It's designed for *fault-tolerant* quantum computers — devices with error correction, where circuit depth is not the bottleneck. The quantum computer extracts the ground-state energy directly via phase kickback.

Both algorithms consume the same object: a Pauli Hamiltonian $\hat{H} = \sum_k c_k P_k$ — exactly what our pipeline produces.

VQE: The Near-Term Algorithm

The Loop

VQE is fundamentally an optimisation loop. The quantum computer serves as a function evaluator — it takes parameters θ and returns an energy estimate $\langle \hat{H} \rangle_\theta$. The classical computer adjusts θ to minimize the energy.

```
flowchart TD
    PREP["Prepare |ψ(θ)⟩<br/>(parameterized ansatz)"]
    MEAS["Measure each Pauli term<br/>⟨ψ|P_k|ψ⟩"]
    SUM["Compute ⟨H⟩ = Σ c_k ⟨P_k⟩"]
    OPT["Update θ<br/>(classical optimizer)"]
    CONV{"Converged?"}
    DONE["Report E₀ ≈ ⟨H⟩_min"]

    PREP --> MEAS --> SUM --> CONV
    CONV -->|No| OPT --> PREP
    CONV -->|Yes| DONE
    style MEAS fill:#fde68a,stroke:#d97706
    style DONE fill:#d1fae5,stroke:#059669
```

The yellow box is where FockMap's output matters most. Measuring $\langle P_k \rangle$ for each Pauli term in the Hamiltonian requires a separate circuit — or, better, measuring several commuting terms simultaneously.

Measurement Grouping

Not every Pauli term needs its own circuit. Two Pauli operators can be measured simultaneously if they **qubit-wise commute** — that is, if they commute on every individual qubit position. For example, ZI and ZZ qubit-wise commute (both have Z or I at each position), so a single Z-basis measurement of all qubits yields both expectation values.

FockMap groups the Hamiltonian terms automatically:

```
let program = groupCommutingTerms hamiltonian
printfn "Total terms: %d" program.TotalTerms
printfn "Measurement groups: %d" program.GroupCount
```

For the 15-term H_2 Hamiltonian, this typically produces 5 measurement groups — a 3× reduction in the number of distinct circuits needed.

The grouping matters because each distinct circuit must be run many times (to accumulate statistics), and each additional group multiplies the total measurement time. Fewer groups = faster VQE iterations.

Shot Counts

Each measurement gives a binary outcome (± 1). To estimate the expectation value $\langle P_k \rangle$ to precision δ, you need approximately $1/\delta^2$ repetitions ("shots"). But where does the total shot count formula come from? Let's derive it.

The energy estimator. The energy is $\langle \hat{H} \rangle = \sum_k c_k \langle P_k \rangle$. Each $\langle P_k \rangle$ is estimated from N_k measurement shots, giving an estimator $\hat{m}_k$ with variance $\mathrm{Var}(\hat{m}_k) \leq 1/N_k$ (since each shot yields ± 1, the variance of a single outcome is at most 1).

Error propagation. The variance of the total energy estimate is:

$$\mathrm{Var}(\hat{E}) = \sum_k c_k^2 \, \mathrm{Var}(\hat{m}_k) \leq \sum_k \frac{c_k^2}{N_k}$$

Optimal allocation. If we have a total shot budget $N_{\text{total}} = \sum_k N_k$ and want to minimise $\mathrm{Var}(\hat{E})$, Lagrange optimisation gives the optimal allocation $N_k \propto |c_k|$. Substituting back:

$$\mathrm{Var}(\hat{E}) = \frac{1}{N_{\text{total}}} \left(\sum_k |c_k| \right)^2$$

Setting the target. For energy precision ϵ (meaning standard deviation $\leq \epsilon$), we need $\mathrm{Var}(\hat{E}) \leq \epsilon^2$, giving:

$$N_{\text{total}} \geq \frac{1}{\epsilon^2} \left(\sum_k |c_k| \right)^2$$

This is the shot-count formula. The key quantity is the **1-norm** $\sum_k |c_k|$ — it determines the measurement cost because terms with larger coefficients contribute more variance and demand more shots. This estimate assumes each Pauli term is measured independently (or in qubit-wise commuting groups) and that the dominant error source is finite sampling. In practice, correlated measurement strategies (classical shadows, derandomisation) can improve the scaling, but the 1-norm estimate

provides a useful and widely-cited baseline (Wecker et al., Phys. Rev. A 92, 042303, 2015).

Tapering reduces this norm (fewer terms, smaller coefficients), which is yet another benefit that compounds with the qubit reduction.

```
let shots = estimateShots 0.0016 hamiltonian  // chemical accuracy = 1.6 mHa
printfn "Estimated shots for chemical accuracy: %d" shots
```

For H_2: $\sum |c_k| \approx 3.7$ Ha, giving ~5 million shots at chemical accuracy. For H_2O: the 1-norm is larger, and the shot count grows accordingly. This is the practical bottleneck of VQE — not circuit depth, but measurement overhead.

Where FockMap Fits

FockMap's contribution to VQE is everything *before* the measurement loop:

1. **The Hamiltonian** $\{c_k, P_k\}$ — the list of Pauli terms and coefficients
2. **The measurement program** — which terms to group and measure together
3. **The shot budget** — how many measurements each group needs
4. **The Trotter circuit** — used as the ansatz or as part of the state preparation

The actual VQE loop — parameter optimisation, circuit execution, shot collection — is handled by execution frameworks like Qiskit, Cirq, or Quokka. FockMap produces the input they consume.

QPE: The Fault-Tolerant Algorithm

The Idea

QPE doesn't optimise — it *reads out* the ground-state energy directly, like a quantum spectrometer. The key insight: if we can implement the time-evolution operator $e^{-i\hat{H}t}$ as a quantum circuit, then phase kickback extracts the eigenvalue.

This is where Trotterization becomes essential. The operator $e^{-i\hat{H}t}$ is approximated by a product of Pauli rotations — exactly the Trotter step from Chapter 13. QPE applies this operation repeatedly, controlled by ancilla qubits, and reads the energy from the ancilla register.

The Circuit

The QPE circuit has two registers. The **ancilla register** (m qubits, initialised to $|0\rangle$, each put into superposition with a Hadamard gate) controls repeated applications of the time-evolution operator on the **system register** (which holds the trial state $|\psi\rangle$):

1. Ancilla qubit j controls the application of U^{2^j} to the system register, where $U = e^{-i\hat{H}t}$ is one Trotter step.

2. After all controlled operations, an inverse QFT on the ancilla register converts the accumulated phases into a binary representation of the energy eigenvalue.
3. Measuring the ancilla register yields m bits of the energy.

The controlled-U^{2^j} operation applies 2^j Trotter steps — so the highest ancilla qubit controls 2^{m-1} steps. This is where the circuit depth comes from — and why QPE demands fault-tolerant hardware.

The cost is dominated by the controlled Trotter steps. Each step requires the same gates as a regular Trotter step (Chapter 15), plus control logic — roughly doubling the CNOT count per step.

Resource Estimation

For a target energy precision ϵ:

- **System qubits**: n (from the encoding, after tapering)
- **Ancilla qubits**: $m = \lceil \log_2(1/\epsilon) \rceil$ (one bit per digit of precision)
- **Trotter steps per controlled-U**: enough to keep the Trotter error below ϵ

```
let resources = qpeResources 10 hamiltonian 0.1
printfn "System qubits:  %d" resources.SystemQubits
printfn "Ancilla qubits: %d" resources.AncillaQubits
printfn "Total CNOTs:    %d" resources.TotalCnots
printfn "Circuit depth:  %d" resources.CircuitDepth
```

For H_2 at chemical accuracy (~10 bits of precision): - System: 2–4 qubits (depending on tapering) - Ancilla: 10 qubits - Total: ~14 qubits, ~10,000 CNOTs

For H_2O at chemical accuracy: - System: ~11 qubits (after tapering) - Ancilla: 10 qubits - Total: ~21 qubits, ~500,000 CNOTs

The 50× jump from H_2 to H_2O is driven by two factors: more qubits (more terms, heavier strings) and more Trotter steps (to keep the error small for a larger Hamiltonian norm). This is why encoding choice and tapering matter — they reduce both factors.

VQE vs QPE: A Summary

	VQE	QPE
Hardware	Near-term (noisy)	Fault-tolerant
Circuit depth	Short (one ansatz layer)	Deep (many Trotter steps)
Measurements	Many ($\sim 10^6$ shots)	Few (one readout)
Classical cost	Optimisation loop	QFT + readout
Bottleneck	Shot count, barren plateaus	Circuit depth, error correction
FockMap provides	Hamiltonian + measurement groups	Hamiltonian + Trotter circuits

For the bond angle scan in Chapter 18, we used exact diagonalisation to obtain the same target quantity that an ideal QPE calculation would produce: the ground-state energy at each geometry. On real near-term hardware, you would use VQE at each angle, accumulating enough shots to estimate the energy to chemical accuracy. The structure of the scan is the same; only the energy-evaluation subroutine changes.

The Measurement Program in Practice

Let's look at the measurement infrastructure for H_2 in detail. The 15-term Hamiltonian groups into measurement bases:

```
let ham = computeHamiltonianWith jordanWignerTerms h2Factory 4u
let program = groupCommutingTerms ham

for i in 0 .. program.Bases.Length - 1 do
    let basis = program.Bases.[i]
    printfn "Group %d: %d terms  (basis: %s)"
        (i + 1)
        basis.Terms.Length
        (basis.BasisRotation |> Array.map string |> String.concat "")
```

A typical grouping might look like:

Group	Terms	Measurement basis
1	5	ZZZZ (computational basis)
2	3	XXXX
3	3	XXYY
4	2	YXXY
5	2	YYXX

Group 1 contains all the diagonal (Z-only) terms — they can all be measured in the standard computational basis. The off-diagonal groups each require basis rotations (Hadamard and S gates) before measurement.

This is the bridge between FockMap's algebraic output and the experimental reality of a quantum computer: each group becomes a circuit variant, each circuit runs thousands of times, and the statistics are combined to estimate the energy.

What FockMap Does — and What It Doesn't

FockMap's scope ends at circuit generation. It is important to be clear about what lies *outside* that scope:

- **Ansatz design**: VQE requires a parameterized circuit (ansatz). FockMap does not design ansätze — it provides Trotter circuits and Pauli operator pools that downstream tools (Qiskit, tket, PennyLane) can use as building blocks.
- **Classical optimisation**: the VQE optimisation loop (choosing θ, handling barren plateaus, selecting an optimizer like L-BFGS or COBYLA) is handled by the execution framework, not FockMap.
- **Hardware execution**: submitting circuits to real devices, managing job queues, handling device calibration — all downstream.
- **Error mitigation**: ZNE, PEC, symmetry verification — these operate on measurement outcomes and are handled by execution frameworks. (FockMap's tapering symmetries are useful *inputs* to symmetry verification, but FockMap doesn't implement the mitigation itself.)
- **Noise modelling**: hardware noise interacts with both Trotter error and measurement error. FockMap's cost estimates are for *ideal* (noiseless) circuits; real performance depends on device characteristics.

  Barren plateaus, briefly: as the number of qubits grows, randomly initialized VQE ansätze tend to produce cost function landscapes where the gradient vanishes exponentially — a phenomenon called a barren plateau. This is a fundamental challenge for VQE at scale, and it is one reason why adaptive methods (ADAPT-VQE) and physically motivated ansätze (e.g., UCCSD) are preferred over hardware-efficient random circuits.

Key Takeaways

- **VQE** uses FockMap's Hamiltonian for measurement-based energy estimation. Fewer Pauli terms and lower 1-norm (from tapering) directly reduce the shot count.
- **QPE** uses FockMap's Trotter circuits for phase estimation. Lower CNOT counts (from encoding choice) directly reduce the circuit depth.
- **Measurement grouping** reduces the number of distinct circuits by combining qubit-wise commuting terms. FockMap does this automatically.
- Both algorithms consume the same input — the Pauli Hamiltonian — but make different demands on the hardware. Encoding and tapering choices help both.

Previous: Chapter 18 — The Water Bond Angle

Next: Chapter 20 — Speaking the Hardware's Language

Chapter 20: Speaking the Hardware's Language

The circuit exists in FockMap's type system. To run it on real hardware, we need to speak the machine's language.

In This Chapter

- **What you'll learn:** How to export FockMap gate sequences as OpenQASM (universal), Q# (Azure Quantum), or JSON (Python ecosystem) — and when to use which.
- **Why this matters:** A gate sequence that only exists in memory is a theoretical result. An exported circuit is an experiment you can run.
- **Prerequisites:** Chapter 17 (the complete pipeline).

Three Formats, One Circuit

FockMap's Trotter decomposition (Chapter 15) produces a concrete gate sequence — an array of `Gate` values representing Hadamard, S, CNOT, and Rz operations. That array is a platform-independent description of the circuit. But to execute it, you need to express it in a format that a quantum platform understands.

The three main targets:

Format	Ecosystem	Platforms
OpenQASM 3.0	Universal standard	IBM Quantum, IonQ, Rigetti, Amazon Braket
Q#	Microsoft	Azure Quantum, QDK simulator
JSON	Python SDKs	Qiskit, Cirq, Quokka

FockMap can export to all three from the same gate array. The structure is always the same: take the Trotter step, decompose it to gates, export.

OpenQASM: The Universal Format

OpenQASM (Open Quantum Assembly Language) is maintained by IBM and accepted by every major quantum platform. Version 3.0 supports qubit declarations, standard gates, parameterized rotations, and classical control.

```
let step = firstOrderTrotter 0.1 hamiltonian
let gates = decomposeTrotterStep step
let qasm = toOpenQasm defaultOpenQasmOptions numQubits gates
printfn "%s" qasm
```

The output looks like:

```
OPENQASM 3.0;
include "stdgates.inc";

qubit[2] q;

h q[0];
cx q[0], q[1];
rz(0.01234567) q[1];
cx q[0], q[1];
h q[0];
```

Each Pauli rotation from Chapter 15 becomes a CNOT staircase bracketed by basis-change gates. The angle in `rz()` is the rotation parameter $\theta = c_k \Delta t$.

Gate Mapping

FockMap Gate	OpenQASM
`Gate.H i`	`h q[i];`
`Gate.S i`	`s q[i];`
`Gate.Sdg i`	`sdg q[i];`
`Gate.CNOT (c, t)`	`cx q[c], q[t];`
`Gate.Rz (i, θ)`	`rz(θ) q[i];`

Configuration

```
type OpenQasmOptions =
    { IncludeHeader : bool      // "OPENQASM 3.0;" + include
      QubitName     : string    // default: "q"
      Precision     : int       // decimal places for angles
      Version       : QasmVersion }  // V2 or V3

// QASM 2.0 for Quokka compatibility
let qasm2 = toOpenQasm defaultOpenQasm2Options numQubits gates
```

QASM 2.0 differs in syntax (e.g., `qreg q[2];` instead of `qubit[2] q;`) but the gate names are identical. Use V2 when the target platform doesn't yet support V3.

Q#: Azure Quantum

Q# is Microsoft's quantum programming language, designed from the ground up for expressing quantum algorithms (Svore et al., "Q#: Enabling Scalable Quantum Computing and Development with a High-level DSL", arXiv:1803.00652). Full disclosure: the author was part of the team at Microsoft that created Q# — so the export format here isn't a reverse-engineering exercise, it's a homecoming. FockMap generates a complete Q# operation — namespace, open statements, qubit allocation, gate calls — that you can compile and run on Azure Quantum or the local QDK simulator.

```
let qsharp = toQSharp defaultQSharpOptions numQubits gates
printfn "%s" qsharp
```

The output:

```
namespace FockMap.Generated {
    open Microsoft.Quantum.Canon;
    open Microsoft.Quantum.Intrinsic;

    operation TrotterStep() : Unit {
        use q = Qubit[2];
        H(q[0]);
        CNOT(q[0], q[1]);
        Rz(0.01234567, q[1]);
        CNOT(q[0], q[1]);
        H(q[0]);
    }
}
```

Configuration

```
type QSharpOptions =
    { Namespace     : string    // default: "FockMap.Generated"
      OperationName : string    // default: "TrotterStep"
      Precision     : int }     // decimal places for angles
```

The generated operation is a pure gate sequence — no measurement, no classical control. The calling code (your VQE or QPE driver) allocates the qubits, calls `TrotterStep()`, and handles measurement.

JSON: The Python Bridge

Most quantum computing platforms have Python SDKs (Qiskit, Cirq, PennyLane, Quokka). FockMap exports circuits as JSON that can be loaded by any Python library:

```
let metadata = Map [
    ("molecule", "H2")
    ("encoding", "ternary-tree")
    ("trotter_order", "1")
    ("time_step", "0.1")
]
let json = toCircuitJson numQubits metadata gates
System.IO.File.WriteAllText("h2_circuit.json", json)
```

The JSON schema:

```
{
  "num_qubits": 2,
  "metadata": {
    "molecule": "H2",
    "encoding": "ternary-tree",
    "trotter_order": "1",
    "time_step": "0.1"
  },
  "gates": [
    { "type": "H", "qubit": 0 },
    { "type": "CNOT", "control": 0, "target": 1 },
    { "type": "Rz", "qubit": 1, "angle": 0.01234567 },
    { "type": "CNOT", "control": 0, "target": 1 },
    { "type": "H", "qubit": 0 }
  ]
}
```

On the Python side:

```
import json
from qiskit import QuantumCircuit

with open("h2_circuit.json") as f:
    data = json.load(f)

qc = QuantumCircuit(data["num_qubits"])
for g in data["gates"]:
    if g["type"] == "H":
        qc.h(g["qubit"])
    elif g["type"] == "CNOT":
        qc.cx(g["control"], g["target"])
    elif g["type"] == "Rz":
        qc.rz(g["angle"], g["qubit"])
    elif g["type"] == "S":
```

```
        qc.s(g["qubit"])
    elif g["type"] == "Sdg":
        qc.sdg(g["qubit"])
```

The JSON format is deliberately simple — flat list of gates, no nested structures — so that any Python library can consume it with a few lines of code.

Same Circuit, Three Formats

Here is the full pipeline for H_2, exporting to all three formats:

```
let ham = computeHamiltonianWith ternaryTreeTerms factory 4u
let tapered = taper defaultTaperingOptions ham
let step = firstOrderTrotter 0.1 tapered.Hamiltonian
let gates = decomposeTrotterStep step
let n = tapered.TaperedQubitCount

// OpenQASM 3.0
let qasm = toOpenQasm defaultOpenQasmOptions n gates
System.IO.File.WriteAllText("h2.qasm", qasm)

// Q#
let qs = toQSharp defaultQSharpOptions n gates
System.IO.File.WriteAllText("h2.qs", qs)

// JSON (for Python/Qiskit)
let json = toCircuitJson n Map.empty gates
System.IO.File.WriteAllText("h2.json", json)

printfn "Exported %d gates to 3 formats" gates.Length
```

The gate array is the same in all three cases. Only the serialisation differs.

When to Use Which

OpenQASM is the safe default. Every major platform accepts it, and QASM files are human-readable. Use QASM when: - You want maximum portability - You're submitting to IBM Quantum, IonQ, or Amazon Braket - You need a format that works with transpilation tools (Qiskit, tket)

Q# when your target is Azure Quantum. The generated operation integrates directly into the Q# project system, with type checking and resource estimation built in.

JSON when your workflow lives in Python. The JSON bridge lets you construct the Hamiltonian and gate sequence in F# (where the algebra is exact) and run the circuit

in Python (where the hardware SDKs live). This is the "best of both worlds" approach: algebraic precision in the construction, ecosystem breadth in the execution.

What Export Does Not Solve

FockMap's circuit export produces *logical* circuits — abstract sequences of ideal gates. Before a logical circuit can run on real hardware, several additional steps are required. These are handled by platform-specific compilers (Qiskit transpiler, tket, Q# resource estimator), not by FockMap:

- **Native gate lowering**: hardware devices implement a small native gate set (e.g., $\{\sqrt{X}, R_z, \text{CZ}\}$ for IBM, $\{R_{xx}, GPI, GPI2\}$ for IonQ). The logical gates must be decomposed into the native set.
- **Qubit routing**: physical qubits have limited connectivity (e.g., heavy-hex topology on IBM devices). SWAP gates must be inserted to bring non-adjacent qubits together for two-qubit gates. This can increase CNOT count by 2–5×.
- **Circuit optimisation**: transpilers cancel redundant gates, commute operations to reduce depth, and apply template-matching identities. This typically reduces gate count by 20–40% beyond FockMap's unoptimised output.
- **Verification after import**: hardware compilers may reorder or decompose gates. Always verify that the transpiled circuit still produces the correct expectation values on a simulator before submitting to hardware.

Verification Checklist

Whichever format you choose, verify the output end-to-end:

1. **Import** the circuit into the target platform (Qiskit, Q#, Cirq)
2. **Simulate** with a statevector backend (zero noise, exact amplitudes)
3. **Compute** $\langle \hat{H} \rangle$ from the statevector and compare against the exact eigenvalue from Chapter 8
4. **Transpile** to the target hardware's native gate set
5. **Re-simulate** the transpiled circuit and check that the energy is unchanged (within Trotter error)
6. **Only then** submit to real hardware or a noisy simulator

For H_2, the exact FCI energy is -1.1373 Ha (electronic only) or -1.8572 Ha (including nuclear repulsion). The Trotterised circuit should produce an energy within $O(\Delta t^2)$ of this — typically within 0.01 Ha at $\Delta t = 0.1$. If the deviation is larger, check the integral convention (Chapter 2) and the encoding consistency (Chapter 8).

Key Takeaways

- FockMap exports to **three formats** from the same gate array: OpenQASM (universal), Q# (Azure), JSON (Python).
- The gate sequence is platform-independent; only the serialisation differs.
- **OpenQASM** for portability, **Q#** for Azure integration, **JSON** for Python workflows.
- Always verify by comparing simulated expectation values against known eigenvalues.

Previous: Chapter 19 — Algorithms: VQE and QPE

Next: Chapter 21 — Scaling: From H_2 to FeMo-co

Chapter 21: Scaling — From H_2 to FeMo-co

H_2 was our teacher. H_2O was our first real test. Now we look at where the pipeline goes — and where it meets its limits.

In This Chapter

- **What you'll learn:** How the pipeline's cost scales from small test systems through medium molecules to the grand challenge of quantum chemistry, and where encoding choice makes — or breaks — the simulation.
- **Why this matters:** The promise of quantum simulation is that it handles molecules classical computers cannot. This chapter asks: at what point does that promise become real, and what does the hardware need to deliver?
- **Prerequisites:** Chapters 16–18 (cost analysis, pipeline, bond angle scan).

The Scaling Landscape

Every quantity in our pipeline grows with the number of spin-orbitals n:

Quantity	Growth	H_2 (n=4)	H_2O (n=14)	N_2 (n=20)	FeMo-co (n≈108)
Qubits	n	4	14	20	~108
Hamiltonian terms	$O(n^4)$	15	~600	~2,000	~10^7
JW max weight	n	4	14	20	108
TT max weight	$O(\log_3 n)$	3	4	5	~5
Configurations (FCI)	$\binom{n}{n_e}$	6	1,001	38,760	~10^{30}

The configuration count is why classical methods fail. Full CI (exact diagonalisation)

scales as $\binom{n}{n_e}$ — the number of ways to place n_e electrons in n spin-orbitals. For FeMo-co, that's roughly 10^{30} determinants. No classical computer will ever enumerate them.

A quantum computer doesn't enumerate configurations — it represents the quantum state directly in n qubits. The cost is in the *circuit*, not the *state space*. And the circuit cost depends on the encoding.

The Encoding Crossover

At 4 qubits, all encodings cost the same (Chapter 17). At 14 qubits, the differences appear (Chapter 18). Let's trace the crossover:

Molecule	n	JW CNOTs/step	TT CNOTs/step	Ratio
H_2	4	12	12	1.0×
LiH	12	~400	~200	2.0×
H_2O	14	~1,800	~600	3.0×
N_2	20	~8,000	~2,000	4.0×
FeMo-co	~108	~10^7	~10^5	~100×

The ratio grows because JW's Pauli weights grow linearly while TT's grow logarithmically. At FeMo-co scale, the difference is roughly two orders of magnitude for the *logical* (pre-transpilation) circuit. After hardware-specific transpilation (qubit routing, native gate decomposition), the absolute gate counts increase for all encodings, but the *relative* advantage of lighter Pauli weights is preserved.

> **Caveat on the estimates:** The CNOT counts in this table are for *unoptimised logical circuits* — the direct output of FockMap's CNOT staircase decomposition. Hardware transpilers typically reduce these by 20–40% through gate cancellation, commutation, and template matching. The estimates for N_2 and FeMo-co are order-of-magnitude projections based on scaling trends, not computed values.

Why the Ratio Matters

On near-term hardware, each CNOT gate has a finite error rate — typically 0.1–1%. The probability that a circuit executes correctly drops exponentially with the CNOT count:

$$P_{\text{success}} \approx (1-\varepsilon)^{C_{\text{CNOT}}}$$

At $\varepsilon = 0.5\%$ and 1,800 CNOTs (JW for H_2O), $P_{\text{success}} \approx 0.01\%$. At 600 CNOTs (TT for H_2O), $P_{\text{success}} \approx 5\%$. Under these typical assumptions, that's roughly a 500×

improvement in success probability from encoding choice alone — before error mitigation, before hardware improvements, before anything else. (In practice, error mitigation techniques like zero-noise extrapolation can partially compensate for low success rates, but they impose their own sampling overhead.)

This is why the chapters on encoding (5–8), tapering (9–12), and cost analysis (16) matter. They're not academic exercises. They directly determine whether a simulation succeeds or fails on real hardware.

The Tapering Dividend

Tapering compounds with encoding choice. At each system size, tapering removes k qubits and often reduces the term count:

Molecule	n	Tapered $n-k$	Terms (before)	Terms (after)
H_2	4	2	15	5
H_2O	14	~11	~600	~300
N_2	20	~16	~2,000	~1,200

Fewer terms means fewer Pauli rotations per Trotter step. Combined with lower Pauli weights from a good encoding, the circuit shrinks multiplicatively. The optimisation stack from Chapter 16 — encoding + tapering + Trotter order — compounds at every scale.

Application: Encoding Choice at H_2O Scale

With 14 spin-orbitals, H_2O is the smallest molecule where encoding choice makes a practical difference. Here's the full comparison after tapering:

Encoding	Tapered qubits	Max weight	CNOTs/step	Depth estimate
Jordan–Wigner	~11	11	~1,800	~3,600
Bravyi–Kitaev	~11	5	~750	~1,500
Parity	~11	11	~1,800	~3,600
Binary Tree	~11	5	~700	~1,400
Ternary Tree	~11	4	~600	~1,200

The 3× reduction from JW to TT is the difference between a circuit that fries on near-term hardware and one that might just survive. On a device with ~99.5% two-qubit gate fidelity, the TT circuit has roughly a 10× higher success probability per shot. Over millions of VQE shots, that translates directly to better energy estimates.

This is the practical answer to "which encoding should I use?" — at H_2O scale and beyond, ternary tree with tapering is the best option in the FockMap toolkit.

The Grand Challenge: FeMo-co

The iron-molybdenum cofactor (FeMo-co) of nitrogenase is the molecule that launched a field. It catalyses nitrogen fixation — converting atmospheric N_2 to ammonia — and understanding its mechanism could transform fertiliser production, one of the most energy-intensive industrial processes on Earth.

FeMo-co has ~54 active electrons in ~108 active spin-orbitals. Classical methods cannot accurately compute its electronic structure because the iron centres are **strongly correlated**: many electron configurations contribute comparably to the ground state, defeating perturbation theory and single-reference methods like coupled cluster.

The Numbers

Quantity	Value
Active electrons	~54
Active spin-orbitals	~108
Qubits (JW)	~108
JW max Pauli weight	108
TT max Pauli weight	~5
JW CNOTs per worst-case rotation	214
TT CNOTs per worst-case rotation	8
Ratio	27×

At 108 qubits, the JW encoding produces Pauli strings where nearly every qubit participates. Each off-diagonal rotation requires a CNOT staircase spanning the entire register. The ternary tree encoding compresses the worst-case weight to ~5 — a chain of 4 CNOTs per direction.

How Far Away Is the Hardware?

For FeMo-co at chemical accuracy via QPE: - **Qubits needed**: ~108 system + ~10 ancilla ≈ 120 logical qubits - **Trotter steps**: $\sim 10^4$ (for the required precision) - **CNOTs per Trotter step** (TT): $\sim 10^5$ - **Total CNOTs**: $\sim 10^9$

At current error rates, this requires full quantum error correction — perhaps 1,000–10,000 physical qubits per logical qubit, depending on the code and hardware. That puts the total physical qubit count at 10^5–10^6.

No device available today can do this. But the same pipeline we developed for H_2 (4 qubits, 15 terms, 12 CNOTs) is the *same code* that would generate the FeMo-co circuit. The bottleneck is hardware, not software. When the hardware arrives, the pipeline is ready.

Where Classical Methods Still Win

It's worth being honest about the landscape. Quantum simulation has a theoretical advantage for strongly correlated systems, but classical methods are formidable:

- **Density functional theory (DFT)** handles hundreds of atoms routinely. It's not exact, but it's remarkably accurate for weakly correlated systems and costs $O(n^3)$.
- **Coupled cluster CCSD(T)** — the "gold standard" of quantum chemistry — handles up to ~50 atoms with chemical accuracy for single-reference systems.
- **DMRG and tensor network methods** exploit the entanglement structure of 1D and quasi-1D systems, often matching quantum simulation accuracy for those geometries.

Quantum simulation's niche is the strongly correlated regime: transition-metal complexes, open-shell systems, conical intersections, and exotic electronic states where no classical method converges reliably. FeMo-co is the poster child, but the real impact may be in catalysis design, high-temperature superconductors, and photochemistry — areas where electron correlation defies classical approximation.

Key Takeaways

- Circuit cost grows with system size, but **encoding choice** determines the growth rate: linear (JW) vs logarithmic (TT) Pauli weight.
- The **encoding crossover** becomes meaningful around 12–14 qubits — exactly the range of near-term quantum hardware.
- **Tapering compounds** with encoding choice, reducing both qubit count and term count multiplicatively.
- **FeMo-co** (~108 qubits) is the grand challenge: classically intractable, but requiring fault-tolerant hardware that doesn't yet exist.
- The same pipeline code works at every scale — from H_2 to FeMo-co. The bottleneck is hardware, not software.

Previous: Chapter 20 — Speaking the Hardware's Language

Next: Chapter 22 — What Comes Next

Chapter 22: What Comes Next

The pipeline is complete. The book is almost over. But the field is just getting started.

In This Chapter

- **What you'll learn:** What lies beyond this book — the extensions, open problems, and emerging directions that build on the foundation we've laid.
- **Why this matters:** A pipeline is only as useful as the questions it can answer. This chapter maps out where those questions lead.
- **Prerequisites:** The whole book (but especially Chapters 17–21).

What We Built

Let's take a final inventory. Over twenty-one chapters, we constructed a complete pipeline:

```
flowchart LR
    MOL["Molecule"]
    INT["Integrals"]
    HAM["Hamiltonian"]
    TAP["Tapered"]
    TROT["Trotter step"]
    CIRC["Circuit"]

    MOL -->|"Ch 1–3"| INT -->|"Ch 5–7"| HAM -->|"Ch 9–12"| TAP -->|"Ch 13–16"| TROT -->
    style MOL fill:#e8ecf1,stroke:#6b7280
    style CIRC fill:#d1fae5,stroke:#059669
```

Every arrow is a function call in FockMap. Every box has been tested on H_2 and applied to H_2O. The pipeline is real, open-source, and published on NuGet.

But a pipeline is an instrument, not a destination. Here's where the instrument gets used.

Near-Term Extensions

Circuit Optimisation

FockMap currently produces *unoptimised* gate sequences — each Pauli rotation becomes a CNOT staircase exactly as described in Chapter 15. Real quantum compilers (Qiskit's transpiler, Cambridge Quantum's tket, BQSKit) apply additional transformations:

- **Gate cancellation**: adjacent CNOT pairs cancel. Identity-equivalent sequences are removed.
- **Commutation-based reordering**: Pauli rotations that commute can be reordered to bring cancellable gates adjacent.
- **Template matching**: known circuit identities replace expensive subcircuits with cheaper equivalents.
- **Hardware-aware routing**: map logical qubits to physical qubits, inserting SWAP gates as needed for limited connectivity.

These optimisations typically reduce CNOT count by 20–40% beyond what FockMap produces. They operate on the gate-level output (QASM, Q#, or JSON) and don't require changes to the Hamiltonian construction.

Error Mitigation

Near-term quantum computers are noisy. Error mitigation techniques extract better estimates from noisy circuits without the overhead of full error correction:

- **Zero-noise extrapolation (ZNE)**: run the circuit at several artificial noise levels, then extrapolate to the zero-noise limit.
- **Probabilistic error cancellation (PEC)**: represent the ideal circuit as a linear combination of noisy circuits and sample the combination.
- **Symmetry verification**: post-select on measurement outcomes that satisfy known symmetries of the Hamiltonian (our tapering symmetries from Chapter 10 are ideal for this).

FockMap's contribution is the symmetry information: the Z_2 generators, Clifford rotations, and parity sectors from the tapering analysis provide exactly the symmetry constraints that verification-based mitigation needs.

Adaptive Ansätze

VQE (Chapter 19) uses a fixed ansatz — a predetermined circuit structure with adjustable parameters. Adaptive methods grow the ansatz one operator at a time:

- **ADAPT-VQE**: at each iteration, select the Pauli operator from the Hamiltonian pool that gives the steepest energy gradient, add it to the ansatz, and re-optimise.
- **Qubit-ADAPT**: same idea, but using single-qubit and two-qubit operators instead of full Pauli strings.

FockMap's Pauli-level representation of the Hamiltonian provides the operator pool that ADAPT-VQE selects from. The encoding choice affects the pool — lighter Pauli strings make better ADAPT operators, which is another argument for ternary tree encoding at scale.

Bosonic Simulation

FockMap already supports bosonic ladder operators and three bosonic-to-qubit encodings (unary, binary, Gray code — see the API documentation). The natural extension is **vibronic simulation**: mixed electron-phonon systems where both fermions and bosons are present.

Applications include: - **Molecular vibrations**: the bond angle scan in Chapter 18 gives the equilibrium geometry; the *curvature* of the PES gives vibrational frequencies, and the vibrational wavefunctions determine infrared spectra. - **Polaron physics**: electron-phonon coupling in materials science, where an electron "dressed" by lattice distortions has different effective mass and mobility. - **Photochemistry**: conical intersections and non-adiabatic dynamics, where nuclear and electronic motion couple strongly.

The encoding pipeline for bosonic modes parallels the fermionic one: ladder operators → Pauli strings → Hamiltonian assembly. The main difference is that bosonic modes require a truncation parameter d (the maximum occupation number per mode), and the encoding choice affects how many qubits each mode costs.

Lattice Models

The second-quantised framework isn't limited to molecules. Lattice models in condensed matter physics use the same operator algebra:

- **Hubbard model**: electrons hopping on a lattice with on-site repulsion. The same ladder operators, the same encodings, the same tapering — different integrals.
- **Heisenberg model**: spin-spin interactions on a lattice. Already expressed in Pauli operators — no encoding step needed.
- **Fermi-Hubbard at half-filling**: a testing ground for quantum advantage, where classical methods (DMRG, QMC) have known limitations in 2D.

FockMap's encoding and tapering infrastructure applies directly to these systems. The Hamiltonian assembly step simplifies (lattice models have regular structure), but the downstream pipeline — Trotter decomposition, cost analysis, circuit export — is identical.

Quantum Error Correction

The tapering machinery from Chapters 9–12 is, at its core, stabiliser theory: finding commuting Pauli operators that generate a symmetry group, and using Clifford rotations to diagonalise them. This is precisely the mathematical framework of quantum error-correcting codes.

- **Stabiliser codes** (surface code, Steane code, etc.) encode logical qubits into physical qubits using a stabiliser group — a set of commuting Pauli operators whose simultaneous eigenspace defines the code space.
- **Syndrome measurement** detects errors by measuring the stabilisers, exactly as our tapering step measures the Z_2 generators.
- **Logical operators** act within the code space, just as our tapered Hamiltonian acts within the symmetry sector.

The algebra is the same. The difference is intent: tapering exploits physical symmetries to reduce qubit count, while error correction engineers artificial symmetries to detect and correct errors. A reader who has understood Chapters 9–12 has already learned half of quantum error correction theory.

Open Problems

Some questions this book doesn't answer — because nobody has yet:

1. **Optimal encoding**: is there a provably optimal encoding for a given Hamiltonian? The ternary tree encoding achieves $O(\log_3 n)$ worst-case weight, which matches a known lower bound for balanced trees. But the best encoding might depend on the Hamiltonian's structure (sparsity, symmetry, locality), not just n.

2. **Tapering beyond Z_2**: our tapering removes symmetries that square to the identity (Z_2 symmetries). What about higher-order symmetries — U(1) particle number conservation, SU(2) spin symmetry? These could remove more qubits, but the Clifford rotation synthesis is harder.

3. **Trotter error bounds**: how many Trotter steps do you actually need? Tight error bounds for product formulas are an active area of research. Tighter bounds mean shorter circuits, which means earlier quantum advantage.

4. **Classical simulation limits**: where exactly does classical simulation become infeasible? DMRG and tensor network methods keep improving. The crossover point — where quantum simulation beats the best classical method — shifts with every algorithmic advance on both sides.

The Pipeline Is Ready

We'll end where we began. Chapter 1 asked: *given a molecule, what is its ground-state energy?* Twenty-one chapters later, we have a complete, tested, open-source pipeline that answers the question — for any fermionic or bosonic system, with five encoding options, symmetry-based tapering, first and second-order Trotterization, and export to every major quantum platform.

The pipeline runs on a laptop for small molecules (H_2, LiH). It produces circuits that near-term hardware can attempt for medium molecules (H_2O, N_2). And it generates the circuits that fault-tolerant hardware will need for the molecules that actually matter (FeMo-co, transition-metal catalysts, photochemical systems).

The H_2 dissociation curve in Chapter 17 showed the pipeline's correctness. The water bond angle scan in Chapter 18 showed its predictive power — we computed a molecular geometry from first principles. The same machinery, with bigger integrals and more qubits, will someday compute the geometry of a catalyst, the spectrum of a photoactive molecule, or the mechanism of nitrogen fixation.

The quantum computer isn't ready yet. The pipeline is.

Key Takeaways

- **Circuit optimisation**, **error mitigation**, and **adaptive ansätze** build directly on FockMap's output.
- **Bosonic simulation** extends the same pipeline to electron-phonon systems, vibrations, and photochemistry.
- **Lattice models** use the same operator algebra and encoding infrastructure.
- **Quantum error correction** uses the same stabiliser theory as tapering — different intent, same mathematics.
- The pipeline works at every scale. The bottleneck is hardware, not software.

Previous: Chapter 21 — Scaling: From H_2 to FeMo-co

Back to: Table of Contents

Appendix A: Library Cookbook Reference

A quick-reference guide to every public type and function in FockMap.

This appendix summarizes the 15-chapter Cookbook that accompanies the main text. For full worked examples, see the Cookbook on the companion website.

Type System at a Glance

Type	What it represents	Chapter
`Pauli`	Single-qubit operator: I, X, Y, Z	1
`Phase`	Exact phase factor: P1 (+1), M1 (-1), Pi (+i), Mi (-i)	1
`C<'T>`	Coefficient × single operator	2
`P<'T>`	Ordered product of operators	2
`S<'T>`	Sum of products (Hamiltonian shape)	2
`IxOp<'idx,'op>`	Operator tagged with a mode index	3
`LadderOperatorUnit`	Raise / Lower / Identity	4
`PauliRegister`	Fixed-width Pauli string with coefficient	1, 6
`PauliRegisterSequence`	Sum of Pauli strings (encoding output)	1, 6
`EncodingScheme`	Three index-set functions → custom encoding	8
`EncodingTree`	Tree shape for tree-based encodings	9

Type	What it represents	Chapter
`FenwickTree<'a>`	Immutable binary indexed tree	9
`SymplecticVector`	Binary Pauli representation for tapering	15
`CliffordGate`	Had / Sgate / CNOT — elementary gates	15
`Z2TaperingResult`	Tapering result (v1)	15
`TaperingResult`	Tapering result (v2, with Clifford)	15

Key Functions

Encoding

Function	Signature	What it does
`jordanWignerTerms`	`LadderOperatorUnit → uint32 → uint32 → PauliRegisterSequence`	JW encoding
`bravyiKitaevTerms`	same	BK encoding
`parityTerms`	same	Parity encoding
`balancedBinaryTreeTerms`	same	Binary tree encoding
`ternaryTreeTerms`	same	Ternary tree encoding
`encodeOperator`	`EncodingScheme → ...`	Custom encoding

Hamiltonian Construction

Function	What it does
`computeHamiltonianWith`	Build Hamiltonian with any encoding
`computeHamiltonianWithParallel`	Parallel version
`computeHamiltonianSkeleton`	Pre-compute Pauli structure
`applyCoefficients`	Dress skeleton with integrals

Tapering

Function	What it does
`diagonalZ2SymmetryQubits`	Find diagonal Z_2 qubits
`taperDiagonalZ2`	Apply diagonal tapering
`findCommutingGenerators`	Find all Z_2 symmetries
`taper`	Unified tapering pipeline

Normal Ordering

Function	What it does
`ConstructNormalOrdered`	Fermionic (CAR) normal ordering
`constructBosonicNormalOrdered`	Bosonic (CCR) normal ordering
`constructMixedNormalOrdered`	Mixed fermion-boson ordering

Appendix B: Mathematical Background

Formal derivations and proofs for readers who want the full treatment.

This appendix summarizes the 7-chapter Theory section that accompanies the main text. For full derivations, see the Theory pages on the companion website.

B.1 Why Encodings? (Theory Ch. 1)

Fermions obey anti-commutation relations: $\{a_i, a_j^\dagger\} = \delta_{ij}$. Qubits obey commutation relations: $[\sigma_i, \sigma_j] = 0$ for $i \neq j$. An encoding is a mapping from the first algebra to the second that preserves the anti-commutation structure.

Key result: Any valid encoding must map each fermionic mode to at least one qubit, so n modes require $\geq n$ qubits. The challenge is to minimize the Pauli weight (number of non-identity Pauli operators) per encoded operator.

B.2 Second Quantization (Theory Ch. 2)

The occupation-number representation replaces N-particle wavefunctions with Fock states $|n_0, n_1, ...\rangle$ and ladder operators $a_p^\dagger, a_p$. The canonical anti-commutation relations (CAR):

$$\{a_p, a_q^\dagger\} = \delta_{pq}, \quad \{a_p, a_q\} = \{a_p^\dagger, a_q^\dagger\} = 0$$

encode the Pauli exclusion principle and fermionic exchange symmetry.

The electronic Hamiltonian in second quantization:

$$\hat{H} = \sum_{pq} h_{pq}\, a_p^\dagger a_q + \frac{1}{2} \sum_{pqrs} \langle pq \mid rs \rangle\, a_p^\dagger a_q^\dagger a_s a_r$$

B.3 Pauli Algebra (Theory Ch. 3)

The single-qubit Pauli matrices $\{I, X, Y, Z\}$ form a basis for all 2×2 Hermitian operators. Their multiplication table is closed with phases in $\{\pm 1, \pm i\}$:

$$XY = iZ, \quad YZ = iX, \quad ZX = iY$$

Any two distinct non-identity Paulis anti-commute: $XY = -YX$, etc.

An n-qubit Pauli string $P = P_0 \otimes P_1 \otimes \cdots \otimes P_{n-1}$ is a tensor product of single-qubit Paulis. The set of all 4^n Pauli strings forms a group under multiplication (the Pauli group).

B.4 Jordan–Wigner Transform (Theory Ch. 4)

The Jordan–Wigner encoding maps:

$$a_j^\dagger \to \frac{1}{2}(X_j - iY_j) \bigotimes_{k<j} Z_k$$

The Z-chain $\bigotimes_{k<j} Z_k$ tracks the parity of occupied orbitals below j, enforcing the fermionic sign convention. The chain length grows linearly with j, giving worst-case Pauli weight $O(n)$.

B.5 Beyond Jordan–Wigner (Theory Ch. 5)

Bravyi–Kitaev: Uses a Fenwick tree to pre-compute partial parity sums, reducing the chain length to $O(\log_2 n)$.

Parity encoding: Swaps the roles of occupation and parity information, placing the total parity on the last qubit (always taperable).

Tree-based encodings: Any rooted labelled ternary tree defines a valid encoding, with each root-to-leaf path contributing Pauli labels. Balanced ternary trees achieve $O(\log_3 n)$ worst-case weight — the best known scaling.

The index-set framework (Seeley–Richard–Love 2012): Unifies JW, BK, and Parity as instances of a common abstraction defined by three set-valued functions: Update(j), Parity(j), Occupation(j).

B.6 Bosonic Operators (Theory Ch. 6)

Bosonic modes obey canonical commutation relations (CCR): $[b_i, b_j^\dagger] = \delta_{ij}$.

Bosonic Fock spaces are infinite-dimensional and must be truncated to d levels for qubit encoding. Three truncation encodings:

- **Unary:** d qubits, weight ≤ 2
- **Standard binary:** $\lceil \log_2 d \rceil$ qubits
- **Gray code:** $\lceil \log_2 d \rceil$ qubits, reduced average weight

B.7 Mixed Systems (Theory Ch. 7)

For Hamiltonians with both fermionic and bosonic modes: - Separate operators by sector (fermionic vs bosonic) - Cross-sector swaps are free (no sign change: $[a_f, b_b^\dagger] = 0$) - Normal-order each sector independently using the appropriate algebra - Encode fermionic sector with JW/BK/etc.; encode bosonic sector with unary/binary/Gray